

CS1050 -  
Computer Organization and Digital Design

# **NANOPROCESSOR**

Group - 23

## **Group members**

| <b>Name</b>              | <b>Index no</b> |
|--------------------------|-----------------|
| Jayathilake P. I .K.     | 240288D         |
| Jayawardana Y. G. L. S.  | 240294R         |
| Jayawardhana J. A. R. P. | 240297E         |
| Kalpashaka N. G. L. G.   | 240312A         |

# Table of Content

|                                                      |    |
|------------------------------------------------------|----|
| 1. Objective.....                                    | 6  |
| 2. Introduction.....                                 | 6  |
| 3. Instruction Set.....                              | 7  |
| Optimized Version.....                               | 7  |
| Extended Version.....                                | 7  |
| 4. High-level Diagram - Original Nano Processor..... | 9  |
| 5. Top Module.....                                   | 10 |
| Optimized Version.....                               | 10 |
| Design Source File.....                              | 10 |
| Elaborated Design Schematic.....                     | 16 |
| Simulation Source File.....                          | 16 |
| Timing Diagram.....                                  | 18 |
| Extended Version.....                                | 18 |
| Design Source File.....                              | 18 |
| Elaborated Design Schematic.....                     | 23 |
| Simulation Source File.....                          | 23 |
| Timing Diagram.....                                  | 25 |
| 6. Core Components - Original Nano Processor.....    | 26 |
| 6.1. Instruction Decoder.....                        | 26 |
| Optimized Version.....                               | 26 |
| Design Source File.....                              | 26 |
| Elaborated Design Schematic.....                     | 28 |
| Simulation Source File.....                          | 29 |
| Timing Diagram.....                                  | 31 |
| Extended Version.....                                | 31 |
| Design Source File.....                              | 31 |
| Elaborated Design Schematic.....                     | 34 |
| Simulation Source File.....                          | 35 |
| Timing Diagram.....                                  | 37 |
| 6.2. Register Bank.....                              | 37 |
| Design Source File.....                              | 37 |
| Elaborated Design Schematic.....                     | 39 |
| Simulation Source File.....                          | 39 |
| Timing Diagram.....                                  | 41 |
| 6.3. Program Counter.....                            | 42 |

|                                     |    |
|-------------------------------------|----|
| Design Source File.....             | 42 |
| Elaborated Design Schematic.....    | 43 |
| Simulation Source File.....         | 44 |
| Timing Diagram.....                 | 46 |
| 6.4. ALU.....                       | 46 |
| Optimized Version.....              | 47 |
| Design Source File.....             | 47 |
| Elaborated Design Schematic.....    | 48 |
| Simulation Source File.....         | 49 |
| Timing Diagram.....                 | 51 |
| Extended Version.....               | 51 |
| Design Source File.....             | 51 |
| Elaborated Design Schematic.....    | 54 |
| Simulation Source File.....         | 55 |
| Timing Diagram.....                 | 57 |
| New Components.....                 | 57 |
| Comparator.....                     | 57 |
| Design Source File.....             | 57 |
| Elaborated Design Schematic.....    | 58 |
| Logic Unit (Binary Operations)..... | 59 |
| Design Source File.....             | 59 |
| Elaborated Design Schematic.....    | 60 |
| 6.5. Seven Segment.....             | 60 |
| Optimized Version.....              | 60 |
| Design Source File.....             | 60 |
| Elaborated Design Schematic.....    | 62 |
| Simulation Source File.....         | 62 |
| Timing Diagram.....                 | 63 |
| Extended Version.....               | 64 |
| Design Source File.....             | 64 |
| 6.6. Program ROM.....               | 66 |
| Optimized Version.....              | 66 |
| Design Source File.....             | 66 |
| Elaborated Design Schematic.....    | 67 |
| Extended Version.....               | 67 |
| Design Source File.....             | 67 |
| 6.7. Slow Clock.....                | 68 |
| Design Source File.....             | 68 |
| Elaborated Design Schematic.....    | 69 |
| Simulation Source File.....         | 69 |
| Timing Diagram.....                 | 70 |

|                                    |    |
|------------------------------------|----|
| 7. Building Blocks.....            | 71 |
| 1. 7.1. Half Adder.....            | 71 |
| Design Source File.....            | 71 |
| Elaborated Design Schematic.....   | 72 |
| Simulation Source File.....        | 72 |
| Timing Diagram.....                | 73 |
| b. Full Adder.....                 | 74 |
| Design Source File.....            | 74 |
| Elaborated Design Schematic.....   | 75 |
| Simulation Source File.....        | 75 |
| Timing Diagram.....                | 77 |
| c. Ripple Carry Adder (4-bit)..... | 77 |
| Design Source File.....            | 77 |
| Elaborated Design Schematic.....   | 78 |
| Simulation Source File.....        | 79 |
| Timing Diagram.....                | 80 |
| d. Adder (3-bit).....              | 80 |
| Design Source File.....            | 80 |
| Elaborated Design Schematic.....   | 81 |
| Simulation Source File.....        | 81 |
| Timing Diagram.....                | 82 |
| e. Decoder (3 to 8).....           | 83 |
| Design Source File.....            | 83 |
| Elaborated Design Schematic.....   | 84 |
| Simulation Source File.....        | 84 |
| Timing Diagram.....                | 86 |
| f. Multiplexer (2 Way 3-bit).....  | 86 |
| Design Source Code.....            | 86 |
| Elaborated Design Schematic.....   | 87 |
| Simulation Source File.....        | 87 |
| Timing Diagram.....                | 89 |
| g. Multiplexer (8-way 4-bit).....  | 89 |
| Design Source Code.....            | 89 |
| Elaborated Design Schematic.....   | 90 |
| Simulation Source File.....        | 90 |
| Timing Diagram.....                | 92 |
| h. Multiplexer (2 Way 4-bit).....  | 93 |
| Design Source Code.....            | 93 |
| Elaborated Design Schematic.....   | 93 |
| Simulation Source File.....        | 94 |
| Timing Diagram.....                | 95 |

|                                  |     |
|----------------------------------|-----|
| i.Register (4-bit).....          | 96  |
| Design Source File.....          | 96  |
| Elaborated Design Schematic..... | 97  |
| Simulation Source File.....      | 97  |
| Timing Diagram.....              | 99  |
| j.PC Register (3-bit).....       | 99  |
| Design Source File.....          | 99  |
| Elaborated design Schematic..... | 101 |
| Simulation Source File.....      | 101 |
| Timing Diagram.....              | 104 |
| k.Flags.....                     | 104 |
| Optimized Version.....           | 105 |
| Design Source File.....          | 105 |
| Elaborated Schematic Design..... | 106 |
| Extended Version.....            | 106 |
| Design Source File.....          | 106 |
| Elaborated Schematic Design..... | 108 |
| 8.Constraints File.....          | 109 |
| Optimized Version.....           | 109 |
| Extended Version.....            | 110 |
| 9.Resource Utilization.....      | 112 |
| Optimized Version.....           | 112 |
| Extended Version.....            | 113 |
| 10.Conclusion.....               | 113 |
| 11.Team Contribution.....        | 114 |

# 1. Objective

The primary objective of this project is the design and development of a 4-bit microprocessor, capable of executing a specific set of instructions. This task involves creation of several fundamental digital components. We implemented the processor in **Vivado**, verified functionality through simulations and conducted final testing on the **Basys 3** development board.

# 2. Introduction

As assigned in the lab task, we built two versions of the nanoprocessor, an optimized version and an extended version.

The optimized version was designed with a primary focus on resource efficiency, aiming to implement a functional processor using a minimum number of basic logic gates. It supports a core set of four instructions: `ADD`, `NEG`, `MOVI` and `JZR` with a 12-bit instruction width and a 2-bit opcode.

The extended version significantly increases the processor's capabilities by expanding the instruction set to 11 operations, including new logical functions like `AND`, `OR`, `XOR` and `NOT`. To support these features, the instruction width was increased to 14-bits with a 4-bit opcode and the architecture was updated to include a dedicated `top_alu` module.

- **Instruction Decoder:** The instruction width was increased from 12-bits to 14-bits to support a larger 4-bit opcode, expanding the instruction set from 4 to 11 operations.
- **ALU (Arithmetic Logic Unit):** A new `top_alu` module was introduced to house the original adder/subtractor alongside new logic and comparator units, supporting operations like `OR`, `AND`, `XOR`, `NOT` and unsigned/signed comparisons.
- **Program ROM:** The data width was expanded to 14 bits to support the new instruction format, while maintaining the same depth of 8 memory locations.

- **Flag Handling:** The zero flag logic was moved to the top\_alu multiplexer, and new flags for `Greater Than (GT)`, `Equal (EQ)` and `Less Than (LT)` were added to support comparison operations.

### 3. Instruction Set

#### Optimized Version

| Instruction | Description                                                                                                                                                         | Format(12-bit instruction)    |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| MOVI R, d   | Move immediate value d to register R, i.e., $R \leftarrow d$<br>$R \in [0, 7], d \in [0, 15]$                                                                       | 1 0 R R R 0 0 0 d d d d       |
| ADD Ra, Rb  | Add values in registers Ra and Rb and store the result in Ra, i.e., $Ra \leftarrow Ra + Rb$                                                                         | 0 0 Ra Ra Ra Rb Rb Rb 0 0 0 0 |
| NEG R       | 2's complement of registers R, i.e., $R \leftarrow (-R)$<br>$R \in [0, 7]$                                                                                          | 0 1 R R R 0 0 0 0 0 0 0       |
| JZR R, d    | Jump if value in register R is 0, i.e.,<br><i>If</i> $R == 0$<br>$PC \leftarrow d$ ;<br><i>Else</i><br>$PC \leftarrow PC + 1$ ;<br><br>$R \in [0, 7], d \in [0, 7]$ | 1 1 R R R 0 0 0 0 d d d       |

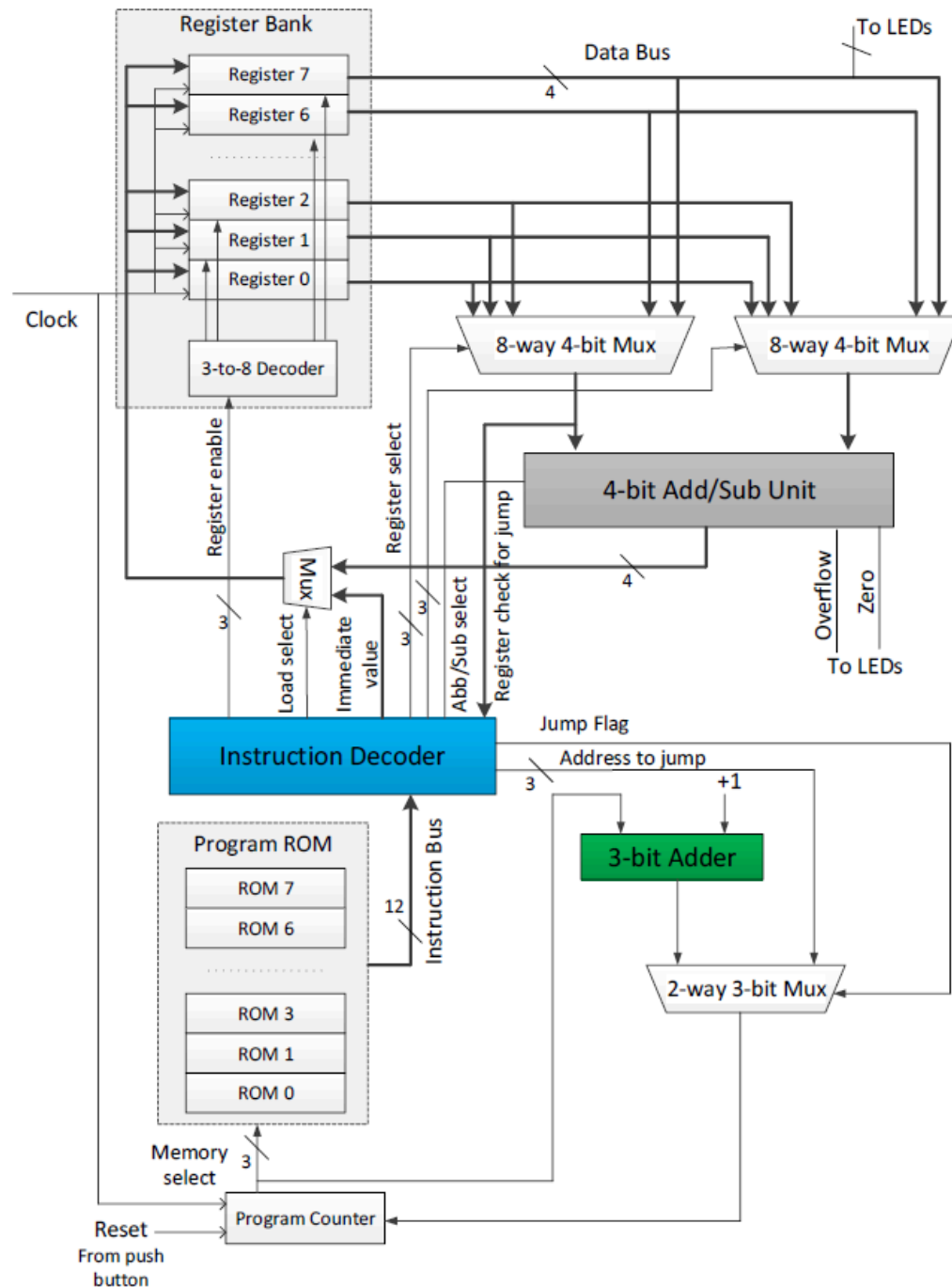
## Extended Version

| Instruction | Description                                                                                                                                                                                  | Format(14-bit instruction)           |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| MOVI R, d   | Move immediate value d to register R,<br>$R \leftarrow d$                                                                                                                                    | 1 0 0 0 R R R 0 0 0 d d d d          |
| NEG R       | 2's complement of register R,<br>$R \leftarrow (-R)$<br><br>$R \in [0, 7]$                                                                                                                   | 1 0 0 1 R R R 0 0 0 0 0 0 0          |
| JZR R, d    | Jump if value in register R is 0,<br><i>If <math>R == 0</math> then <math>PC \leftarrow d</math>;</i><br><i>Else <math>PC \leftarrow PC + 1</math>;</i><br><br>$R \in [0, 7], d \in [-8, 7]$ | 1 0 1 0 R R R 0 0 0 d d d d          |
| ADD Ra, Rb  | Add values in registers Ra and Rb<br>and store the result in Ra,<br>$Ra \leftarrow Ra + Rb$                                                                                                  | 0 0 0 0 Ra Ra Ra Rb Rb Rb 0 0<br>0 0 |
| SUB Ra, Rb  | Subtract Rb from Ra and store<br>result in Ra,<br>$Ra \leftarrow Ra - Rb$                                                                                                                    | 0 0 0 1 Ra Ra Ra Rb Rb Rb 0 0<br>0 0 |
| CMPU Ra, Rb | Unsigned comparison of Ra and Rb<br>(updates flags only)                                                                                                                                     | 0 0 1 0 Ra Ra Ra Rb Rb Rb 0 0<br>0 0 |
| CMP Ra, Rb  | Signed comparison of Ra and Rb<br>(updates flags only)                                                                                                                                       | 0 0 1 1 Ra Ra Ra Rb Rb Rb 0 0<br>0 0 |



|            |                                                                                  |                                      |
|------------|----------------------------------------------------------------------------------|--------------------------------------|
| OR Ra, Rb  | Bitwise OR of Ra and Rb and store result in Ra,<br>$Ra \leftarrow Ra \vee Rb$    | 0 1 0 0 Ra Ra Ra Rb Rb Rb 0 0<br>0 0 |
| AND Ra, Rb | Bitwise AND of Ra and Rb and store result in Ra,<br>$Ra \leftarrow Ra \wedge Rb$ | 0 1 0 1 Ra Ra Ra Rb Rb Rb 0 0<br>0 0 |
| XOR Ra, Rb | Bitwise XOR of Ra and Rb and store result in Ra,<br>$Ra \leftarrow Ra \oplus Rb$ | 0 1 1 0 Ra Ra Ra Rb Rb Rb 0 0<br>0 0 |
| NOT Ra, Rb | Bitwise NOT of Ra and store result in Ra,<br>$Ra \leftarrow \sim Ra$             | 0 1 1 1 Ra Ra Ra Rb Rb Rb 0 0<br>0 0 |

## 4. High-level Diagram - Original Nano Processor



## 5. Top Module

### Optimized Version

#### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Nanoprocessor is
  Port (
    Clk_100MHz : in  STD_LOGIC;
    reset      : in  std_logic;
    result      : out std_logic_vector(3 downto 0); -- R7
    zero        : out std_logic;
    overflow    : out std_logic
  );
end Nanoprocessor;

architecture Behavioral of Nanoprocessor is

  component program_rom
  Port (
    addr : in  std_logic_vector(2 downto 0);
    data : out std_logic_vector(11 downto 0)
  );
end component;

  component instruction_decoder
  Port (
    instruction      : in  STD_LOGIC_VECTOR (11 downto 0);
    zero_flag        : in  STD_LOGIC;
    reg_write_en     : out STD_LOGIC;
    reg_write_sel    : out STD_LOGIC_VECTOR (2 downto 0);
    read_sel_a       : out STD_LOGIC_VECTOR (2 downto 0);
    read_sel_b       : out STD_LOGIC_VECTOR (2 downto 0);
    alu_sub          : out STD_LOGIC;
    wb_sel_imm       : out STD_LOGIC;
    pc_load          : out STD_LOGIC
  );
end component;

```

```

component ADD_SUB_4bit
  Port (
    A      : in STD_LOGIC_VECTOR (3 downto 0);
    B      : in STD_LOGIC_VECTOR (3 downto 0);
    alu_sel: in STD_LOGIC;
    Y      : out STD_LOGIC_VECTOR (3 downto 0);
    carry  : out STD_LOGIC;
    overflow : out STD_LOGIC;
    zero   : out STD_LOGIC
  );
end component;

component flags
  Port (
    clk      : in  STD_LOGIC;
    enable   : in  STD_LOGIC;
    reset    : in  STD_LOGIC;
    flag_en  : in  STD_LOGIC;
    alu_zero : in  STD_LOGIC;
    alu_overflow : in STD_LOGIC;
    Z        : out STD_LOGIC;
    V        : out STD_LOGIC
  );
end component;

component register_bank
  port(
    clk, reset , enable : in std_logic;
    write_en           : in std_logic;
    write_sel          : in std_logic_vector(2 downto 0);
    write_data         : in std_logic_vector(3 downto 0);
    R0,R1,R2,R3,R4,R5,R6,R7 : out std_logic_vector (3 downto
0)
  );
end component;

component mux2_4bit
  Port (
    sel : in STD_LOGIC;
    A   : in STD_LOGIC_VECTOR (3 downto 0);
    B   : in STD_LOGIC_VECTOR (3 downto 0);
    Y   : out STD_LOGIC_VECTOR (3 downto 0)
  );
end component;

```

```

component mux8_4bit
  port(
    sel : in std_logic_vector(2 downto 0);
    I0, I1, I2, I3, I4, I5, I6, I7 : in std_logic_vector(3
downto 0);
    Y : out std_logic_vector(3 downto 0)
  );
end component;

component slow_clock
  port ( clk_in : in std_logic;
         enable : out std_logic );
end component;

component pc
  Port (
    clk      : in STD_LOGIC;
    enable   : in STD_LOGIC;
    reset    : in STD_LOGIC;
    pc_load  : in STD_LOGIC;
    jump_addr : in STD_LOGIC_VECTOR (2 downto 0);
    pc_out   : out STD_LOGIC_VECTOR (2 downto 0)
  );
end component;

-- internal signals
signal pc_addr      : STD_LOGIC_VECTOR(2 downto 0);
signal instruction  : STD_LOGIC_VECTOR(11 downto 0);
signal jump_addr    : STD_LOGIC_VECTOR(2 downto 0);
signal imm_val      : STD_LOGIC_VECTOR(3 downto 0);

signal R0, R1, R2, R3, R4, R5, R6, R7 : STD_LOGIC_VECTOR(3 downto
0);

signal reg_write_en   : STD_LOGIC;
signal reg_write_sel  : STD_LOGIC_VECTOR(2 downto 0);
signal reg_write_data : STD_LOGIC_VECTOR(3 downto 0);

signal alu_A, alu_B   : STD_LOGIC_VECTOR(3 downto 0);
signal alu_Y          : STD_LOGIC_VECTOR(3 downto 0);
signal alu_carry      : STD_LOGIC;
signal alu_overflow   : STD_LOGIC;
signal alu_zero       : STD_LOGIC;
signal alu_active     : STD_LOGIC;

```

```

signal read_sel_a      : STD_LOGIC_VECTOR(2 downto 0);
signal read_sel_b      : STD_LOGIC_VECTOR(2 downto 0);
signal alu_sub          : STD_LOGIC;
signal wb_sel_imm       : STD_LOGIC;
signal pc_load          : STD_LOGIC;

signal Z_flag          : STD_LOGIC;
signal V_flag          : STD_LOGIC;
signal flag_en         : STD_LOGIC;
signal jzr_reg_value   : STD_LOGIC_VECTOR(3 downto 0);
signal jzr_zero        : STD_LOGIC;

signal enable : std_logic := '0';

begin
  -- immediate and jump address extraction
  imm_val  <= instruction(3 downto 0);
  jump_addr <= instruction(2 downto 0);

  slow_clk : slow_clock
    port map (
      clk_in => Clk_100MHz,
      enable => enable );

  -- Program ROM
  u_rom: program_rom
    port map (
      addr => pc_addr,
      data => instruction
    );

  -- Instruction Decoder
  u_decoder: instruction_decoder
    port map (
      instruction      => instruction,
      zero_flag        => jzr_zero,
      reg_write_en     => reg_write_en,
      reg_write_sel    => reg_write_sel,
      read_sel_a       => read_sel_a,
      read_sel_b       => read_sel_b,
      alu_sub          => alu_sub,
      wb_sel_imm       => wb_sel_imm,
      pc_load          => pc_load
    );

```

```

-- Program Counter
u_pc: pc
    port map (
        clk        => Clk_100MHz,
        enable     => enable,
        reset      => reset,
        pc_load    => pc_load,
        jump_addr  => jump_addr,
        pc_out     => pc_addr
    );

-- flags
u_flags : flags
    port map (
        clk        => Clk_100MHz,
        enable     => enable,
        reset      => reset,
        flag_en    => flag_en,
        alu_zero   => alu_zero,
        alu_overflow => alu_overflow,
        Z          => Z_flag,
        V          => V_flag
    );

-- Register Bank
u_regbank: register_bank
    port map (
        clk        => Clk_100MHz,
        enable     => enable,
        reset      => reset,
        write_en   => reg_write_en,
        write_sel  => reg_write_sel,
        write_data => reg_write_data,
        R0         => R0,
        R1         => R1,
        R2         => R2,
        R3         => R3,
        R4         => R4,
        R5         => R5,
        R6         => R6,
        R7         => R7
    );

-- 8-way mux for ALU input A
u_muxA: mux8_4bit

```

```

    port map (
        sel => read_sel_a,
        I0  => R0, I1 => R1, I2 => R2, I3 => R3,
        I4  => R4, I5 => R5, I6 => R6, I7 => R7,
        Y   => alu_A
    );

-- 8-way mux for ALU input B
u_muxB: mux8_4bit
    port map (
        sel => read_sel_b,
        I0  => R0, I1 => R1, I2 => R2, I3 => R3,
        I4  => R4, I5 => R5, I6 => R6, I7 => R7,
        Y   => alu_B
    );

-- Add/Subtract Unit
u_alu: ADD_SUB_4bit
    port map (
        A      => alu_A,
        B      => alu_B,
        alu_sel => alu_sub,
        Y      => alu_Y,
        carry   => alu_carry,
        overflow => alu_overflow,
        zero    => alu_zero
    );

-- 2-way mux for writeback selection (ALU result vs immediate)
u_wb_mux: mux2_4bit
    port map (
        sel => wb_sel_imm,
        A   => alu_Y,
        B   => imm_val,
        Y   => reg_write_data
    );

-- Mux to get the register value for JZR zero test
u_mux_jzr: mux8_4bit
    port map (
        sel => instruction(9 downto 7),
        I0  => R0, I1 => R1, I2 => R2, I3 => R3,
        I4  => R4, I5 => R5, I6 => R6, I7 => R7,
        Y   => jzr_reg_value
    );

```



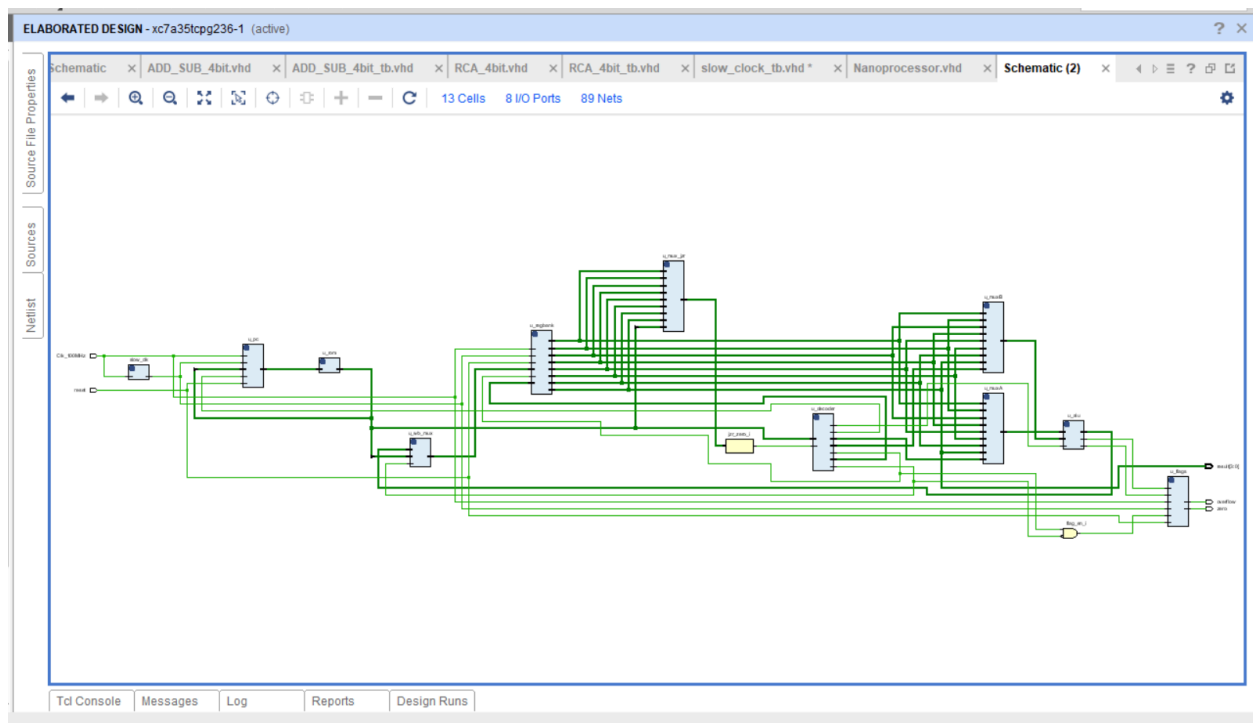
```

jzr_zero <= '1' when jzr_reg_value = "0000" else '0';
flag_en <= reg_write_en and not wb_sel_imm;
result <= R7;
zero    <= Z_flag;
overflow <= V_flag;

```

```
end Behavioral;
```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Nanoprocessor_tb is
-- Port ( );
end Nanoprocessor_tb;

architecture Behavioral of Nanoprocessor_tb is

```

```

component Nanoprocessor
  Port (
    Clk_100MHz : in  std_logic;
    reset      : in  std_logic;
    result     : out std_logic_vector(3 downto 0);
    zero       : out std_logic;
    overflow   : out std_logic
  );
end component;

signal clk      : std_logic := '0';
signal reset    : std_logic := '1';
signal result   : std_logic_vector(3 downto 0);
signal zero     : std_logic;
signal overflow : std_logic;

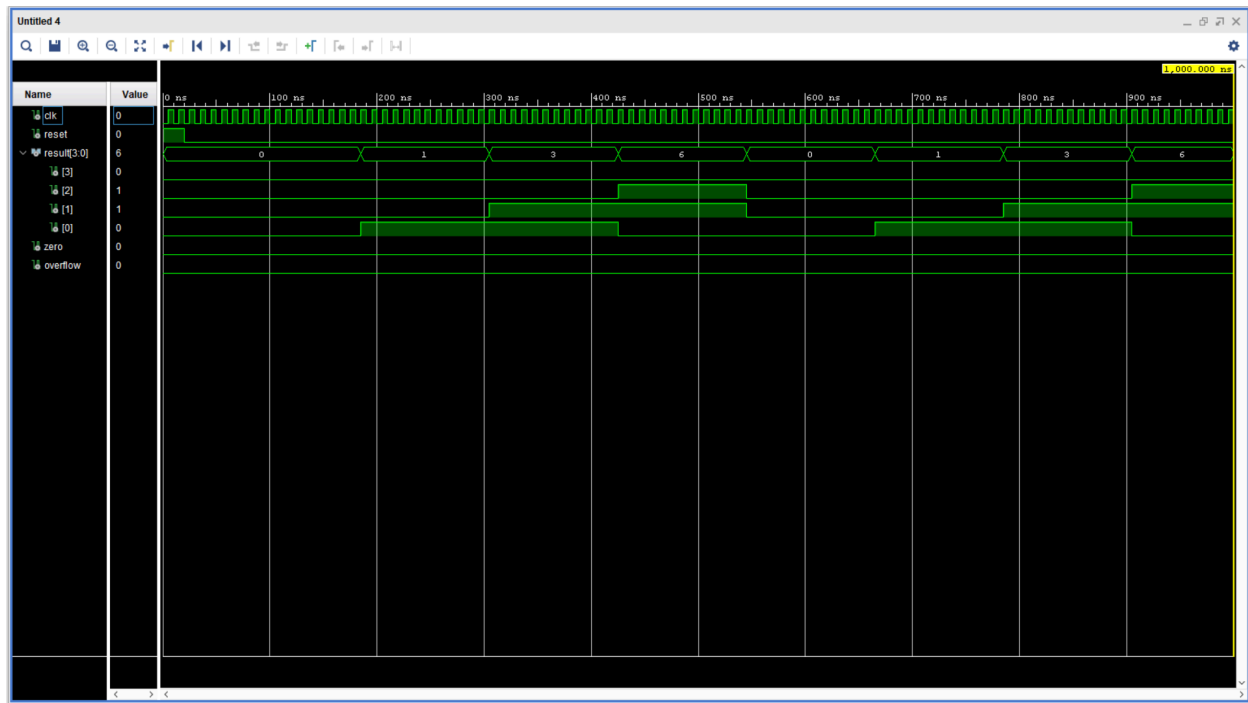
begin
  uut: Nanoprocessor
    port map (
      Clk_100MHz => clk,
      reset      => reset,
      result     => result,
      zero       => zero,
      overflow   => overflow
    );

  clk_process : process
  begin
    while true loop
      clk <= '0'; wait for 5ns;
      clk <= '1'; wait for 5ns;
    end loop;
  end process;

  stim_proc: process
  begin
    reset <= '1';
    wait for 20 ns;      -- hold reset for 2 clock cycles
    reset <= '0';
    wait for 400 us;     -- run program
    report "Simulation finished" severity note;
    wait;
  end process;
end Behavioral;

```

## Timing Diagram



## Extended Version

### Design Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Nanoprocessor_tb is
-- Port ( );
end Nanoprocessor_tb;

architecture Behavioral of Nanoprocessor_tb is

    component Nanoprocessor
        Port (
            Clk_100MHz : in  STD_LOGIC;
            reset       : in  std_logic;
            mode_sel    : in  STD_LOGIC;
            sw_inst      : in  STD_LOGIC_VECTOR(13 downto 0);
```

```

        btn_step      : in  STD_LOGIC;
        result        : out std_logic_vector(3 downto 0);
        zero          : out std_logic;
        overflow       : out std_logic;
        greater        : out std_logic;
        equal          : out std_logic;
        lower          : out std_logic
    );
end component;

signal Clk_100MHz : STD_LOGIC := '0';
signal reset      : STD_LOGIC := '0';
signal mode_sel   : STD_LOGIC := '0';
signal sw_inst     : STD_LOGIC_VECTOR(13 downto 0) := (others =>
'0');
signal btn_step    : STD_LOGIC := '0';
signal result      : STD_LOGIC_VECTOR(3 downto 0);
signal zero        : STD_LOGIC;
signal overflow     : STD_LOGIC;
signal greater      : STD_LOGIC;
signal equal        : STD_LOGIC;
signal lower        : STD_LOGIC;

constant clk_period : time := 10 ns;

begin

    uut: Nanoprocessor
        port map (
            Clk_100MHz => Clk_100MHz,
            reset      => reset,
            mode_sel   => mode_sel,
            sw_inst     => sw_inst,
            btn_step    => btn_step,
            result      => result,
            zero        => zero,
            overflow     => overflow,
            greater      => greater,
            equal        => equal,
            lower        => lower
        );

    -- Clock process
    clk_process : process
    begin

```

```

    while true loop
        Clk_100MHz <= '0';
        wait for clk_period / 2;
        Clk_100MHz <= '1';
        wait for clk_period / 2;
    end loop;
end process;

-- Stimulus process
stim_proc : process
begin
    -- Reset
    reset <= '1';
    wait for 40 ns;
    reset <= '0';
    wait for 20 ns;

    -- Use step mode
    mode_sel <= '1';

    -- 1. MOVI R0, 5 (load 5 into R0)
    sw_inst <= "10000000000101";
    btn_step <= '1';
    wait for clk_period;
    btn_step <= '0';
    wait for 40 ns;

    -- 2. MOVI R1, 3 (load 3 into R1)
    sw_inst <= "10000100000011";
    btn_step <= '1';
    wait for clk_period;
    btn_step <= '0';
    wait for 40 ns;

    -- 3. ADD R0, R1 (R0 = R0 + R1 = 8)
    sw_inst <= "00000000010000";
    btn_step <= '1';
    wait for clk_period;
    btn_step <= '0';
    wait for 40 ns;

    -- 4. SUB R0, R1 (R0 = R0 - R1 = 5)
    sw_inst <= "00010000010000";
    btn_step <= '1';
    wait for clk_period;

```

```

btn_step <= '0';
wait for 40 ns;

-- 5. MOVI R2, 8 (load 8 into R2)
sw_inst <= "100010000001000";
btn_step <= '1';
wait for clk_period;
btn_step <= '0';
wait for 40 ns;

-- 6. AND R0, R2 (bitwise AND)
sw_inst <= "010100000010000";
btn_step <= '1';
wait for clk_period;
btn_step <= '0';
wait for 40 ns;

-- 7. OR R0, R2 (bitwise OR)
sw_inst <= "010000000010000";
btn_step <= '1';
wait for clk_period;
btn_step <= '0';
wait for 40 ns;

-- 8. XOR R0, R2 (bitwise XOR)
sw_inst <= "011000000010000";
btn_step <= '1';
wait for clk_period;
btn_step <= '0';
wait for 40 ns;

-- 9. NOT R0 (bitwise NOT)
sw_inst <= "011100000000000";
btn_step <= '1';
wait for clk_period;
btn_step <= '0';
wait for 40 ns;

-- 10. NEG R0 (two's complement negation)
sw_inst <= "100100000000000";
btn_step <= '1';
wait for clk_period;
btn_step <= '0';
wait for 40 ns;

```

```

-- 11. CMP R0, R1 (compare)
sw_inst <= "00110000010000";
btn_step <= '1';
wait for clk_period;
btn_step <= '0';
wait for 40 ns;

-- 12. MOVI R3, 0 (load zero for jump test)
sw_inst <= "100011000000000";
btn_step <= '1';
wait for clk_period;
btn_step <= '0';
wait for 40 ns;

-- 13. JZR R3, 2 (jump if zero - should jump)
sw_inst <= "101011000000010";
btn_step <= '1';
wait for clk_period;
btn_step <= '0';
wait for 40 ns;

-- End of test
wait;
end process;

end Behavioral;

```





```

        equal  : out std_logic;
        lower  : out std_logic
    );
end component;

signal Clk_100MHz : STD_LOGIC := '0';
signal reset      : STD_LOGIC := '0';
signal mode_sel   : STD_LOGIC := '0';
signal sw_inst    : STD_LOGIC_VECTOR(13 downto 0) := (others =>
'0');
signal btn_step   : STD_LOGIC := '0';
signal result     : STD_LOGIC_VECTOR(3 downto 0);
signal zero       : STD_LOGIC;
signal overflow   : STD_LOGIC;
signal greater, equal, lower : STD_LOGIC;

constant clk_period : time := 10 ns;

begin

    uut: Nanoprocessor
        port map (
            Clk_100MHz => Clk_100MHz,
            reset      => reset,
            mode_sel   => mode_sel,
            sw_inst    => sw_inst,
            btn_step   => btn_step,
            result     => result,
            zero       => zero,
            overflow   => overflow,
            greater    => greater,
            equal      => equal,
            lower      => lower
        );

    clk_process : process
    begin
        while true loop
            Clk_100MHz <= '0';
            wait for clk_period / 2;
            Clk_100MHz <= '1';
            wait for clk_period / 2;
        end loop;
    end process;
end process;
```

```

stim_proc : process
begin
    -- Assert reset for a few cycles
    reset <= '1';
    wait for 40 ns;
    reset <= '0';

    -- Auto run mode (uses internal ROM)
    mode_sel <= '0';
    sw_inst <= (others => '0');
    btn_step <= '0';

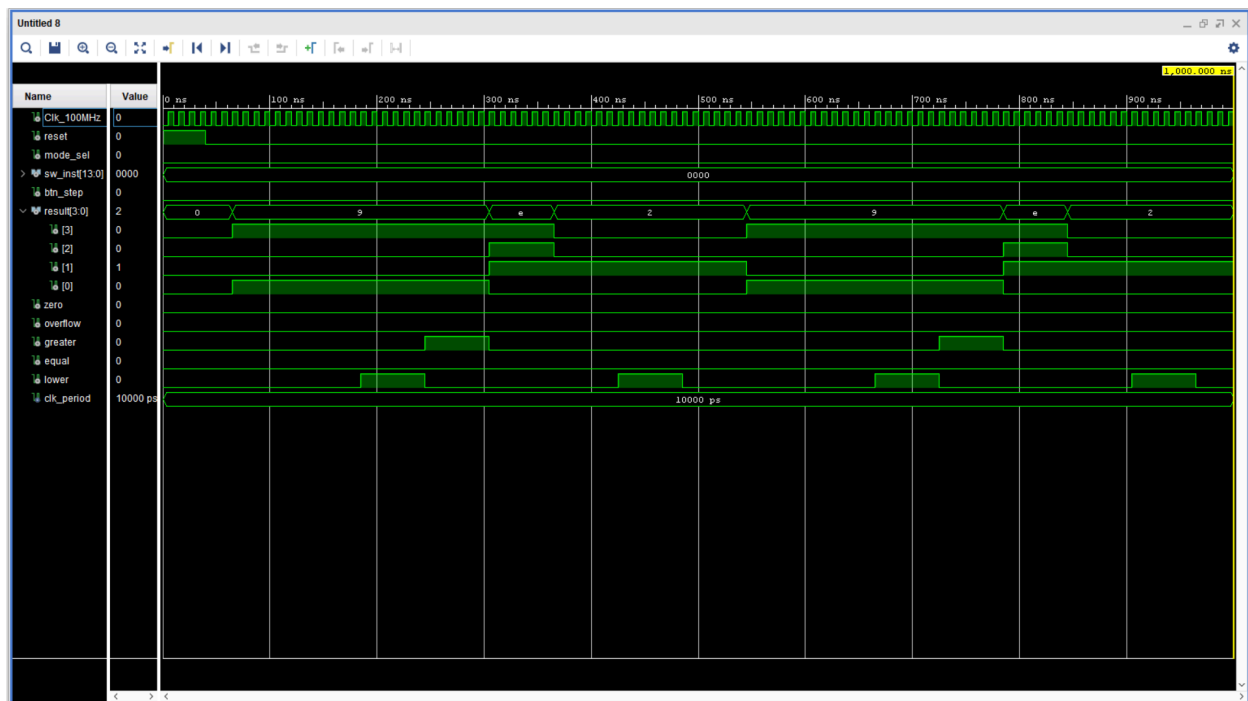
    -- Wait long enough for many instructions
    wait for 5000 ns;

    wait;    -- stop simulation
end process;

```

end Behavioral;

## Timing Diagram



## 6.Core Components - Original Nano Processor

### 6.1. Instruction Decoder

| Item                   | Optimized Version       | Extended Version                                            | Change Type |
|------------------------|-------------------------|-------------------------------------------------------------|-------------|
| Instruction Width      | 12 bits                 | 14 bits                                                     | Increased   |
| Opcode bits            | 2 bits                  | 4 bits                                                      | Increased   |
| ALU control output     | alu_sub (1bit)          | Alu_op (3 bits)                                             | Replaced    |
| Number of instructions | 4 (ADD, NEG, MOVI, JZR) | 11 (ADD, SUB, CMPU, CMP, OR, AND, XOR, NOT, MOVI, NEG, JZR) | Added       |
| Comparison outputs     | None                    | CMPU, CMP handled by ALU                                    | New feature |
| Logic operations       | None                    | OR, AND, XOR, NOT                                           | New feature |
| Default assignments    | Same style              | Same style                                                  | No change   |

### Optimized Version

#### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity instruction_decoder is
    Port ( instruction : in  STD_LOGIC_VECTOR (11 downto 0);
          zero_flag    : in  STD_LOGIC;      -- '1' when the tested
register is zero
          reg_write_en: out STD_LOGIC;
          reg_write_sel: out STD_LOGIC_VECTOR (2 downto 0);
          read_sel_a   : out STD_LOGIC_VECTOR (2 downto 0);
          read_sel_b   : out STD_LOGIC_VECTOR (2 downto 0);
          alu_sub       : out STD_LOGIC;      -- '1' for subtract, '0'
for add
          wb_sel_imm   : out STD_LOGIC;      -- '1' to write immediate,
'0' for ALU result

```

```

        pc_load      : out STD_LOGIC);    -- jump enable to program
counter
end instruction_decoder;

```

architecture Behavioral of instruction\_decoder is

```

signal opcode : std_logic_vector(1 downto 0);
begin
    opcode <= instruction(11 downto 10);

    process(opcode, instruction, zero_flag)
    begin
        -- Defaults (no operation)
        reg_write_en <= '0';
        reg_write_sel <= "000";
        read_sel_a <= "000";
        read_sel_b <= "000";
        alu_sub <= '0';
        wb_sel_imm <= '0';
        pc_load <= '0';

        case opcode is
            when "00" =>    -- ADD Ra, Rb
                read_sel_a <= instruction(9 downto 7); -- Ra
                read_sel_b <= instruction(6 downto 4); -- Rb
                reg_write_en <= '1';
                reg_write_sel <= instruction(9 downto 7); --
destination Ra
                alu_sub <= '0';
                wb_sel_imm <= '0';

            when "01" =>    -- NEG R
                read_sel_a <= "000"; -- R0
(hardwired to 0)
                read_sel_b <= instruction(9 downto 7); -- R
                reg_write_en <= '1';
                reg_write_sel <= instruction(9 downto 7); -- same R
                alu_sub <= '1';
                wb_sel_imm <= '0';

            when "10" =>    -- MOVI R, d
                read_sel_a <= "000";
                read_sel_b <= "000";
                reg_write_en <= '1';

```

```

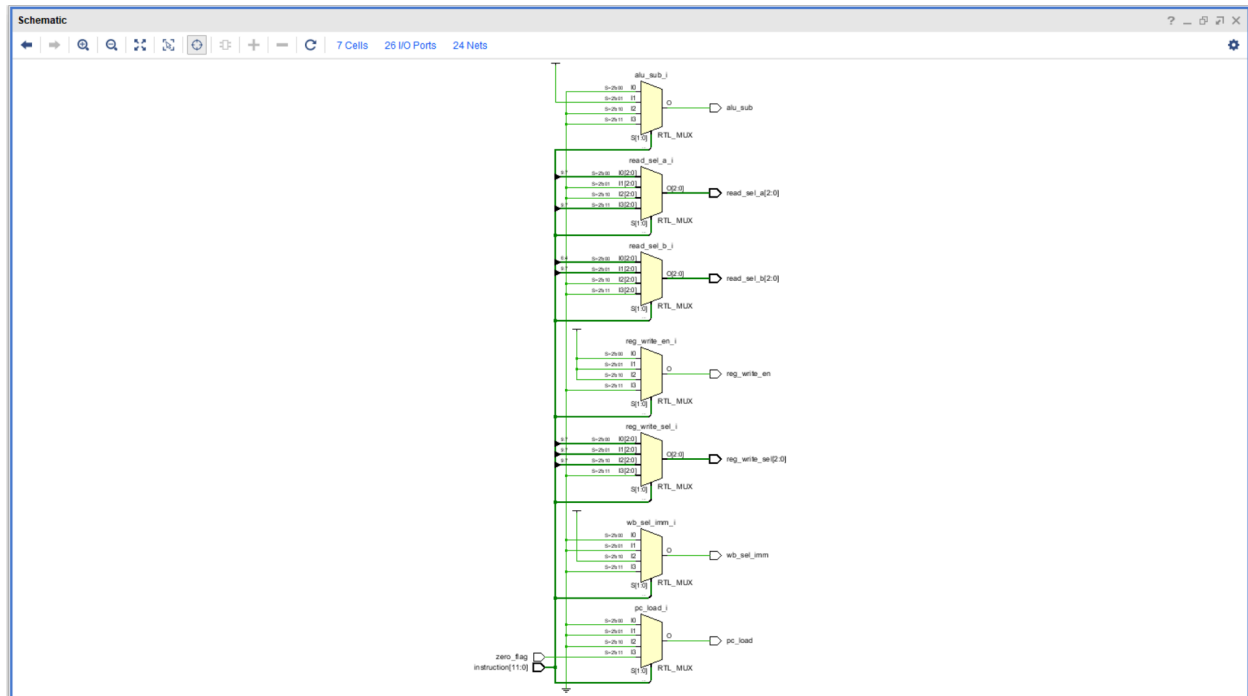
        reg_write_sel <= instruction(9 downto 7); --
destination register
        wb_sel_imm <= '1';                        -- write
immediate value

        when "11" => -- JZR R, d
            read_sel_a <= instruction(9 downto 7); -- register
to test
            read_sel_b <= "000";
            reg_write_en <= '0';
            if zero_flag = '1' then
                pc_load <= '1';
            end if;

        when others =>
            null;
        end case;
    end process;
end Behavioral;

```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity instruction_decoder_tb is
-- Port ( );
end instruction_decoder_tb;

architecture Behavioral of instruction_decoder_tb is

component instruction_decoder
    Port ( instruction : in  STD_LOGIC_VECTOR (11 downto 0);
          zero_flag    : in  STD_LOGIC;
          reg_write_en : out STD_LOGIC;
          reg_write_sel : out STD_LOGIC_VECTOR (2 downto 0);
          read_sel_a    : out STD_LOGIC_VECTOR (2 downto 0);
          read_sel_b    : out STD_LOGIC_VECTOR (2 downto 0);
          alu_sub        : out STD_LOGIC;
          wb_sel_imm     : out STD_LOGIC;
          pc_load        : out STD_LOGIC);
end component;

signal instruction : STD_LOGIC_VECTOR(11 downto 0) := (others => '0');
signal zero_flag   : STD_LOGIC := '0';
signal reg_write_en : STD_LOGIC;
signal reg_write_sel : STD_LOGIC_VECTOR(2 downto 0);
signal read_sel_a    : STD_LOGIC_VECTOR(2 downto 0);
signal read_sel_b    : STD_LOGIC_VECTOR(2 downto 0);
signal alu_sub, wb_sel_imm, pc_load : STD_LOGIC;

begin
    uut: instruction_decoder
        port map (
            instruction => instruction,
            zero_flag   => zero_flag,
            reg_write_en => reg_write_en,
            reg_write_sel=> reg_write_sel,
            read_sel_a   => read_sel_a,
            read_sel_b   => read_sel_b,
            alu_sub      => alu_sub,
            wb_sel_imm   => wb_sel_imm,
            pc_load      => pc_load );
    stim_proc: process

```

```

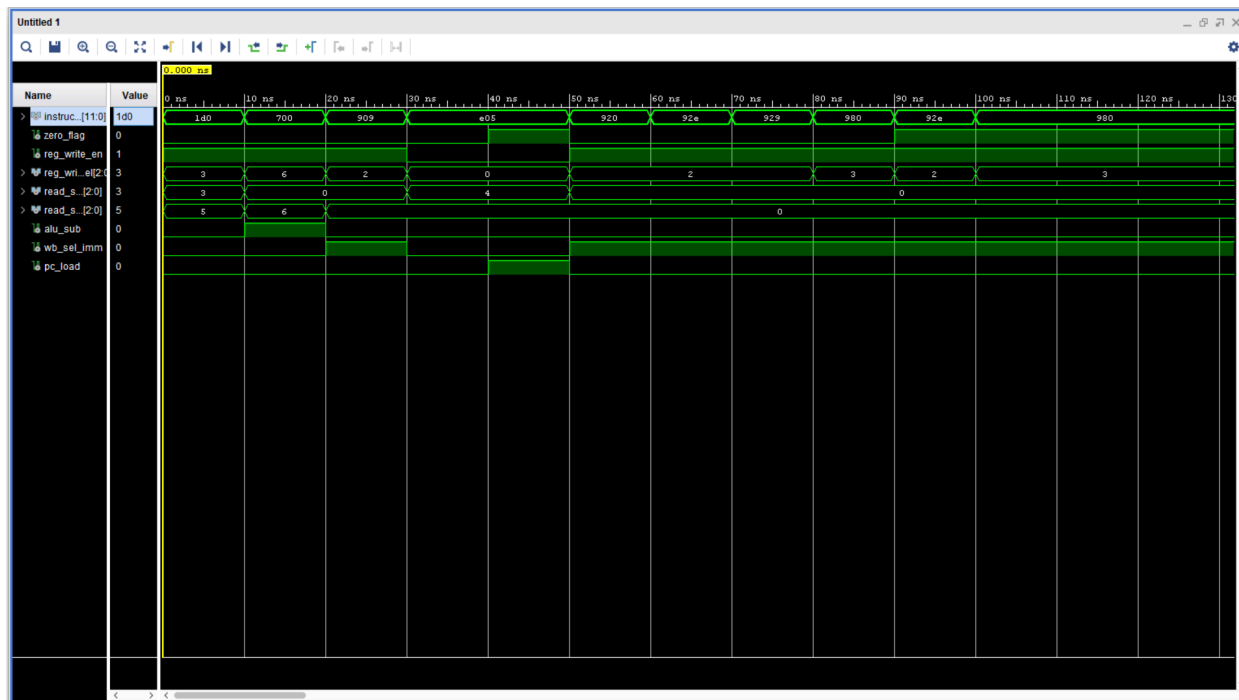
begin
    instruction <= "000111010000"; zero_flag <= '0'; wait for 10 ns;
    instruction <= "011100000000"; zero_flag <= '0'; wait for 10 ns;
    instruction <= "100100001001"; zero_flag <= '0'; wait for 10 ns;
    instruction <= "111000000101"; zero_flag <= '0'; wait for 10 ns;
    instruction <= "111000000101"; zero_flag <= '1'; wait for 10 ns;

    instruction <= "100100100000"; zero_flag <= '0'; wait for 10 ns;
-- 240288 pattern
    instruction <= "100100101110"; zero_flag <= '0'; wait for 10 ns;
-- 240294 pattern
    instruction <= "100100101001"; zero_flag <= '0'; wait for 10 ns;
-- 240297 pattern
    instruction <= "100110000000"; zero_flag <= '0'; wait for 10 ns;
-- 240312 pattern

    -- variation with zero_flag dependency
    instruction <= "100100101110"; zero_flag <= '1'; wait for 10 ns;
    instruction <= "100110000000"; zero_flag <= '1'; wait for 10 ns;
    wait;
end process;
end Behavioral;

```

## Timing Diagram



## Extended Version

### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity instruction_decoder is
    Port ( instruction : in  STD_LOGIC_VECTOR (13 downto 0);
          zero_flag    : in  STD_LOGIC;      -- '1' when the tested
register is zero
          reg_write_en: out STD_LOGIC;
          reg_write_sel: out STD_LOGIC_VECTOR (2 downto 0);
          read_sel_a   : out STD_LOGIC_VECTOR (2 downto 0);
          read_sel_b   : out STD_LOGIC_VECTOR (2 downto 0);
          alu_op       : out STD_LOGIC_VECTOR(2 downto 0);
          wb_sel_imm   : out STD_LOGIC;      -- '1' to write immediate,
'0' for ALU result
          pc_load      : out STD_LOGIC);    -- jump enable to program
counter
end instruction_decoder;

architecture Behavioral of instruction_decoder is

    signal opcode : std_logic_vector(3 downto 0);
begin
    opcode <= instruction(13 downto 10);

    process(opcode, instruction, zero_flag)
    begin
        -- Defaults (no operation)
        reg_write_en <= '0';
        reg_write_sel <= "000";
        read_sel_a <= "000";
        read_sel_b <= "000";
        alu_op <= "000";
        wb_sel_imm <= '0';
        pc_load <= '0';

        case opcode is
            when "0000" => -- ADD Ra, Rb
                read_sel_a <= instruction(9 downto 7); -- Ra
                read_sel_b <= instruction(6 downto 4); -- Rb
                reg_write_en <= '1';
        end case;
    end process;
end Behavioral;

```



```

        reg_write_sel <= instruction(9 downto 7); --
destination Ra
        alu_op <= opcode(2 downto 0);
        wb_sel_imm <= '0';

when "0001" =>    -- SUB Ra, Rb
        read_sel_a <= instruction(9 downto 7); -- Ra
        read_sel_b <= instruction(6 downto 4); -- Rb
        reg_write_en <= '1';
        reg_write_sel <= instruction(9 downto 7); --
destination Ra
        alu_op <= opcode(2 downto 0);
        wb_sel_imm <= '0';

when "0010" =>    -- CMPL Ra, Rb
        read_sel_a <= instruction(9 downto 7);
        read_sel_b <= instruction(6 downto 4);
        reg_write_en <= '0';
        reg_write_sel <= instruction(9 downto 7);
        alu_op <= opcode(2 downto 0);
        wb_sel_imm <= '0';

when "0011" =>    -- CMP Ra, Rb
        read_sel_a <= instruction(9 downto 7);
        read_sel_b <= instruction(6 downto 4);
        reg_write_en <= '0';
        reg_write_sel <= instruction(9 downto 7);
        alu_op <= opcode(2 downto 0);
        wb_sel_imm <= '0';

when "0100" =>    -- OR
        read_sel_a <= instruction(9 downto 7);
        read_sel_b <= instruction(6 downto 4);
        reg_write_en <= '1';
        reg_write_sel <= instruction(9 downto 7);
        alu_op <= opcode(2 downto 0);

when "0101" =>    -- AND
        read_sel_a <= instruction(9 downto 7);
        read_sel_b <= instruction(6 downto 4);
        reg_write_en <= '1';
        reg_write_sel <= instruction(9 downto 7);
        alu_op <= opcode(2 downto 0);

when "0110" =>    -- XOR

```

```

        read_sel_a <= instruction(9 downto 7);
        read_sel_b <= instruction(6 downto 4);
        reg_write_en <= '1';
        reg_write_sel <= instruction(9 downto 7);
        alu_op <= opcode(2 downto 0);

when "0111" =>    -- NOT
    read_sel_a <= instruction(9 downto 7);
    reg_write_en <= '1';
    reg_write_sel <= instruction(9 downto 7);
    alu_op <= opcode(2 downto 0);

when "1000" =>    -- MOVI R, d
    read_sel_a <= "000";
    read_sel_b <= "000";
    reg_write_en <= '1';
    reg_write_sel <= instruction(9 downto 7); --
destination register
    wb_sel_imm <= '1';                        -- write
immediate value

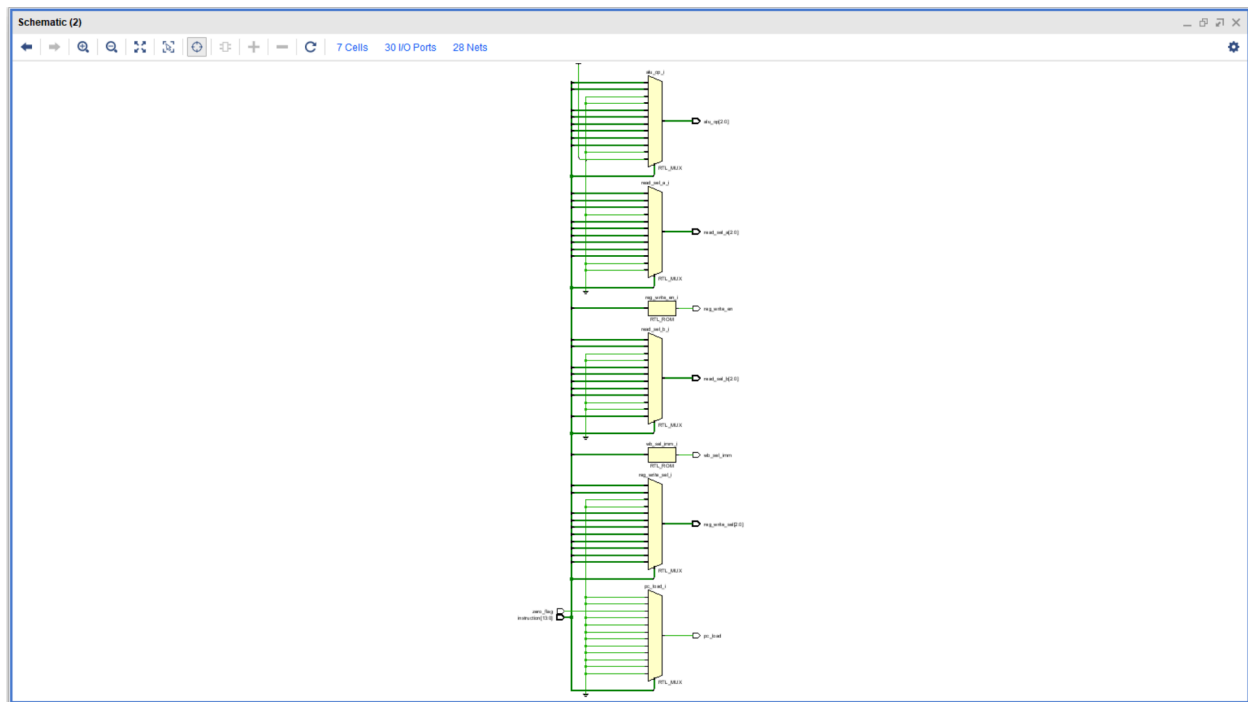
when "1001" =>    -- NEG R
    read_sel_a <= "000";                      -- R0
(hardwired to 0)
    read_sel_b <= instruction(9 downto 7); -- R
    reg_write_en <= '1';
    reg_write_sel <= instruction(9 downto 7); -- same R
    alu_op <= "001";
    wb_sel_imm <= '0';

when "1010" =>    -- JZR R, d
    read_sel_a <= instruction(9 downto 7);    -- register
to test
    read_sel_b <= "000";
    reg_write_en <= '0';
    pc_load <= zero_flag;

when others =>
    null;
end case;
end process;
end Behavioral;

```

## **Elaborated Design Schematic**



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity instruction_decoder_tb is
-- Port ( );
end instruction_decoder_tb;

architecture Behavioral of instruction_decoder_tb is

component instruction_decoder
    Port ( instruction : in  STD_LOGIC_VECTOR (13 downto 0);
          zero_flag   : in  STD_LOGIC;
          reg_write_en: out STD_LOGIC;
          reg_write_sel: out STD_LOGIC_VECTOR (2 downto 0);
          read_sel_a   : out STD_LOGIC_VECTOR (2 downto 0);
          read_sel_b   : out STD_LOGIC_VECTOR (2 downto 0);
          alu_op       : out STD_LOGIC_VECTOR(2 downto 0);
          wb_sel_imm   : out STD_LOGIC;
          pc_load      : out STD_LOGIC);
end component;

```

```

signal instruction : STD_LOGIC_VECTOR(13 downto 0) := (others => '0');
signal zero_flag   : STD_LOGIC := '0';
signal reg_write_en : STD_LOGIC;
signal reg_write_sel : STD_LOGIC_VECTOR(2 downto 0);
signal read_sel_a   : STD_LOGIC_VECTOR(2 downto 0);
signal read_sel_b   : STD_LOGIC_VECTOR(2 downto 0);
signal alu_op       : STD_LOGIC_VECTOR(2 downto 0);
signal wb_sel_imm, pc_load : STD_LOGIC;

```

```
begin
```

```
uut: instruction_decoder
```

```

    port map (
        instruction => instruction,
        zero_flag   => zero_flag,
        reg_write_en => reg_write_en,
        reg_write_sel=> reg_write_sel,
        read_sel_a   => read_sel_a,
        read_sel_b   => read_sel_b,
        alu_op       => alu_op,
        wb_sel_imm   => wb_sel_imm,
        pc_load      => pc_load );

```

```
stim_proc: process
```

```
begin
```

```

    -- Test ADD R1, R2 (opcode 0000, Ra=001, Rb=010)
    instruction <= "00000010010000"; zero_flag <= '0'; wait for 10 ns;

    -- Test SUB R3, R4 (opcode 0001, Ra=011, Rb=100)
    instruction <= "00010110010000"; zero_flag <= '0'; wait for 10 ns;

    -- Test CMPU R5, R6 (opcode 0010, Ra=101, Rb=110)
    instruction <= "00101010110000"; zero_flag <= '0'; wait for 10 ns;

    -- Test CMP R7, R0 (opcode 0011, Ra=111, Rb=000)
    instruction <= "00111110000000"; zero_flag <= '0'; wait for 10 ns;

    -- Test OR R1, R2 (opcode 0100, Ra=001, Rb=010)
    instruction <= "01000010010000"; zero_flag <= '0'; wait for 10 ns;

    -- Test AND R3, R4 (opcode 0101, Ra=011, Rb=100)
    instruction <= "01010110010000"; zero_flag <= '0'; wait for 10 ns;

    -- Test XOR R5, R6 (opcode 0110, Ra=101, Rb=110)
    instruction <= "01101010110000"; zero_flag <= '0'; wait for 10 ns;

```

Untitled 2

0.000 ns

| Name              | Value |
|-------------------|-------|
| >  instruc.[13:0] | 0090  |
| zero_flag         | 0     |
| reg_wrt_en        | 1     |
| >  read_s.[2:0]   | 1     |
| >  read_s.[2:0]   | 1     |
| >  alu_op[2:0]    | 0     |
| wb_sel_imm        | 0     |
| pc_load           | 0     |

0 ns 10 ns 20 ns 30 ns 40 ns 50 ns 60 ns 70 ns 80 ns 90 ns 100 ns 110 ns 120 ns 130 ns

0090 0590 0ab0 0f80 1090 1590 1ab0 1c80 2100 2580 2880 3c00

1 3 5 7 1 3 5 1 2 3 0

1 3 5 7 1 3 5 1 0 3 1 0

1 3 0 1 3 0 3 0 0

0 1 2 3 4 5 6 7 0 1 0

## 6.2.Register Bank

### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity register_bank is
    port(
        clk, reset, enable : in std_logic;
        write_en           : in std_logic;
        write_sel          : in std_logic_vector(2 downto 0);
        write_data         : in std_logic_vector(3 downto 0);
        R0,R1,R2,R3,R4,R5,R6,R7 : out std_logic_vector (3 downto 0));

end register_bank;

architecture Behavioral of register_bank is

    component decoder_3_to_8
        port(reg_sel : in std_logic_vector(2 downto 0);
            en : in std_logic;
            Y : out std_logic_vector(7 downto 0));
    end component;

    component register_4bit
        port(clk, reset, enable, we : in std_logic;
            D : in std_logic_vector(3 downto 0);
            Q : out std_logic_vector(3 downto 0));
    end component;

    signal we_lines : std_logic_vector(7 downto 0);
    --signal R0,R1,R2,R3,R4,R5,R6,R7 : std_logic_vector(3 downto 0);

begin

    DEC: decoder_3_to_8
        port map(reg_sel=>write_sel, en=>write_en, Y=>we_lines);

    REG0: register_4bit port map(clk => clk, reset=>reset,
        enable=>enable, we=>we_lines(0), D=>write_data, Q=>R0);
    REG1: register_4bit port map(clk => clk, reset=>reset,
        enable=>enable, we=>we_lines(1), D=>write_data, Q=>R1);

```

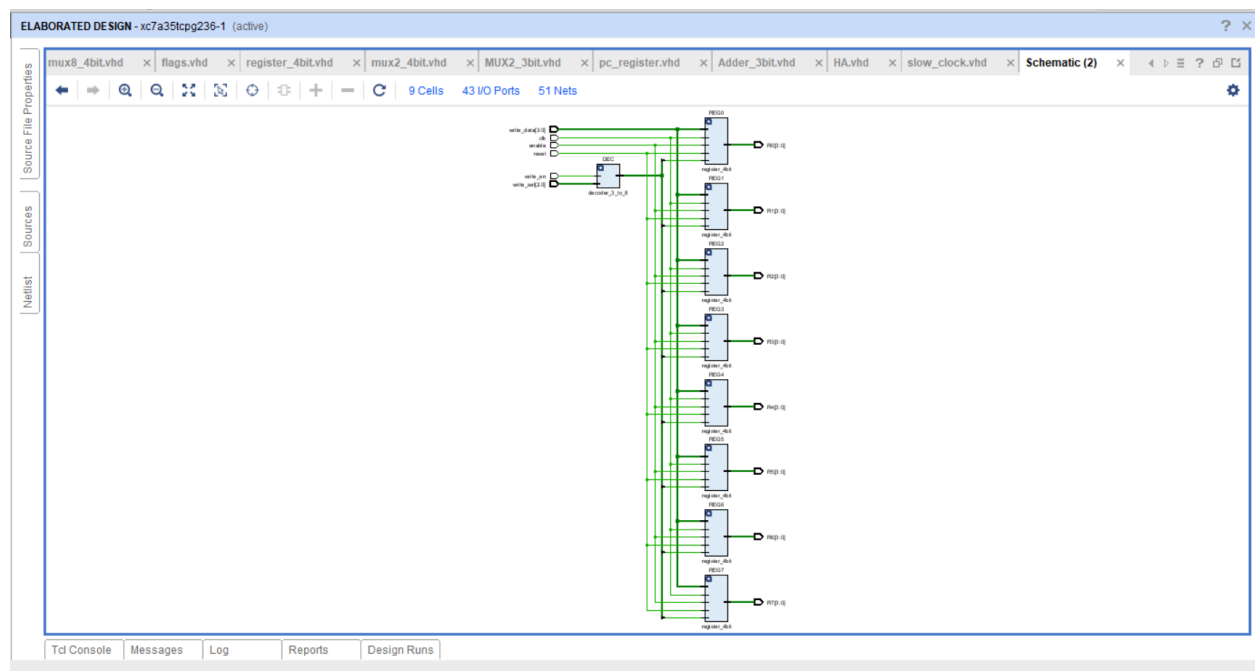
```

    REG2: register_4bit port map(clk => clk, reset=>reset,
enable=>enable, we=>we_lines(2), D=>write_data, Q=>R2);
    REG3: register_4bit port map(clk => clk, reset=>reset,
enable=>enable, we=>we_lines(3), D=>write_data, Q=>R3);
    REG4: register_4bit port map(clk => clk, reset=>reset,
enable=>enable, we=>we_lines(4), D=>write_data, Q=>R4);
    REG5: register_4bit port map(clk => clk, reset=>reset,
enable=>enable, we=>we_lines(5), D=>write_data, Q=>R5);
    REG6: register_4bit port map(clk => clk, reset=>reset,
enable=>enable, we=>we_lines(6), D=>write_data, Q=>R6);
    REG7: register_4bit port map(clk => clk, reset=>reset,
enable=>enable, we=>we_lines(7), D=>write_data, Q=>R7);

end Behavioral;

```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity register_bank_tb is
-- Port ( );
end register_bank_tb;

architecture Behavioral of register_bank_tb is

component register_bank
    port(
        clk, reset, enable : in std_logic;
        write_en           : in std_logic;
        write_sel           : in std_logic_vector(2 downto 0);
        write_data          : in std_logic_vector(3 downto 0);
        R0,R1,R2,R3,R4,R5,R6,R7 : out std_logic_vector(3 downto
0));
    end component;

    signal clk           : std_logic := '0';
    signal reset         : std_logic := '0';
    signal enable        : std_logic := '0';
    signal write_en      : std_logic := '0';
    signal write_sel     : std_logic_vector(2 downto 0) := "000";
    signal write_data    : std_logic_vector(3 downto 0) := "0000";
    signal R0, R1, R2, R3, R4, R5, R6, R7 : std_logic_vector(3 downto
0);

begin

    uut: register_bank
        port map (
            clk           => clk,
            reset         => reset,
            enable        => enable,
            write_en      => write_en,
            write_sel     => write_sel,
            write_data    => write_data,
            R0            => R0,
            R1            => R1,
            R2            => R2,
            R3            => R3,

```



```

        R4          => R4,
        R5          => R5,
        R6          => R6,
        R7          => R7
    );
    clk_process : process
    begin
        while true loop
            clk <= '0'; wait for 10 ns;
            clk <= '1'; wait for 10 ns;
        end loop;
    end process;

    stim_proc: process
    begin
--index no : 240312A - 111 010 101 010 111 000
                    -- 0011 1010 1010 1011 1000

        reset <= '1'; enable <= '0'; write_en <= '0'; wait for 25 ns;
        reset <= '0'; enable <= '1'; wait for 10 ns;

        write_en <= '1'; write_sel <= "000"; write_data <= "1000";
wait for 20 ns;
        write_en <= '0'; wait for 10 ns;

        write_en <= '1'; write_sel <= "111"; write_data <= "1011";
wait for 20 ns;
        write_en <= '0'; wait for 10 ns;

        write_en <= '1'; write_sel <= "010"; write_data <= "1010";
wait for 20 ns;
        write_en <= '0'; wait for 10 ns;

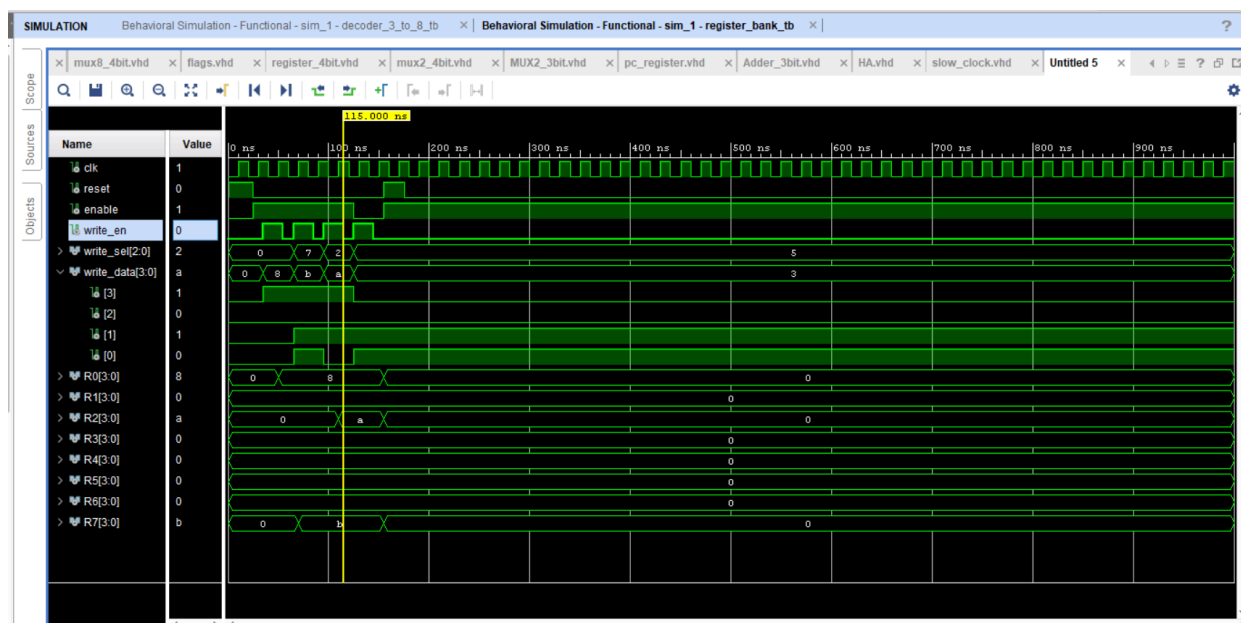
        enable <= '0';
        write_en <= '1'; write_sel <= "101"; write_data <= "0011";
wait for 20 ns;
        write_en <= '0'; wait for 10 ns;
        enable <= '1';

        reset <= '1'; wait for 20 ns;
        reset <= '0'; wait for 20 ns;

        wait;
    end process;
end Behavioral;

```

## Timing Diagram



## 6.3. Program Counter

### Design Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity pc is
    Port ( clk : in STD_LOGIC;
          enable : in STD_LOGIC;
          reset : in STD_LOGIC;
          pc_load : in STD_LOGIC;
          jump_addr : in STD_LOGIC_VECTOR (2 downto 0);
          pc_out : out STD_LOGIC_VECTOR (2 downto 0));
end pc;

architecture Behavioral of pc is

    component pc_register
        port(
```

```

        clk      : in std_logic;
        enable   : in std_logic;
        reset    : in std_logic;
        D        : in std_logic_vector(2 downto 0);
        Q        : out std_logic_vector(2 downto 0)
    );
end component;

component adder_3bit
    port(
        A : in std_logic_vector(2 downto 0);
        Y : out std_logic_vector(2 downto 0)
    );
end component;

component mux2_3bit
    port(
        sel : in std_logic;
        A   : in std_logic_vector(2 downto 0);
        B   : in std_logic_vector(2 downto 0);
        Y   : out std_logic_vector(2 downto 0)
    );
end component;

signal pc_reg      : std_logic_vector(2 downto 0);
signal pc_plus1    : std_logic_vector(2 downto 0);
signal pc_next     : std_logic_vector(2 downto 0);

begin

U1: pc_register
    port map(
        clk      => clk,
        enable   => enable,
        reset    => reset,
        D        => pc_next,
        Q        => pc_reg
    );

U2: adder_3bit
    port map(
        A => pc_reg,
        Y => pc_plus1
    );

```

```

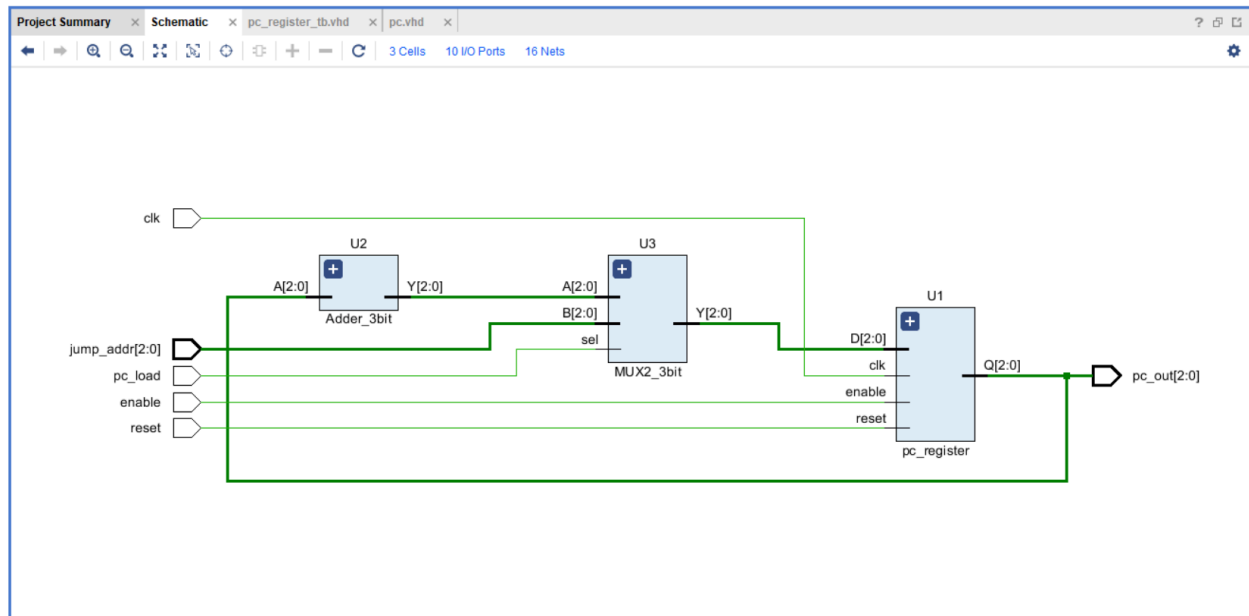
U3: mux2_3bit
  port map(
    sel => pc_load,
    A   => pc_plus1,
    B   => jump_addr,
    Y   => pc_next
  );

```

```
pc_out <= pc_reg;
```

```
end Behavioral;
```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity pc_tb is
--  Port ( );
end pc_tb;

architecture Behavioral of pc_tb is
component pc
    Port ( clk      : in  STD_LOGIC;
          enable    : in  STD_LOGIC;
          reset     : in  STD_LOGIC;
          pc_load   : in  STD_LOGIC;
          jump_addr : in  STD_LOGIC_VECTOR (2 downto 0);
          pc_out    : out STD_LOGIC_VECTOR (2 downto 0));
end component;

signal clk      : STD_LOGIC := '0';
signal enable   : STD_LOGIC := '0';
signal reset    : STD_LOGIC := '0';
signal pc_load  : STD_LOGIC := '0';
signal jump_addr : STD_LOGIC_VECTOR(2 downto 0) := "000";
signal pc_out   : STD_LOGIC_VECTOR(2 downto 0);

begin

    uut: pc
        port map (
            clk      => clk,
            enable   => enable,
            reset    => reset,
            pc_load  => pc_load,
            jump_addr => jump_addr,
            pc_out   => pc_out
        );

    clk_process: process
    begin
        while true loop
            clk <= '0'; wait for 10 ns;
            clk <= '1'; wait for 10 ns;
        end loop
    end process
end pc_tb;

```

```

        end loop;
    end process;

    stim_proc: process
    begin
        enable <= '0';
        reset <= '1';
        pc_load <= '0'; jump_addr <= "001"; wait for 25 ns;
        reset <= '0';
        enable <= '1';                                wait for 70 ns;

        pc_load <= '1'; jump_addr <= "110"; wait for 25 ns;
        pc_load <= '0';                                wait for 70 ns;

        -- index No : 240297 - 111 010 101 010 101 001

        pc_load <= '1'; jump_addr <= "111"; wait for 25 ns;
        pc_load <= '0';                                wait for 70 ns;

        pc_load <= '1'; jump_addr <= "010"; wait for 25 ns;
        pc_load <= '0';                                wait for 70 ns;

        pc_load <= '1'; jump_addr <= "101"; wait for 25 ns;
        pc_load <= '0';                                wait for 70 ns;

        pc_load <= '1'; jump_addr <= "010"; wait for 25 ns;
        pc_load <= '0';                                wait for 70 ns;

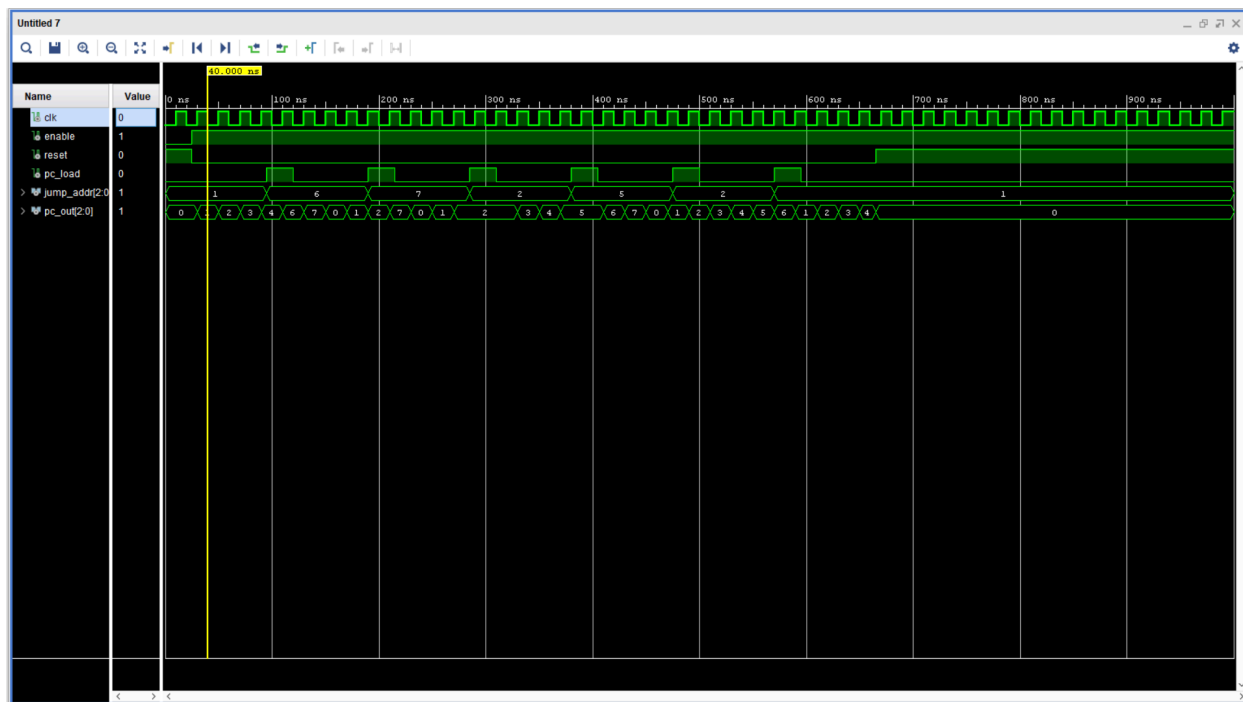
        pc_load <= '1'; jump_addr <= "001"; wait for 25 ns;
        pc_load <= '0';                                wait for 70 ns;

        reset <= '1';wait for 25 ns;
        wait;
    end process;

end Behavioral;

```

## Timing Diagram



## 6.4. ALU

| Item               | Optimized Version        | Extended Version                         | Change Type   |
|--------------------|--------------------------|------------------------------------------|---------------|
| Top_alu entity     | Does not exist           | Exists with 3-bit alu_op                 | New module    |
| ADD_SUB_4bit       | Used standalone          | Used as submodule inside top_alu         | Reused        |
| ALU operations     | Only ADD/SUB via alu_sel | ADD, SUB, OR, AND, XOR, NOT, comparisons | Expanded      |
| Comparator module  | Not present              | Used inside top_alu                      | New submodule |
| Logic unit (binOp) | Not present              | Used for AND/OR/XOR/NOT                  | New submodule |
| Output flags       | overflow, zero           | overflow, zero, GT, EQ, LT               | Added         |
| Flag behavior      | Direct from ADD/SUB      | Muxed based on alu_top                   | Changed       |

## Optimized Version

### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ADD_SUB_4bit is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);
          alu_sel : in STD_LOGIC;
          Y : out STD_LOGIC_VECTOR (3 downto 0);
          carry : out STD_LOGIC;
          overflow : out STD_LOGIC;
          zero : out STD_LOGIC);
end ADD_SUB_4bit;

architecture Behavioral of ADD_SUB_4bit is
    component RCA_4bit
        port(
            A      : in  std_logic_vector(3 downto 0);
            B      : in  std_logic_vector(3 downto 0);
            C_in   : in  std_logic;
            S      : out std_logic_vector(3 downto 0);
            C_out  : out std_logic
        );
    end component;
    signal B_mod : std_logic_vector(3 downto 0);
    signal sum   : std_logic_vector(3 downto 0);
    signal c_out : std_logic;

begin

    B_mod <= B xor (alu_sel & alu_sel & alu_sel & alu_sel);

    U1: rca_4bit
        port map(
            A      => A,
            B      => B_mod,
            C_in   => alu_sel,  -- 0 for add, 1 for subtract
            S      => sum,
            C_out  => c_out
        );

```



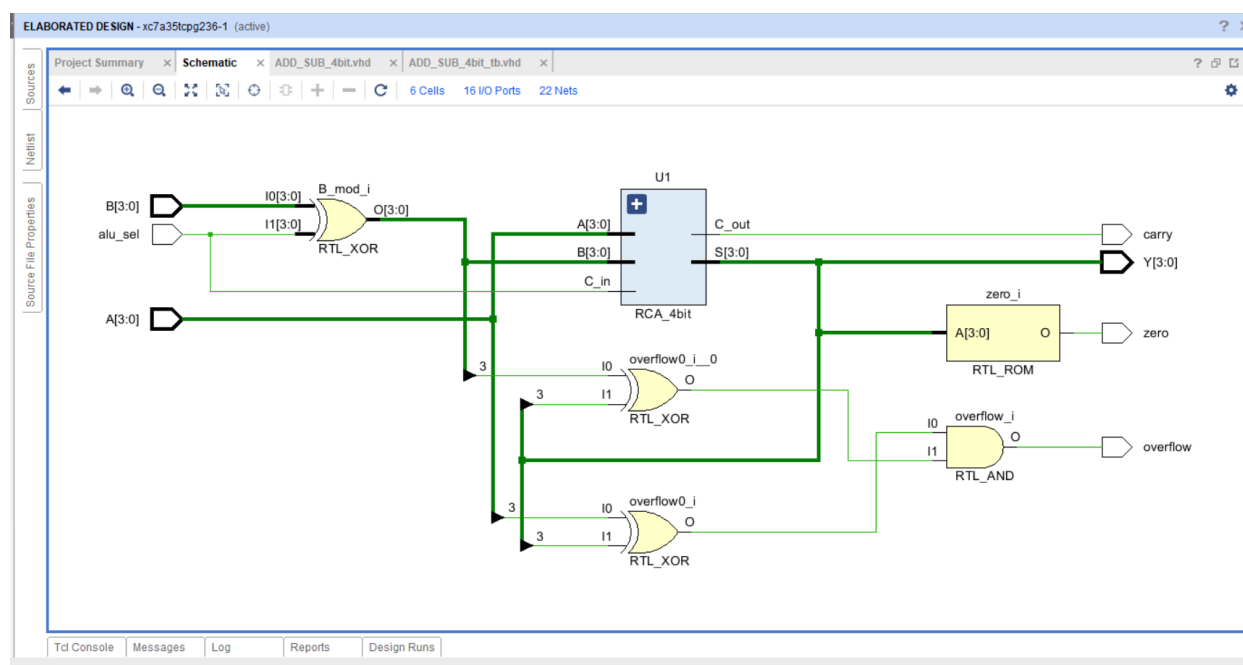
```

Y      <= sum;
carry <= c_out;
overflow <= (A(3) xor sum(3)) and (B_mod(3) xor sum(3));
zero <= '1' when sum = "0000" else '0';

end Behavioral;

```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ADD_SUB_4bit_tb is
-- Port ( );
end ADD_SUB_4bit_tb;

architecture Behavioral of ADD_SUB_4bit_tb is

component ADD_SUB_4bit
Port ( A      : in  STD_LOGIC_VECTOR (3 downto 0);
      B      : in  STD_LOGIC_VECTOR (3 downto 0);

```

```

        alu_sel  : in  STD_LOGIC;
        Y        : out STD_LOGIC_VECTOR (3 downto 0);
        carry    : out STD_LOGIC;
        overflow  : out STD_LOGIC;
        zero     : out STD_LOGIC);
end component;

signal A, B      : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
signal alu_sel   : STD_LOGIC := '0';
signal Y         : STD_LOGIC_VECTOR(3 downto 0);
signal carry     : STD_LOGIC;
signal overflow  : STD_LOGIC;
signal zero      : STD_LOGIC;

begin

UUT: ADD_SUB_4bit
    port map (
        A      => A,
        B      => B,
        alu_sel => alu_sel,
        Y      => Y,
        carry  => carry,
        overflow => overflow,
        zero   => zero
    );

-- Stimulus process
stim_proc: process
begin
--index no : 240288D - 0011 1010 1010 1010 0000
    -- ===== ADDITION TESTS (alu_sel = '0') =====
    alu_sel <= '0';
    A <= "0000"; B <= "1010"; wait for 10 ns;
    A <= "1010"; B <= "1010"; wait for 10 ns;
    A <= "0011"; B <= "1010"; wait for 10 ns;

    A <= "0001"; B <= "0101"; wait for 10 ns;    --random
    A <= "0111"; B <= "0001"; wait for 10 ns;    --random

    -- ===== SUBTRACTION TESTS (alu_sel = '1') =====
    alu_sel <= '1';
    A <= "0000"; B <= "1010"; wait for 10 ns;
    A <= "1010"; B <= "1010"; wait for 10 ns;
    A <= "0011"; B <= "1010"; wait for 10 ns;

```

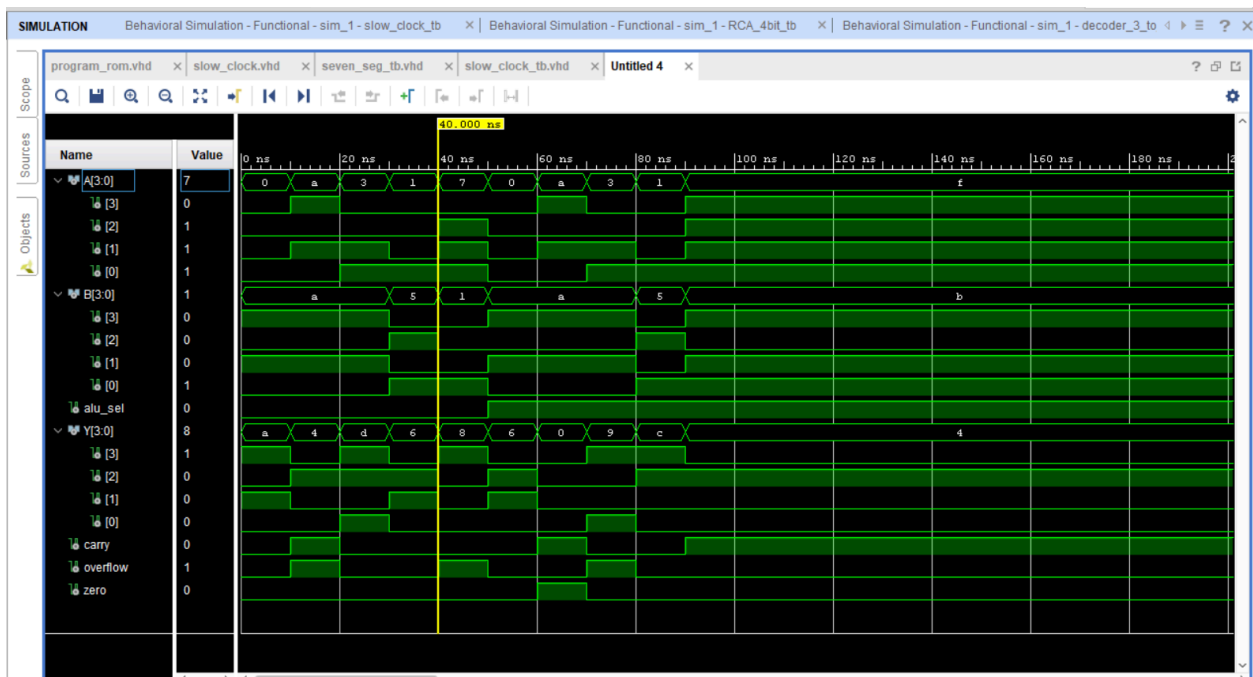
```

    A <= "0001"; B <= "0101"; wait for 10 ns;    --random
    A <= "1111"; B <= "1011"; wait for 10 ns;    --random
    wait;
end process;

end Behavioral;

```

## Timing Diagram



## Extended Version

### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity top_alu is
    Port (
        A      : in  STD_LOGIC_VECTOR (3 downto 0);
        B      : in  STD_LOGIC_VECTOR (3 downto 0);
        alu_op  : in  STD_LOGIC_VECTOR (2 downto 0);
        Y       : out STD_LOGIC_VECTOR (3 downto 0);
        carry   : out STD_LOGIC;
    );
end entity top_alu;

```

```

        overflow : out STD_LOGIC;
        zero     : out STD_LOGIC;
        GT       : out STD_LOGIC;
        EQ       : out STD_LOGIC;
        LT       : out STD_LOGIC);
end top_alu;

```

architecture Behavioral of top\_alu is

```

component binOp
    Port ( I : in  STD_LOGIC_VECTOR (1 downto 0);
          A : in  STD_LOGIC_VECTOR (3 downto 0);
          B : in  STD_LOGIC_VECTOR (3 downto 0);
          Y : out STD_LOGIC_VECTOR (3 downto 0));
end component;

component ADD_SUB_4bit
    Port ( A      : in  STD_LOGIC_VECTOR (3 downto 0);
          B      : in  STD_LOGIC_VECTOR (3 downto 0);
          alu_sel : in  STD_LOGIC;
          Y      : out STD_LOGIC_VECTOR (3 downto 0);
          carry   : out STD_LOGIC;
          overflow : out STD_LOGIC;
          zero    : out STD_LOGIC);
end component;

component comparator
    Port ( A, B      : in  STD_LOGIC_VECTOR(3 downto 0);
          sign_en : in  STD_LOGIC;
          GT, EQ, LT : out STD_LOGIC
        );
end component;

signal logic_result      : STD_LOGIC_VECTOR (3 downto 0);
signal arith_result      : STD_LOGIC_VECTOR (3 downto 0);
signal arith_carry       : STD_LOGIC;
signal arith_overflow    : STD_LOGIC;
signal arith_zero        : STD_LOGIC;
signal logic_ctrl        : STD_LOGIC_VECTOR (1 downto 0);
signal sub_sel           : STD_LOGIC;
signal cmp_GT, cmp_EQ, cmp_LT : STD_LOGIC;

begin

    logic_ctrl <= alu_op(1 downto 0);

```

```

logic_unit: binOp
    port map (
        I => logic_ctrl,
        A => A,
        B => B,
        Y => logic_result
    );

sub_sel <= '1' when alu_op = "001" else '0';

arith_unit: ADD_SUB_4bit
    port map (
        A      => A,
        B      => B,
        alu_sel => sub_sel,
        Y      => arith_result,
        carry   => arith_carry,
        overflow => arith_overflow,
        zero    => arith_zero
    );

comp_unit: comparator
    port map (
        A      => A,
        B      => B,
        sign_en => alu_op(0),
        GT      => cmp_GT,
        EQ      => cmp_EQ,
        LT      => cmp_LT );

    process(alu_op, logic_result, arith_result, arith_carry,
        arith_overflow, arith_zero, cmp_GT, cmp_EQ, cmp_LT)

begin

    case alu_op is
        when "000" | "001" =>
            Y      <= arith_result;
            carry   <= arith_carry;
            overflow <= arith_overflow;
            zero    <= arith_zero;
            GT      <= '0';
            EQ      <= '0';
    end case;

```

```

        LT          <= '0';

    when "010" | "011" =>
        Y           <= "0000";
        carry       <= '0';
        overflow    <= '0';
        zero        <= '0';
        GT          <= cmp_GT;
        EQ          <= cmp_EQ;
        LT          <= cmp_LT;

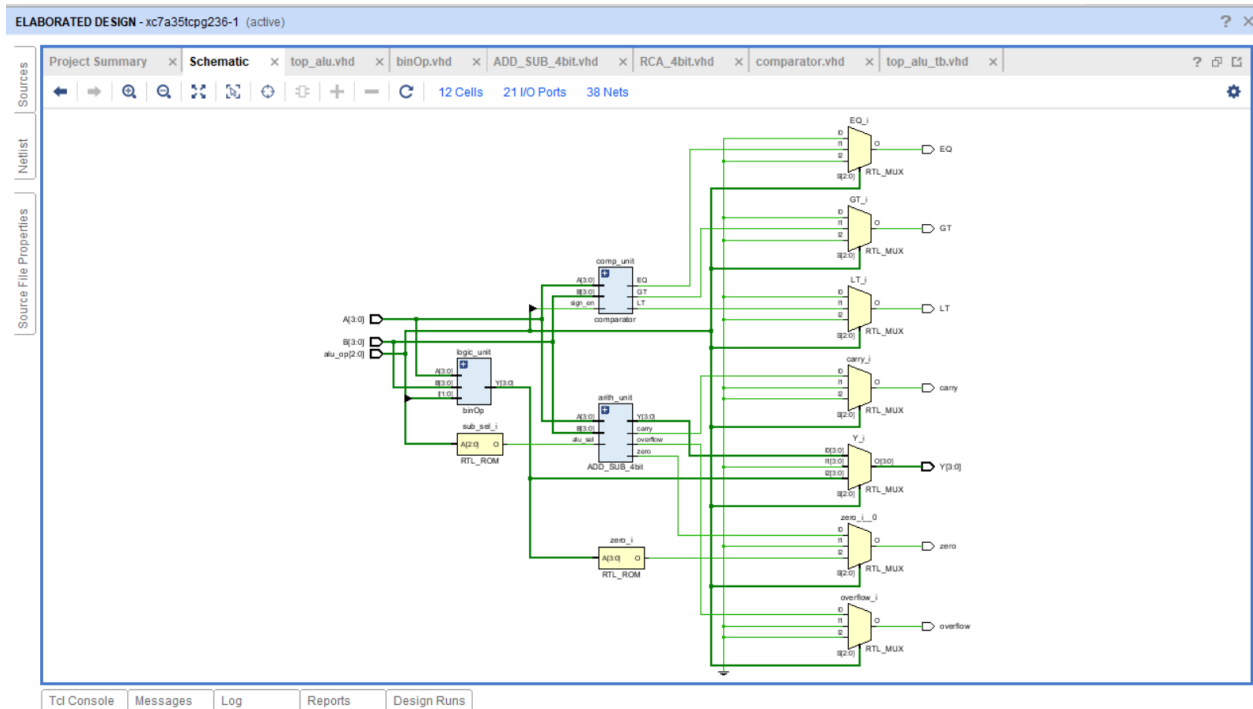
    when "100" | "101" | "110" | "111" =>
        Y           <= logic_result;
        GT          <= '0';
        EQ          <= '0';
        LT          <= '0';
        carry       <= '0';
        overflow    <= '0';
        if logic_result = "0000" then
            zero <= '1';
        else
            zero <= '0';
        end if;
    when others =>
        Y           <= (others => '0');
        carry       <= '0';
        overflow    <= '0';
        zero        <= '0';
    end case;

end process;

end Behavioral;

```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity top_alu_tb is
-- Port ( );
end top_alu_tb;

architecture Behavioral of top_alu_tb is

component top_alu
    Port ( A          : in  STD_LOGIC_VECTOR (3 downto 0);
          B          : in  STD_LOGIC_VECTOR (3 downto 0);
          alu_op      : in  STD_LOGIC_VECTOR (2 downto 0);
          Y           : out STD_LOGIC_VECTOR (3 downto 0);
          carry       : out STD_LOGIC;
          overflow    : out STD_LOGIC;
          zero        : out STD_LOGIC;
          GT          : out STD_LOGIC;
    );
end component;

-- Testbench logic would go here

```

```

        EQ          : out STD_LOGIC;
        LT          : out STD_LOGIC);
end component;

signal A, B      : STD_LOGIC_VECTOR(3 downto 0) := "0000";
signal alu_op    : STD_LOGIC_VECTOR(2 downto 0) := "000";
signal Y         : STD_LOGIC_VECTOR(3 downto 0);
signal carry     : STD_LOGIC;
signal overflow  : STD_LOGIC;
signal zero      : STD_LOGIC;
signal GT, EQ, LT : STD_LOGIC;

```

```
begin
```

```

    uut: top_alu
        port map (
            A      => A,
            B      => B,
            alu_op  => alu_op,
            Y       => Y,
            carry   => carry,
            overflow => overflow,
            zero    => zero,
            GT      => GT,
            EQ      => EQ,
            LT      => LT
        );

```

```

stim_proc : process
begin
--index no : 240288D - 0011 1010 1010 1010 0000
--          : 240294R - 0011 1010 1010 1010 0110
--          : 240297E - 0011 1010 1010 1010 1001
--          : 240312A - 0011 1010 1010 1011 1000

-- Unsigned compare (alu_op = "010")
alu_op <= "010";
-- 0 < 10
A <= "0000"; B <= "1010"; wait for 10 ns;
-- 10 > 16
A <= "1010"; B <= "0110"; wait for 10 ns;
-- 9 < 10
A <= "1001"; B <= "1010"; wait for 10 ns;
-- 8 < 11
A <= "1000"; B <= "1011"; wait for 10 ns;

```



```

-- 7 = 7
A <= "0111"; B <= "0111"; wait for 10 ns;

-- Signed compare (alu_op = "011")
alu_op <= "011";
-- -1 < 1 (1111 vs 0001)
A <= "1111"; B <= "0001"; wait for 10 ns;
-- 2 > -3 (0010 vs 1101)
A <= "0010"; B <= "1101"; wait for 10 ns;
-- -4 = -4 (1100 vs 1100)
A <= "1100"; B <= "1100"; wait for 10 ns;

-- Edge: 0 vs 0
A <= "0000"; B <= "0000";
alu_op <= "010"; wait for 10 ns;
alu_op <= "011"; wait for 10 ns;

wait;
end process;

end behavioral;

```

## Timing Diagram



## New Components

### Comparator

#### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity comparator is
Port ( A, B      : in  STD_LOGIC_VECTOR(3 downto 0);
      sign_en : in  STD_LOGIC;
      GT, EQ, LT : out STD_LOGIC
      );
end comparator;

architecture Behavioral of comparator is

signal E : STD_LOGIC_VECTOR(3 downto 0);
signal GT_u, LT_u : STD_LOGIC;
signal GT_s, LT_s : STD_LOGIC;

begin

    -- Equality bits
    E(0) <= A(0) xnor B(0);
    E(1) <= A(1) xnor B(1);
    E(2) <= A(2) xnor B(2);
    E(3) <= A(3) xnor B(3);

    EQ <= E(3) and E(2) and E(1) and E(0);

    -- Unsigned comparator
    GT_u <= (A(3) and not B(3)) or
            (E(3) and A(2) and not B(2)) or
            (E(3) and E(2) and A(1) and not B(1)) or
            (E(3) and E(2) and E(1) and A(0) and not B(0));

    LT_u <= (not A(3) and B(3)) or
            (E(3) and not A(2) and B(2)) or
            (E(3) and E(2) and not A(1) and B(1)) or
            (E(3) and E(2) and E(1) and not A(0) and B(0));

    -- Signed comparator

```

```
GT_s <= (not A(3) and B(3)) or
        (E(3) and GT_u);
```

```
LT_s <= (A(3) and not B(3)) or
        (E(3) and LT_u);
```

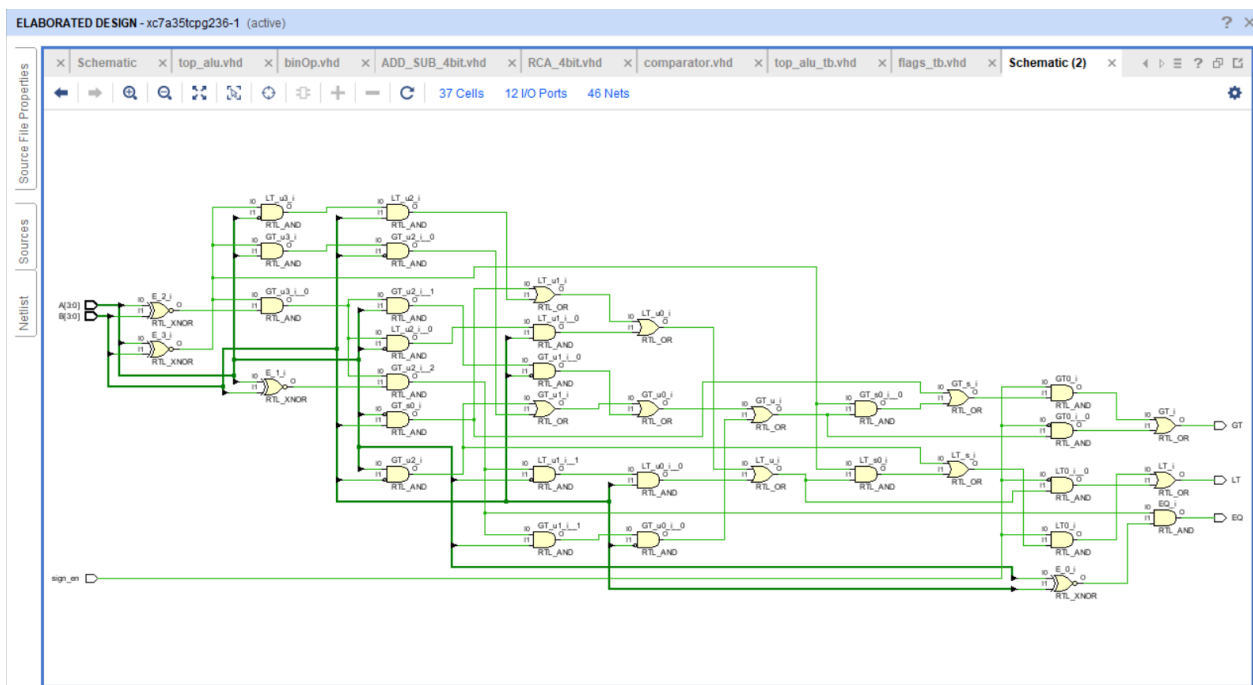
```
-- Mode selection
```

```
GT <= (sign_en and GT_s) or ((not sign_en) and GT_u);
```

```
LT <= (sign_en and LT_s) or ((not sign_en) and LT_u);
```

```
end Behavioral;
```

## Elaborated Design Schematic



## Logic Unit (Binary Operations)

### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity binOp is
    Port ( I : in STD_LOGIC_VECTOR (1 downto 0);
          A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);
          Y : out STD_LOGIC_VECTOR (3 downto 0));
end binOp;

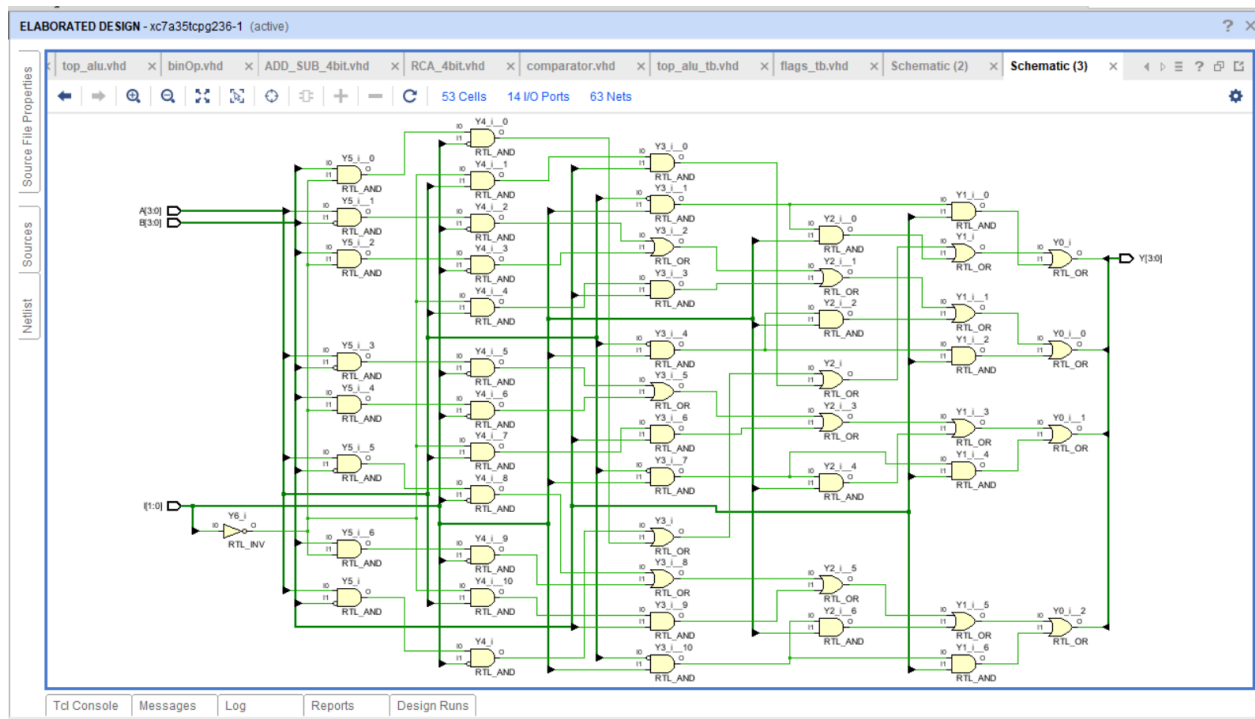
architecture Behavioral of binOp is

begin
    Y(0) <= (A(0) AND NOT B(0) AND NOT I(0)) OR (B(0)AND NOT I(1) AND
NOT I(0)) OR (NOT I(1) AND A(0) AND B(0)) OR (NOT A(0) AND I(1) AND
I(0)) OR (I(1) AND NOT A(0) AND B(0));
    Y(1) <= (A(1) AND NOT B(1) AND NOT I(0)) OR (B(1)AND NOT I(1) AND
NOT I(0)) OR (NOT I(1) AND A(1) AND B(1)) OR (NOT A(1) AND I(1) AND
I(0)) OR (I(1) AND NOT A(1) AND B(1));
    Y(2) <= (A(2) AND NOT B(2) AND NOT I(0)) OR (B(2)AND NOT I(1) AND
NOT I(0)) OR (NOT I(1) AND A(2) AND B(2)) OR (NOT A(2) AND I(1) AND
I(0)) OR (I(1) AND NOT A(2) AND B(2));
    Y(3) <= (A(3) AND NOT B(3) AND NOT I(0)) OR (B(3)AND NOT I(1) AND
NOT I(0)) OR (NOT I(1) AND A(3) AND B(3)) OR (NOT A(3) AND I(1) AND
I(0)) OR (I(1) AND NOT A(3) AND B(3));

end Behavioral;

```

## Elaborated Design Schematic



## 6.5.Seven Segment

### Optimized Version

### Design Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

--use UNISIM.VComponents.all;

entity seven_seg is
    Port (
        data_in    : in  std_logic_vector(3 downto 0);
        digit0     : out std_logic_vector(6 downto 0); -- right (LSB)
        digit1     : out std_logic_vector(6 downto 0)  -- left (sign)
    );
end seven_seg;
```

```

architecture Behavioral of seven_seg is
    constant SEG_0      : std_logic_vector(6 downto 0) := "1000000";
    constant SEG_1      : std_logic_vector(6 downto 0) := "1111001";
    constant SEG_2      : std_logic_vector(6 downto 0) := "0100100";
    constant SEG_3      : std_logic_vector(6 downto 0) := "0110000";
    constant SEG_4      : std_logic_vector(6 downto 0) := "0011001";
    constant SEG_5      : std_logic_vector(6 downto 0) := "0010010";
    constant SEG_6      : std_logic_vector(6 downto 0) := "0000010";
    constant SEG_7      : std_logic_vector(6 downto 0) := "1111000";
    constant SEG_8      : std_logic_vector(6 downto 0) := "0000000";

    constant SEG_MINUS  : std_logic_vector(6 downto 0) := "0111111";
    constant SEG_BLANK  : std_logic_vector(6 downto 0) := "1111111";

    signal abs_val : std_logic_vector(3 downto 0);

begin
    -- absolute value for negative numbers (2's complement)
    abs_val <= std_logic_vector(unsigned(-signed(data_in))) when
data_in(3) = '1'
        else data_in;

    process(data_in, abs_val)
    begin
        -- default
        digit0 <= SEG_BLANK;
        digit1 <= SEG_BLANK;

        -- sign
        if data_in(3) = '1' then
            digit1 <= SEG_MINUS;
        end if;

        -- magnitude (0-7 only)
        case abs_val is
            when "0000" => digit0 <= SEG_0;
            when "0001" => digit0 <= SEG_1;
            when "0010" => digit0 <= SEG_2;
            when "0011" => digit0 <= SEG_3;
            when "0100" => digit0 <= SEG_4;
            when "0101" => digit0 <= SEG_5;
            when "0110" => digit0 <= SEG_6;
            when "0111" => digit0 <= SEG_7;
            when others => digit0 <= SEG_8;
        end case;
    end process;
end Behavioral;

```

```

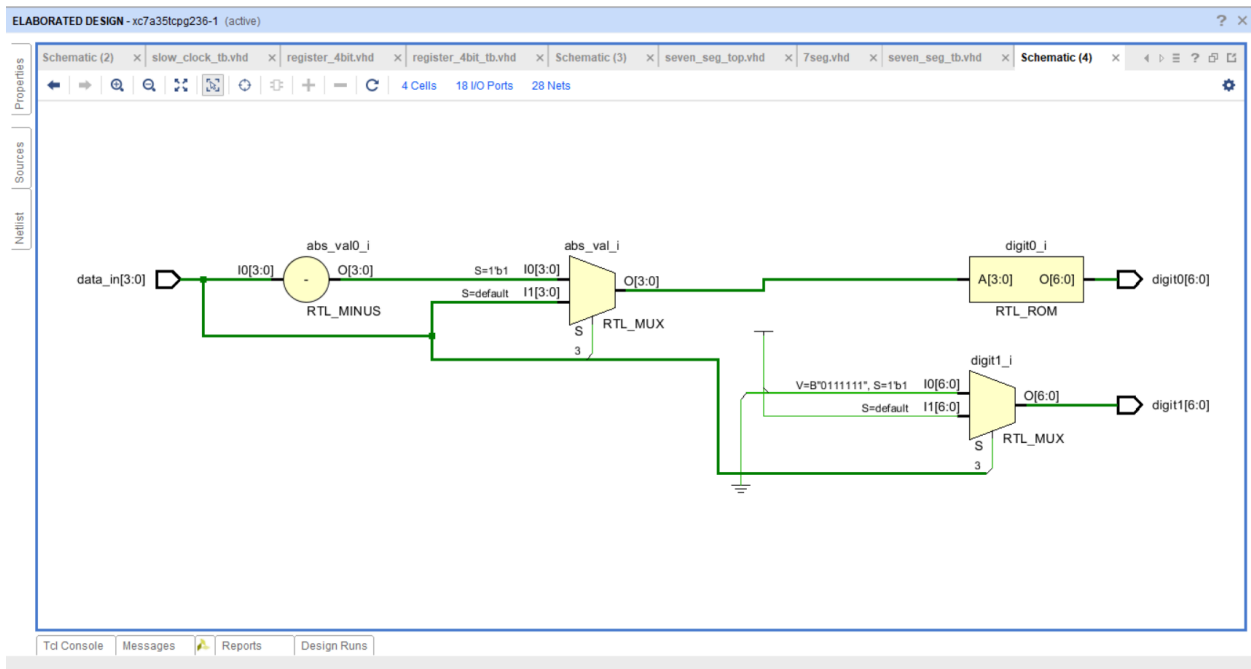
        end case;

    end process;

end Behavioral;

```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity seven_seg_tb is
    -- Port ( );
end seven_seg_tb;

architecture Behavioral of seven_seg_tb is

    component seven_seg
        Port (
            data_in : in  std_logic_vector(3 downto 0);
            digit0   : out std_logic_vector(6 downto 0);
            digit1   : out std_logic_vector(6 downto 0)
        );
    end component seven_seg;

    -- Implementation details would follow here

```

```

end component;

signal data_in : std_logic_vector(3 downto 0) := (others => '0');
signal digit0  : std_logic_vector(6 downto 0);
signal digit1  : std_logic_vector(6 downto 0);

begin

    uut: seven_seg
        port map (
            data_in => data_in,
            digit0  => digit0,
            digit1  => digit1
        );

    stim_proc: process
    begin
        for i in 0 to 15 loop
            data_in <= std_logic_vector(to_unsigned(i, 4));
            wait for 10 ns;
        end loop;
        wait;
    end process;
end Behavioral;

```

## Timing Diagram





## Extended Version

### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity seven_seg is
    Port (
        data_in    : in  std_logic_vector(3 downto 0);
        digit0     : out std_logic_vector(6 downto 0); -- right (LSB)
        digit1     : out std_logic_vector(6 downto 0)  -- left (sign)
    );
end seven_seg;

architecture Behavioral of seven_seg is
    constant SEG_0      : std_logic_vector(6 downto 0) := "1000000";
    constant SEG_1      : std_logic_vector(6 downto 0) := "1111001";
    constant SEG_2      : std_logic_vector(6 downto 0) := "0100100";
    constant SEG_3      : std_logic_vector(6 downto 0) := "0110000";
    constant SEG_4      : std_logic_vector(6 downto 0) := "0011001";
    constant SEG_5      : std_logic_vector(6 downto 0) := "0010010";
    constant SEG_6      : std_logic_vector(6 downto 0) := "0000010";
    constant SEG_7      : std_logic_vector(6 downto 0) := "1111000";
    constant SEG_8      : std_logic_vector(6 downto 0) := "0000000";

    constant SEG_MINUS  : std_logic_vector(6 downto 0) := "0111111";
    constant SEG_BLANK  : std_logic_vector(6 downto 0) := "1111111";

    signal abs_val : std_logic_vector(3 downto 0);

begin
    -- absolute value for negative numbers (2's complement)
    abs_val <= std_logic_vector(unsigned(-signed(data_in))) when
data_in(3) = '1'
        else data_in;

    process(data_in, abs_val)
    begin
        -- default
        digit0 <= SEG_BLANK;
        digit1 <= SEG_BLANK;
    end process;
end Behavioral;

```

```

-- sign
if data_in(3) = '1' then
    digit1 <= SEG_MINUS;
end if;

-- magnitude (0-7 only)

case abs_val is
    when "0000" => digit0 <= SEG_0;
    when "0001" => digit0 <= SEG_1;
    when "0010" => digit0 <= SEG_2;
    when "0011" => digit0 <= SEG_3;
    when "0100" => digit0 <= SEG_4;
    when "0101" => digit0 <= SEG_5;
    when "0110" => digit0 <= SEG_6;
    when "0111" => digit0 <= SEG_7;
    when others => digit0 <= SEG_8;
end case;

end process;

end Behavioral;

```

## 6.6.Program ROM

| Item                      | Optimized Version              | Extended Version               | Change Type |
|---------------------------|--------------------------------|--------------------------------|-------------|
| Data width                | 12 bits                        | 14 bits                        | Increased   |
| Instruction format        | 2-bit opcode + 10-bit operands | 4-bit opcode + 10-bit operands | Changed     |
| ROM depth                 | 8 locations                    | 8 locations                    | Same        |
| Constant definition style | Explicit binary                | Explicit binary                | Same        |

## Optimized Version

### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity program_rom is
    Port ( addr : in  STD_LOGIC_VECTOR (2 downto 0);
          data : out STD_LOGIC_VECTOR (11 downto 0));
end program_rom;

architecture Behavioral of program_rom is

    type rom_array is array (0 to 7) of std_logic_vector(11 downto 0);

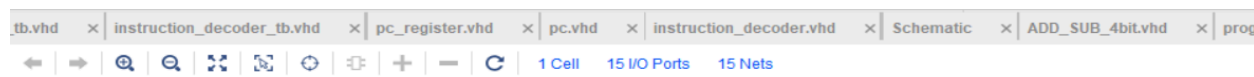
    constant ROM : rom_array := (
        0 => "101110000000",    -- MOVI R7, 0    (R7 = 0)
        1 => "100010000001",    -- MOVI R1, 1    (R1 = 1)
        2 => "001110010000",    -- ADD  R7, R1    (R7 = 0+1 = 1)
        3 => "100010000010",    -- MOVI R1, 2    (R1 = 2)
        4 => "001110010000",    -- ADD  R7, R1    (R7 = 1+2 = 3)
        5 => "100010000011",    -- MOVI R1, 3    (R1 = 3)
        6 => "001110010000",    -- ADD  R7, R1    (R7 = 3+3 = 6)
        7 => "110000000000",    -- JZR  R0, 0
        others => (others => '0')
    );

begin
    data <= ROM(to_integer(unsigned(addr)));

end Behavioral;

```

## Elaborated Design Schematic



## Extended Version

### Design Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity program_rom is
    Port ( addr : in  STD_LOGIC_VECTOR (2 downto 0);
          data : out STD_LOGIC_VECTOR (13 downto 0));
end program_rom;

architecture Behavioral of program_rom is

    type rom_array is array (0 to 7) of std_logic_vector(13 downto 0);

    constant ROM : rom_array := (
        0 => "10001110001001",
```

```

        1 => "100000010000101",
        2 => "00111110010001",
        3 => "00101110010000",
        4 => "00001110010000",
        5 => "10011110000000",
        6 => "00111110010000",
        7 => "10100000000000",
        others => (others => '0')
    );
begin
    data <= ROM(to_integer(unsigned(addr)));

end Behavioral;

```

## 6.7.Slow Clock

### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity slow_clock is
    Port ( clk_in : in STD_LOGIC;
          enable : out STD_LOGIC);
end slow_clock;

architecture Behavioral of slow_clock is
    signal count : unsigned(25 downto 0) := (others => '0');

begin
    process(clk_in)
    begin
        if rising_edge(clk_in) then
            if count = 5 then
                enable <= '1';
                count <= (others => '0');
            else
                enable <= '0';
                count <= count + 1;
            end if;
        end if;
    end process;
end Behavioral;

```

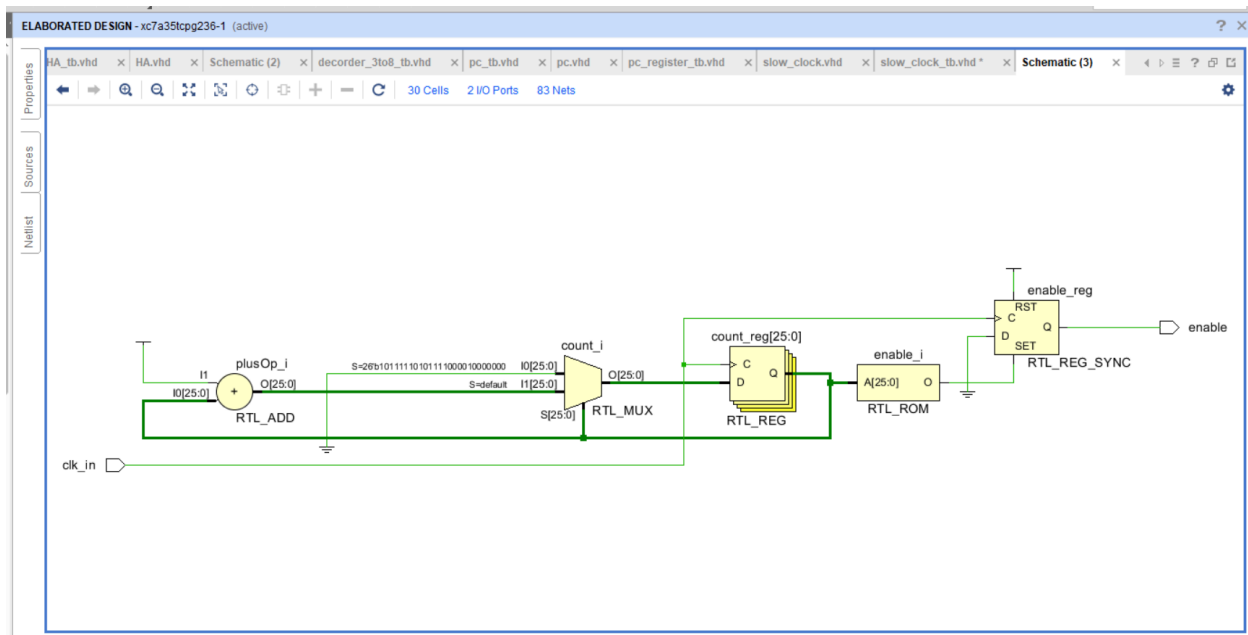
```

        end if;
    end process;

    end Behavioral;

```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity slow_clock_tb is
-- Port ( );
end slow_clock_tb;

architecture Behavioral of slow_clock_tb is
    component slow_clock
        Port ( clk_in : in STD_LOGIC;
              enable : out STD_LOGIC);
    end component;

    signal clk_in, enable : std_logic;
begin

```

```

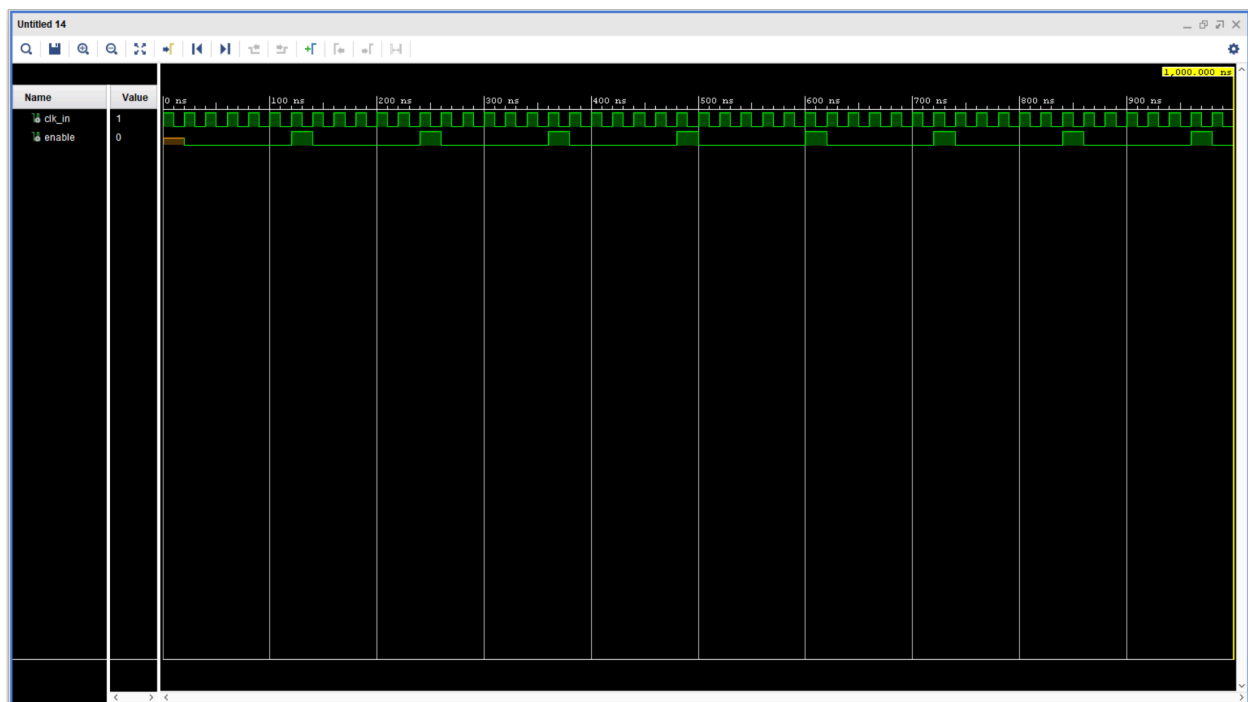
UUT : slow_clock
    port map ( clk_in => clk_in,
               enable => enable );

clk_process : process
begin
    while true loop
        clk_in <= '1'; wait for 10ns;
        clk_in <= '0'; wait for 10 ns;
    end loop;
end process;

end Behavioral;

```

## Timing Diagram



## 7. Building Blocks

### 1. 7.1. Half Adder

#### Design Source File

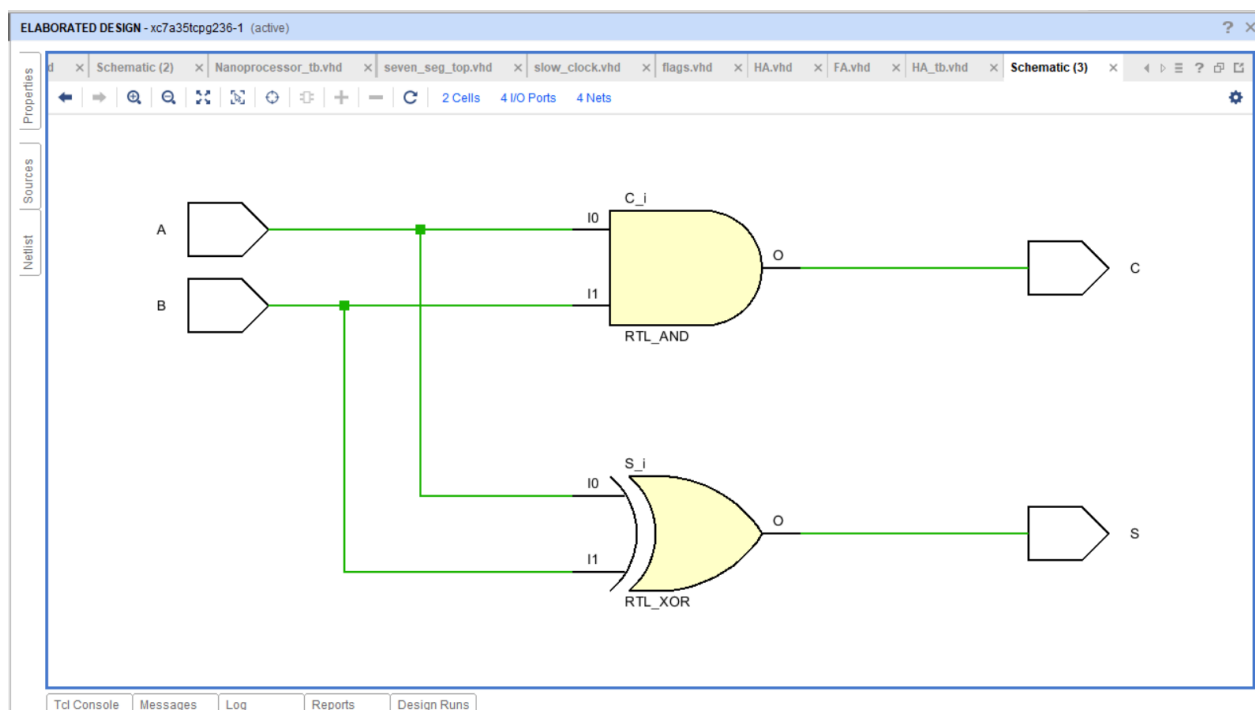
```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HA is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          C : out STD_LOGIC;
          S : out STD_LOGIC);
end HA;
architecture Behavioral of HA is
begin
    S <= A xor B;
    C <= A and B;
end Behavioral;

```

#### Elaborated Design Schematic





## Simulation Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HA_tb is
-- Port ( );
end HA_tb;

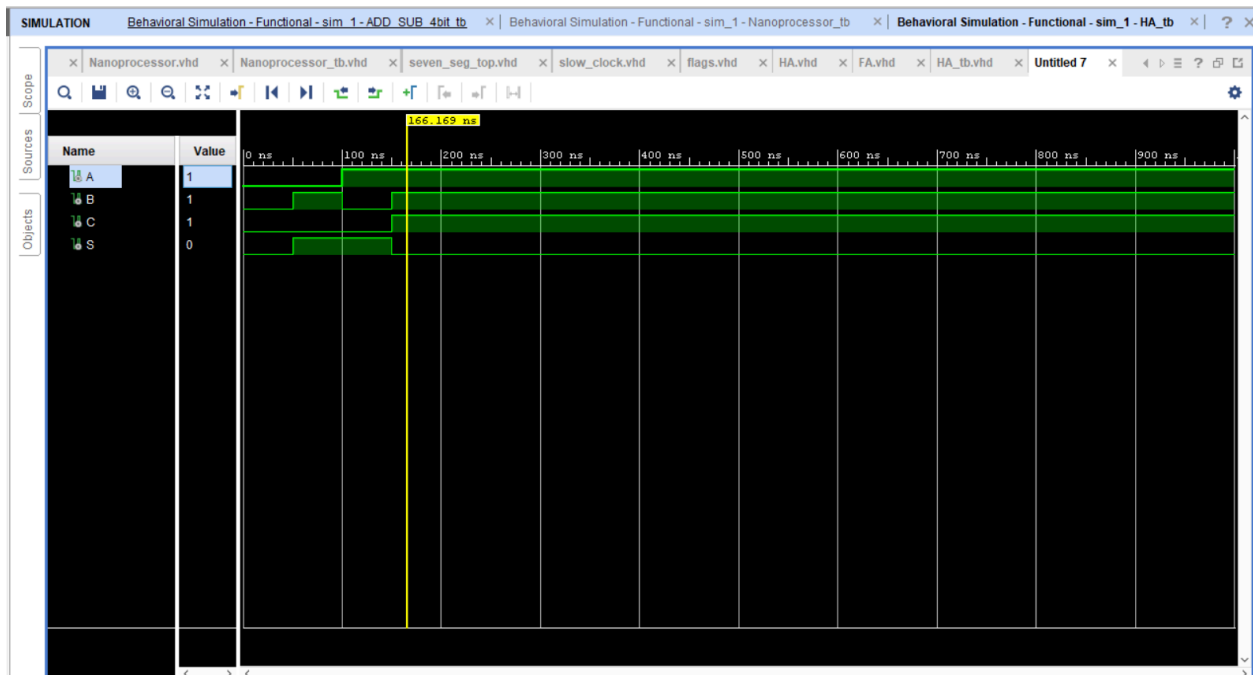
architecture Behavioral of HA_tb is
component HA
    Port ( A : in  STD_LOGIC;
          B : in  STD_LOGIC;
          C : out STD_LOGIC;
          S : out STD_LOGIC);
end component;
signal A, B : STD_LOGIC := '0';
signal C, S : STD_LOGIC;

begin

    uut: HA port map (
        A => A,
        B => B,
        C => C,
        S => S
    );

    stim_proc: process
    begin
        A <= '0'; B <= '0'; wait for 50 ns;
        A <= '0'; B <= '1'; wait for 50 ns;
        A <= '1'; B <= '0'; wait for 50 ns;
        A <= '1'; B <= '1'; wait for 50 ns;
        wait;
    end process;
end Behavioral;
```

## Timing Diagram



## 7.2. Full Adder

### Design Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FA is
    Port ( A : in STD_LOGIC;
          B : in STD_LOGIC;
          C_in : in STD_LOGIC;
          S : out STD_LOGIC;
          C_out : out STD_LOGIC);
end FA;

architecture Behavioral of FA is
    component HA
        port (
            A: in std_logic;
            B: in std_logic;
```

```

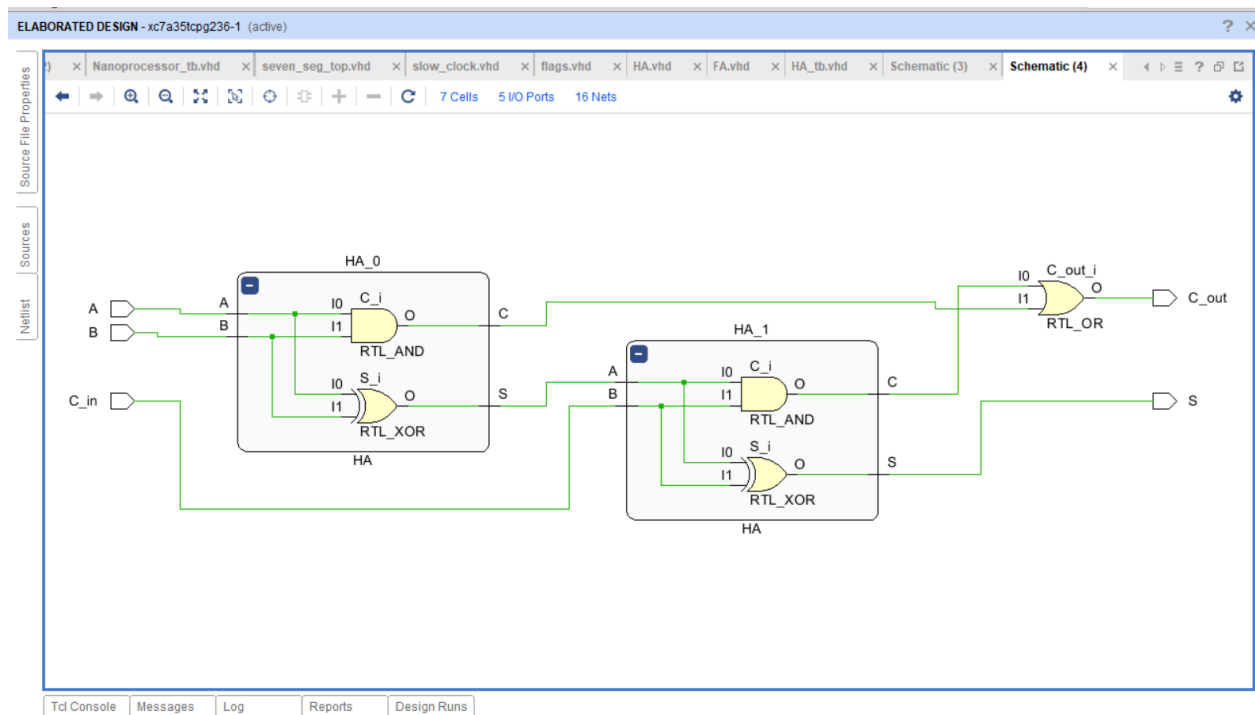
        S: out std_logic;
        C: out std_logic);
end component;
signal HA0_C, HA0_S, HA1_C, HA1_S : std_logic;

begin
HA_0 : HA
    port map(
        A => A,
        B => B,
        C => HA0_C,
        S => HA0_S);
HA_1 : HA
    port map(
        A => HA0_S,
        B => C_in,
        C => HA1_C,
        S => HA1_S);

S <= HA1_S;
C_out <= HA1_C or HA0_C;
end Behavioral;

```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FA_tb is
--  Port ( );
end FA_tb;

architecture Behavioral of FA_tb is
component HA
    Port ( A : in  STD_LOGIC;
          B : in  STD_LOGIC;
          C : out STD_LOGIC;
          S : out STD_LOGIC);
end component;

component FA
    Port ( A      : in  STD_LOGIC;
          B      : in  STD_LOGIC;
          C_in   : in  STD_LOGIC;
          S      : out STD_LOGIC;
          C_out  : out STD_LOGIC);
end component;

signal A, B, C_in : STD_LOGIC := '0';
signal S, C_out   : STD_LOGIC;

begin

 uut: FA port map (
    A      => A,
    B      => B,
    C_in   => C_in,
    S      => S,
    C_out  => C_out
);

stim_proc: process
begin
-- index no : 240288 111 010 101 010 100 000
--           : 240294 111 010 101 010 100 110
--           : 240297 111 010 101 010 101 001
--           : 240312 111 010 101 010 111 000

```

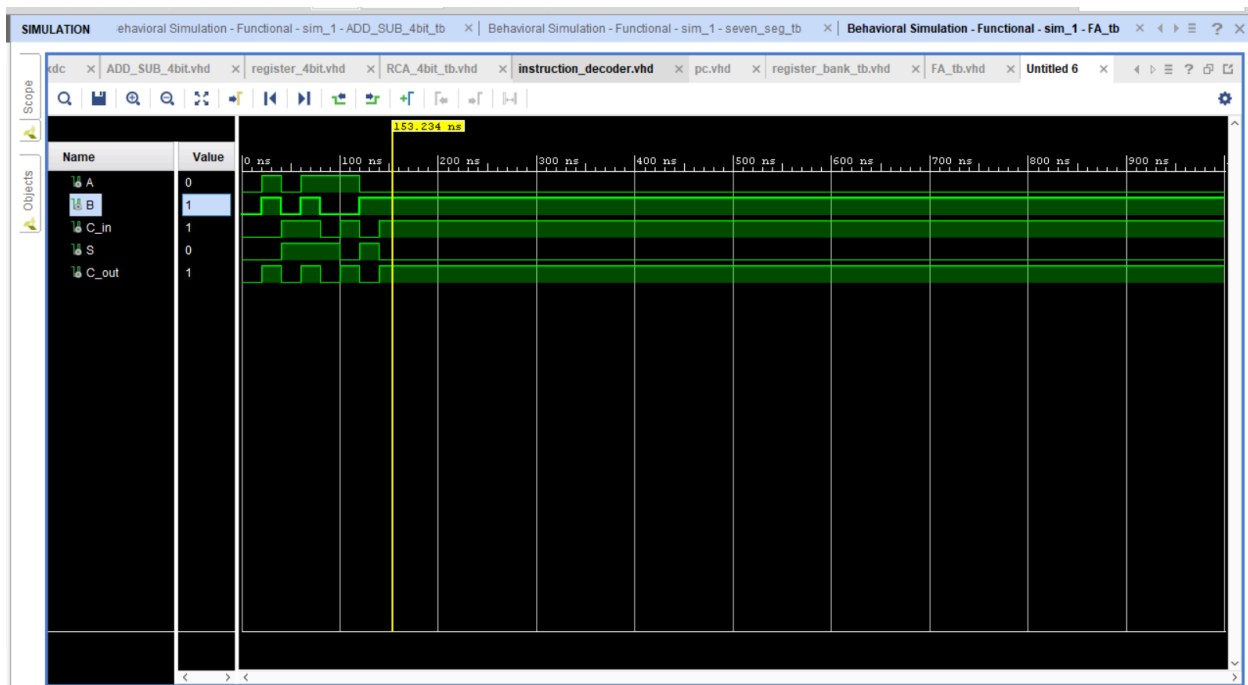
```

A <= '0'; B <= '0'; C_in <= '0'; wait for 20 ns;
A <= '1'; B <= '1'; C_in <= '0'; wait for 20 ns;
A <= '0'; B <= '0'; C_in <= '1'; wait for 20 ns;
A <= '1'; B <= '1'; C_in <= '1'; wait for 20 ns;
A <= '1'; B <= '0'; C_in <= '0'; wait for 20 ns;
A <= '1'; B <= '0'; C_in <= '1'; wait for 20 ns;
A <= '0'; B <= '1'; C_in <= '0'; wait for 20 ns;
A <= '0'; B <= '1'; C_in <= '1'; wait for 20 ns; --random
wait;
end process;

end Behavioral;

```

## Timing Diagram



## 7.3.Ripple Carry Adder (4-bit)

### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RCA_4bit is
    Port ( A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);
          C_in : in STD_LOGIC;
          S : out STD_LOGIC_VECTOR (3 downto 0);
          C_out : out STD_LOGIC);
end RCA_4bit;

architecture Behavioral of RCA_4bit is

    component FA
        port (
            A      : in std_logic;
            B      : in std_logic;
            C_in   : in std_logic;
            S      : out std_logic;
            C_out  : out std_logic
        );
    end component;

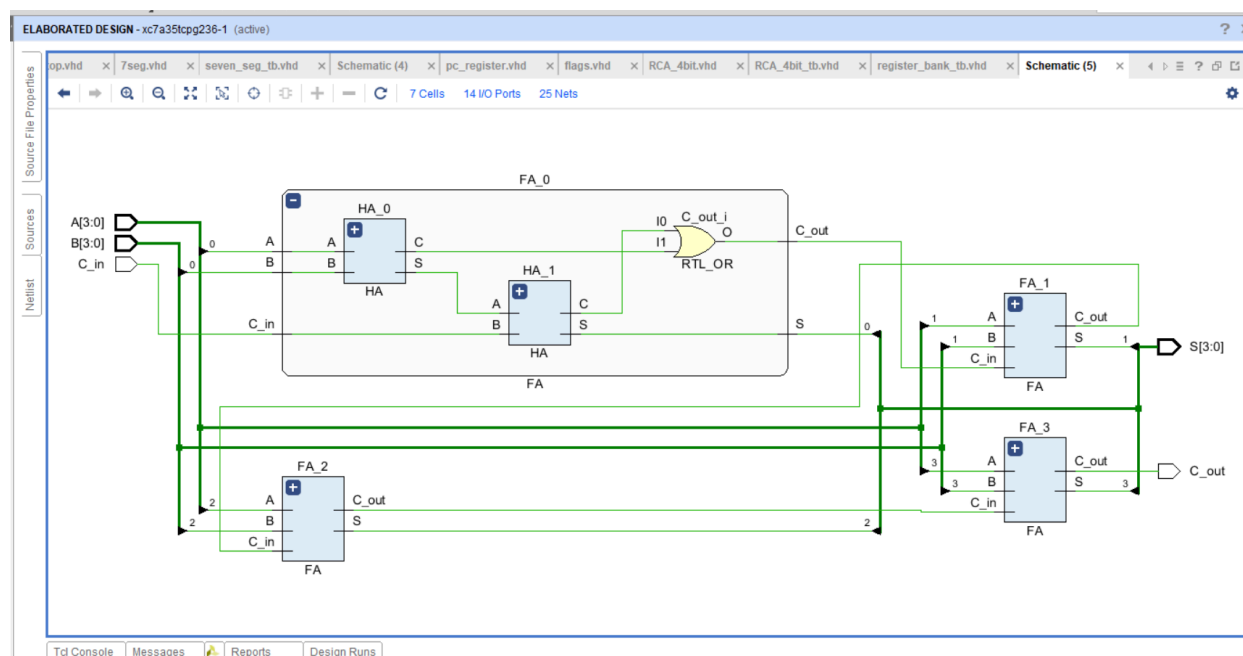
    signal C : std_logic_vector(4 downto 0);

begin
    C(0) <= C_in;
    FA_0: FA port map(A => A(0), B => B(0), C_in => C(0), S => S(0), C_out
    => C(1));
    FA_1: FA port map(A => A(1), B => B(1), C_in => C(1), S => S(1), C_out
    => C(2));
    FA_2: FA port map(A => A(2), B => B(2), C_in => C(2), S => S(2), C_out
    => C(3));
    FA_3: FA port map(A => A(3), B => B(3), C_in => C(3), S => S(3), C_out
    => C(4));
    C_out <= C(4);

end Behavioral;

```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RCA_4bit_tb is
-- Port ( );
end RCA_4bit_tb;

architecture Behavioral of RCA_4bit_tb is
component RCA_4bit
    Port ( A : in  STD_LOGIC_VECTOR (3 downto 0);
          B : in  STD_LOGIC_VECTOR (3 downto 0);
          C_in : in  STD_LOGIC;
          S : out STD_LOGIC_VECTOR (3 downto 0);
          C_out : out STD_LOGIC);
end component;

signal A, B, S : STD_LOGIC_VECTOR (3 downto 0);
signal C_in, C_out : STD_LOGIC;

begin
uut: RCA_4bit port map (

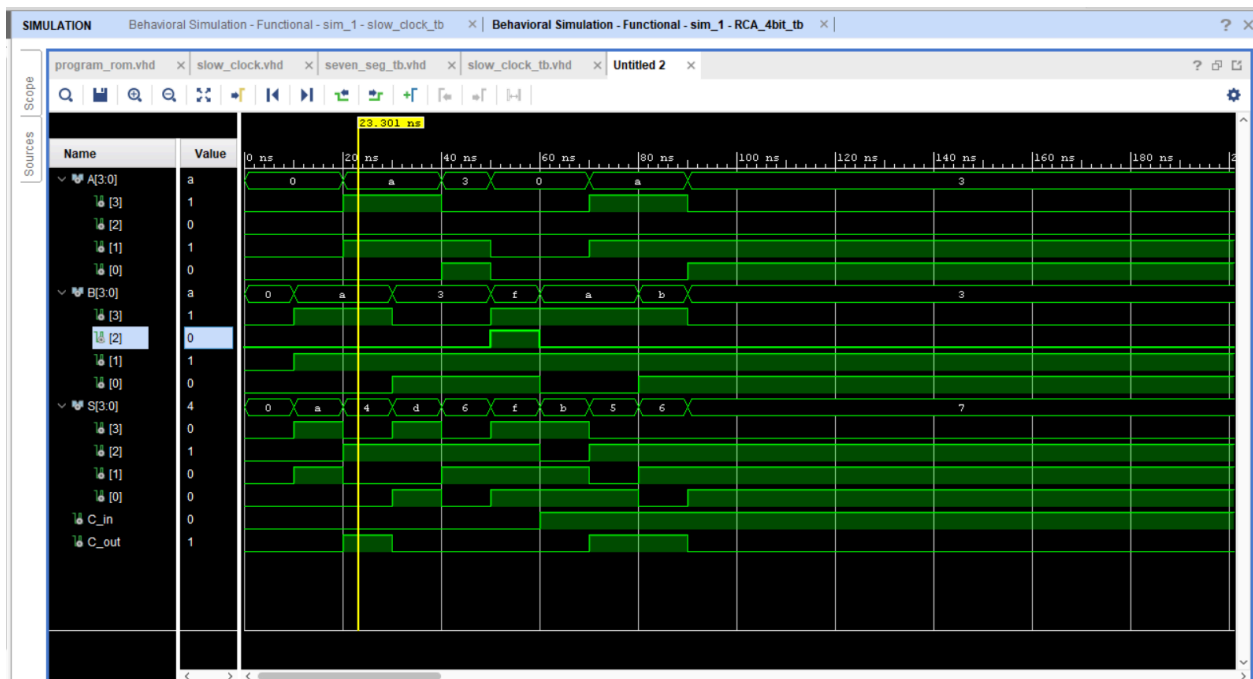
```

```

        A      => A,
        B      => B,
        C_in   => C_in,
        S      => S,
        C_out  => C_out
    );
--index no : 240288D - 0011 1010 1010 1010 0000
stim_proc: process
begin
    A <= "0000"; B <= "0000"; C_in <= '0'; wait for 10 ns;
    A <= "0000"; B <= "1010"; C_in <= '0'; wait for 10 ns;
    A <= "1010"; B <= "1010"; C_in <= '0'; wait for 10 ns;
    A <= "1010"; B <= "0011"; C_in <= '0'; wait for 10 ns;
    A <= "0011"; B <= "0011"; C_in <= '0'; wait for 10 ns;
    A <= "0000"; B <= "1111"; C_in <= '0'; wait for 10 ns;
    A <= "0000"; B <= "1010"; C_in <= '1'; wait for 10 ns;
    A <= "1010"; B <= "1010"; C_in <= '1'; wait for 10 ns;
    A <= "1010"; B <= "1011"; C_in <= '1'; wait for 10 ns;
    A <= "0011"; B <= "0011"; C_in <= '1'; wait for 10 ns;
    wait;
end process;
end Behavioral;

```

## Timing Diagram





## 7.4. Adder (3-bit)

### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Adder_3bit is
    Port ( A : in STD_LOGIC_VECTOR (2 downto 0);
          Y : out STD_LOGIC_VECTOR (2 downto 0));
end Adder_3bit;

architecture Behavioral of Adder_3bit is

    component rca_4bit
        port(
            A      : in  std_logic_vector(3 downto 0);
            B      : in  std_logic_vector(3 downto 0);
            C_in   : in  std_logic;
            S      : out std_logic_vector(3 downto 0);
            C_out  : out std_logic
        );
    end component;

    signal A_ext : std_logic_vector(3 downto 0);
    signal one   : std_logic_vector(3 downto 0) := "0001";
    signal sum   : std_logic_vector(3 downto 0);

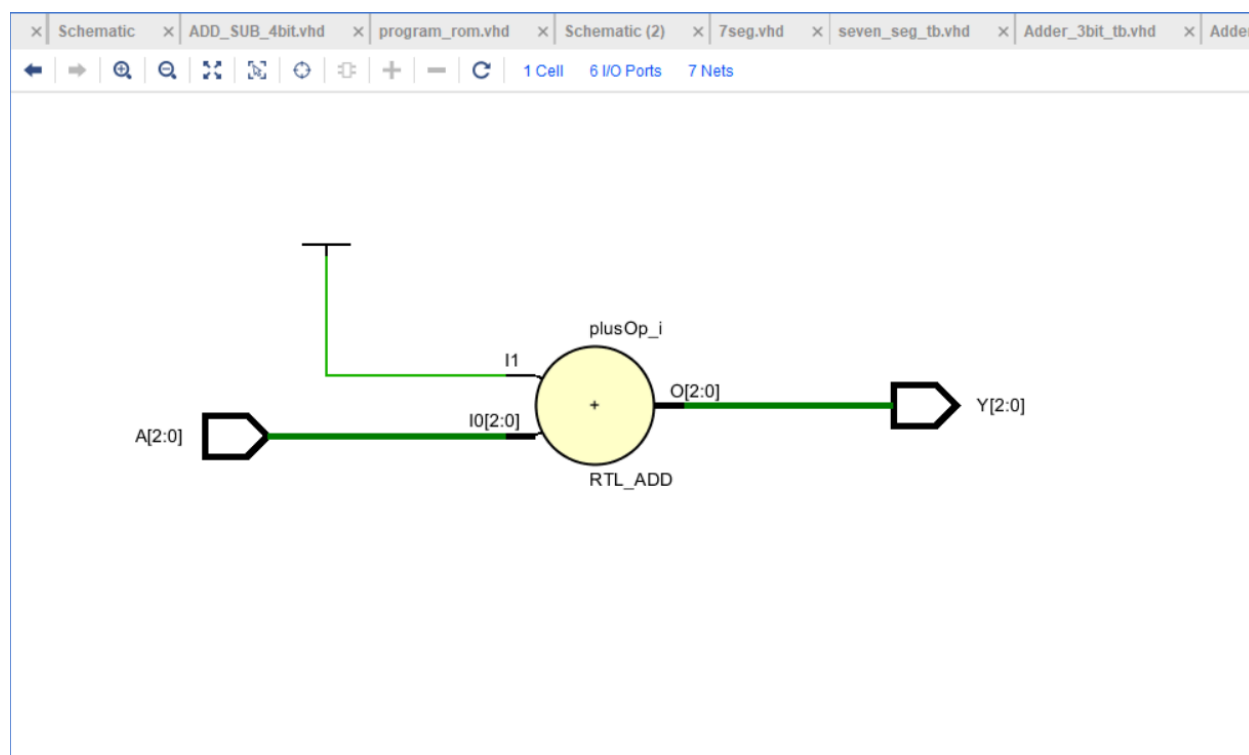
begin

    Y <= std_logic_vector(unsigned(A)+1);

end Behavioral;

```

## Elaborated Design Schematic



## Simulation Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Adder_3bit_tb is
-- Port ( );
end Adder_3bit_tb;

architecture Behavioral of Adder_3bit_tb is

component Adder_3bit
    Port ( A : in  STD_LOGIC_VECTOR (2 downto 0);
          Y : out STD_LOGIC_VECTOR (2 downto 0));
end component;

signal A : STD_LOGIC_VECTOR(2 downto 0) := (others => '0');
signal Y : STD_LOGIC_VECTOR(2 downto 0);

begin
```

```

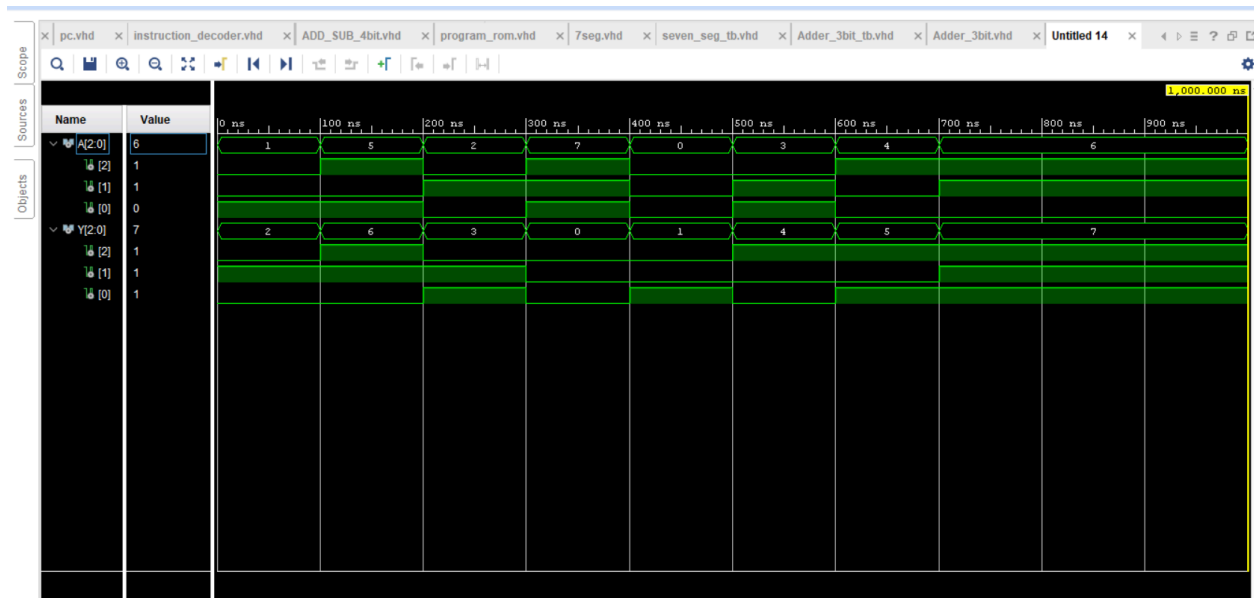
UUT: Adder_3bit port map (A => A, Y => Y);

stim_proc: process
begin
--- index no :- 240297E - 111 010 101 010 101 001
    A <= "001"; wait for 100ns;
    A <= "101"; wait for 100ns;
    A <= "010"; wait for 100ns;
    A <= "111"; wait for 100ns;
    --- random test values
    A <= "000"; wait for 100ns;
    A <= "011"; wait for 100ns;
    A <= "100"; wait for 100ns;
    A <= "110"; wait for 100ns;
    wait;
end process;

end Behavioral;

```

## Timing Diagram



## 7.5.Decoder (3 to 8)

### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

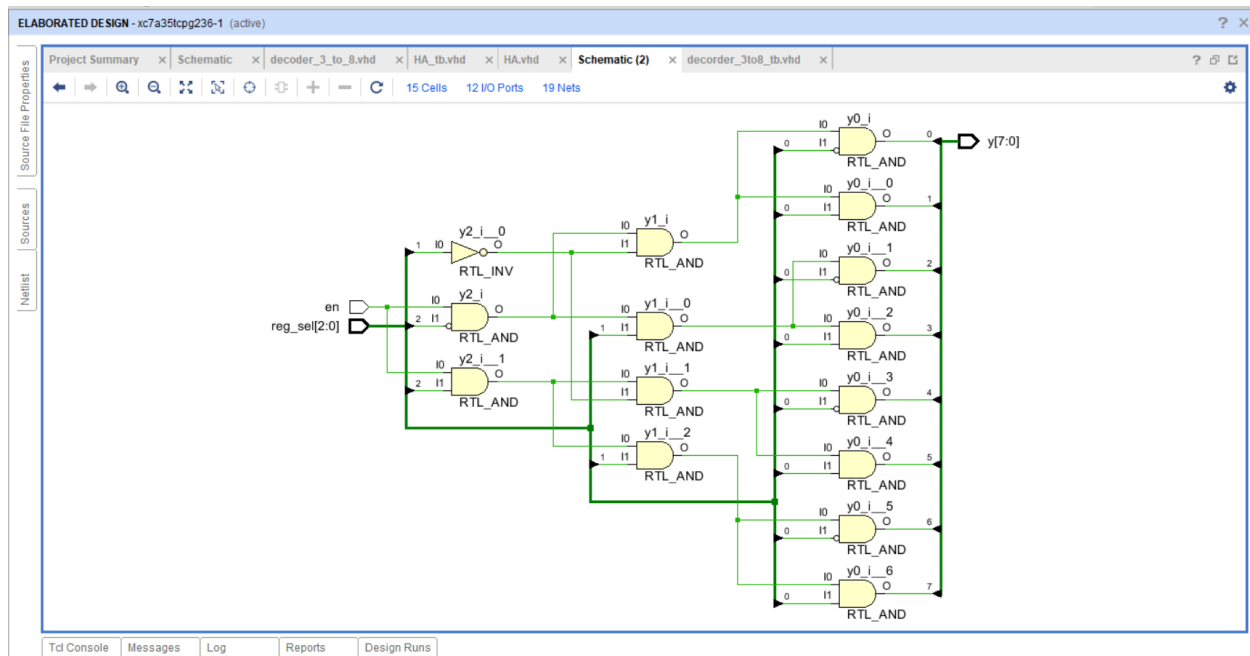
entity decoder_3_to_8 is
    Port ( reg_sel : in STD_LOGIC_VECTOR (2 downto 0);
          en : in STD_LOGIC;
          y : out STD_LOGIC_VECTOR (7 downto 0));
end decoder_3_to_8;

architecture Dataflow of decoder_3_to_8 is

begin
    y(0) <= en and (not reg_sel(2)) and (not reg_sel(1)) and (not
reg_sel(0));
    y(1) <= en and (not reg_sel(2)) and (not reg_sel(1)) and
reg_sel(0) ;
    y(2) <= en and (not reg_sel(2)) and      reg_sel(1)  and (not
reg_sel(0));
    y(3) <= en and (not reg_sel(2)) and      reg_sel(1)  and
reg_sel(0) ;
    y(4) <= en and      reg_sel(2)  and (not reg_sel(1)) and (not
reg_sel(0));
    y(5) <= en and      reg_sel(2)  and (not reg_sel(1)) and
reg_sel(0) ;
    y(6) <= en and      reg_sel(2)  and      reg_sel(1)  and (not
reg_sel(0));
    y(7) <= en and      reg_sel(2)  and      reg_sel(1)  and
reg_sel(0) ;
end Dataflow;

```

## Elaborated Design Schematic



## Simulation Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity decoder_3_to_8_tb is
-- Port ( );
end decoder_3_to_8_tb;

architecture Behavioral of decoder_3_to_8_tb is

component decoder_3_to_8
    Port ( reg_sel : in  STD_LOGIC_VECTOR (2 downto 0);
          en      : in  STD_LOGIC;
          y       : out STD_LOGIC_VECTOR (7 downto 0));
end component;

signal reg_sel : STD_LOGIC_VECTOR(2 downto 0) := "000";
signal en      : STD_LOGIC := '0';
signal y       : STD_LOGIC_VECTOR(7 downto 0);

begin
```

```

 uut: decoder_3_to_8
   port map (
     reg_sel => reg_sel,
     en      => en,
     y       => y
   );
--index no : 240312A - 111 010 101 010 111 000
stim_proc: process
begin
  en  <= '0';
  reg_sel <= "000"; wait for 10 ns;
  reg_sel <= "111"; wait for 10 ns;

  en  <= '1';
  reg_sel <= "000"; wait for 10 ns;
  reg_sel <= "111"; wait for 10 ns;
  reg_sel <= "010"; wait for 10 ns;
  reg_sel <= "101"; wait for 10 ns;

  reg_sel <= "001"; wait for 10 ns;  --random
  reg_sel <= "011"; wait for 10 ns;  --random
  reg_sel <= "100"; wait for 10 ns;  --random
  reg_sel <= "110"; wait for 10 ns;  --random

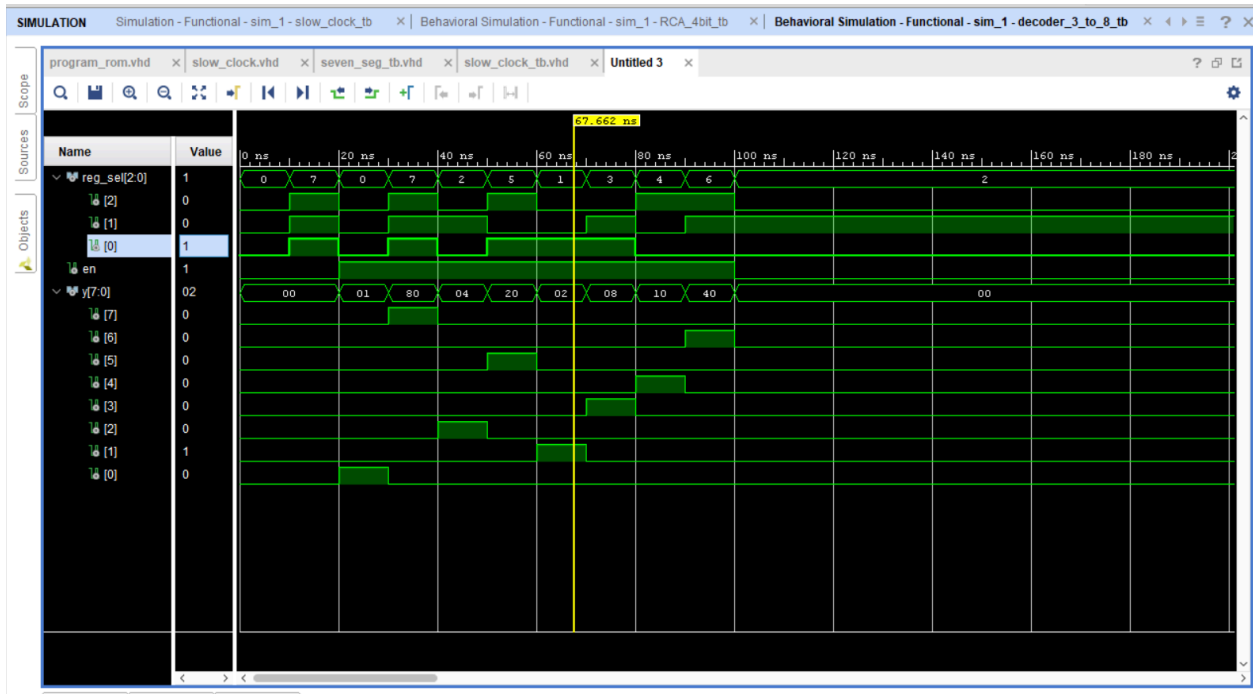
  en  <= '0';
  reg_sel <= "010"; wait for 10 ns;

  wait;
end process;

end Behavioral;

```

## Timing Diagram



## 7.6. Multiplexer (2 Way 3-bit)

### Design Source Code

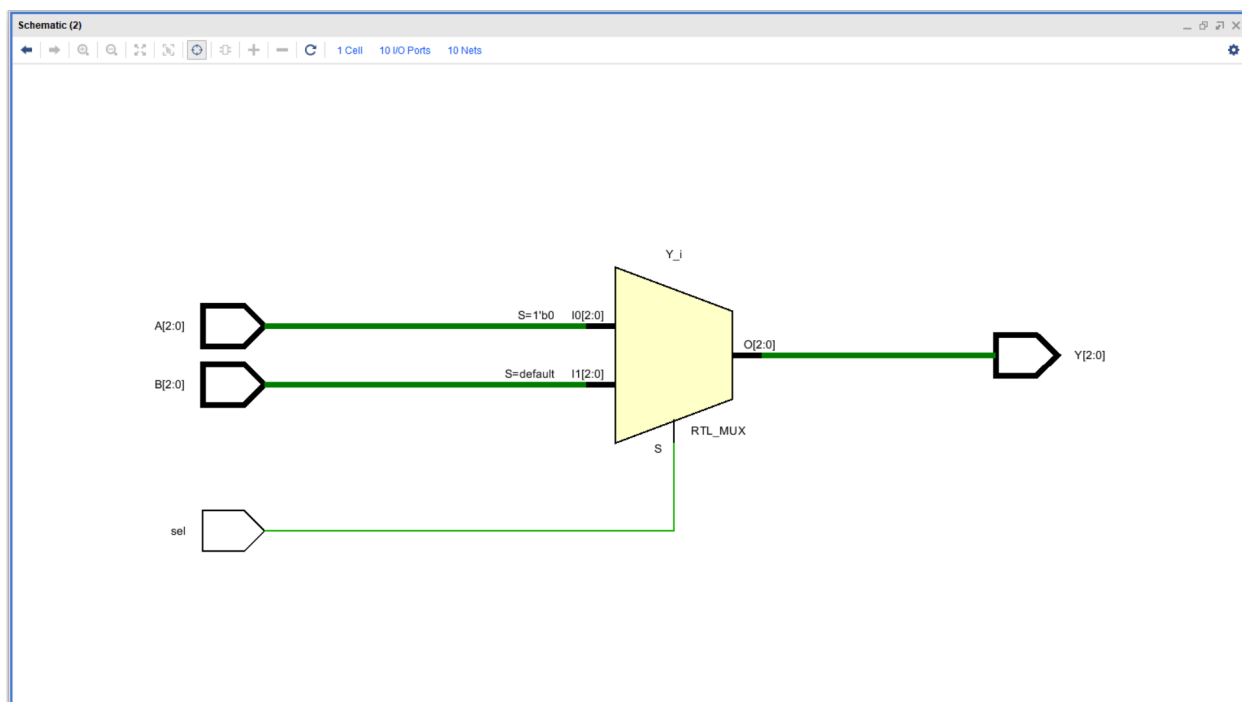
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX2_3bit is
    Port ( sel : in STD_LOGIC;
          A : in STD_LOGIC_VECTOR (2 downto 0);
          B : in STD_LOGIC_VECTOR (2 downto 0);
          Y : out STD_LOGIC_VECTOR (2 downto 0));
end MUX2_3bit;

architecture Behavioral of MUX2_3bit is

begin
    Y <= A when sel = '0' else B;
end Behavioral;
```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity MUX2_3bit_tb is
-- Port ( );
end MUX2_3bit_tb;

architecture Behavioral of MUX2_3bit_tb is

component MUX2_3bit
  Port ( sel : in  STD_LOGIC;
        A   : in  STD_LOGIC_VECTOR (2 downto 0);
        B   : in  STD_LOGIC_VECTOR (2 downto 0);
        Y   : out STD_LOGIC_VECTOR (2 downto 0));
end component;

signal sel : STD_LOGIC := '0';
signal A   : STD_LOGIC_VECTOR(2 downto 0) := "000";
signal B   : STD_LOGIC_VECTOR(2 downto 0) := "000";
signal Y   : STD_LOGIC_VECTOR(2 downto 0);

```



```

begin
  uut: MUX2_3bit
    port map (
      sel => sel,
      A   => A,
      B   => B,
      Y   => Y
    );

  stim_proc: process
  begin
    sel <= '0'; A <= "101"; B <= "010"; wait for 10 ns;
    sel <= '0'; A <= "001"; B <= "110"; wait for 10 ns;
    sel <= '1'; A <= "101"; B <= "010"; wait for 10 ns;
    sel <= '1'; A <= "000"; B <= "111"; wait for 10 ns;
    sel <= '0'; A <= "011"; B <= "011"; wait for 10 ns;
    sel <= '1'; A <= "011"; B <= "011"; wait for 10 ns;

    -- index No 240297 - 111 010 101 010 101 001

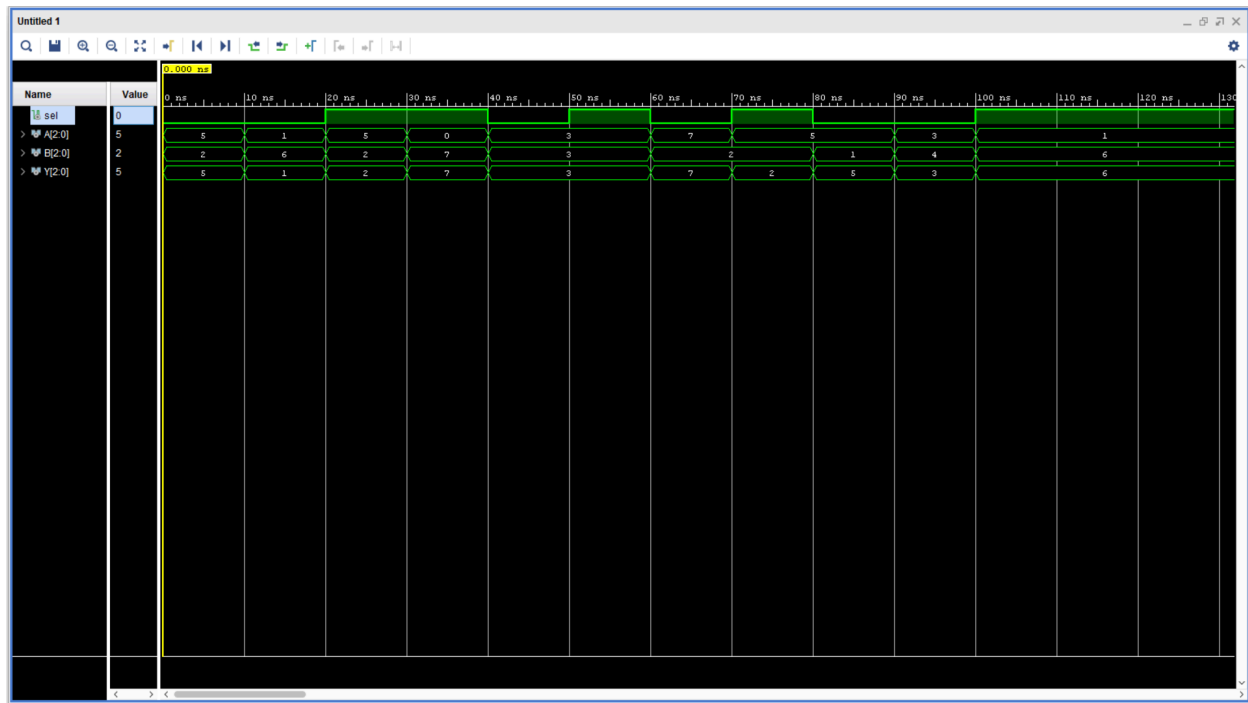
    sel <= '0'; A <= "111"; B <= "010"; wait for 10 ns; -- 240288
  pattern
    sel <= '1'; A <= "101"; B <= "010"; wait for 10 ns; -- 240294
  pattern
    sel <= '0'; A <= "101"; B <= "001"; wait for 10 ns; -- 240297
  pattern

    -- additional mixed stress cases
    sel <= '0'; A <= "011"; B <= "100"; wait for 10 ns;
    sel <= '1'; A <= "001"; B <= "110"; wait for 10 ns;
    wait;
  end process;

end Behavioral;

```

## Timing Diagram



## 7.7. Multiplexer (8-way 4-bit)

### Design Source Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux8_4bit is
    port(
        sel : in std_logic_vector(2 downto 0);
        I0, I1, I2, I3, I4, I5, I6, I7 : in std_logic_vector(3 downto 0);
        Y : out std_logic_vector(3 downto 0)
    );
end mux8_4bit;

architecture Behavioral of mux8_4bit is

begin
    process(sel, I0, I1, I2, I3, I4, I5, I6, I7)
    begin
```

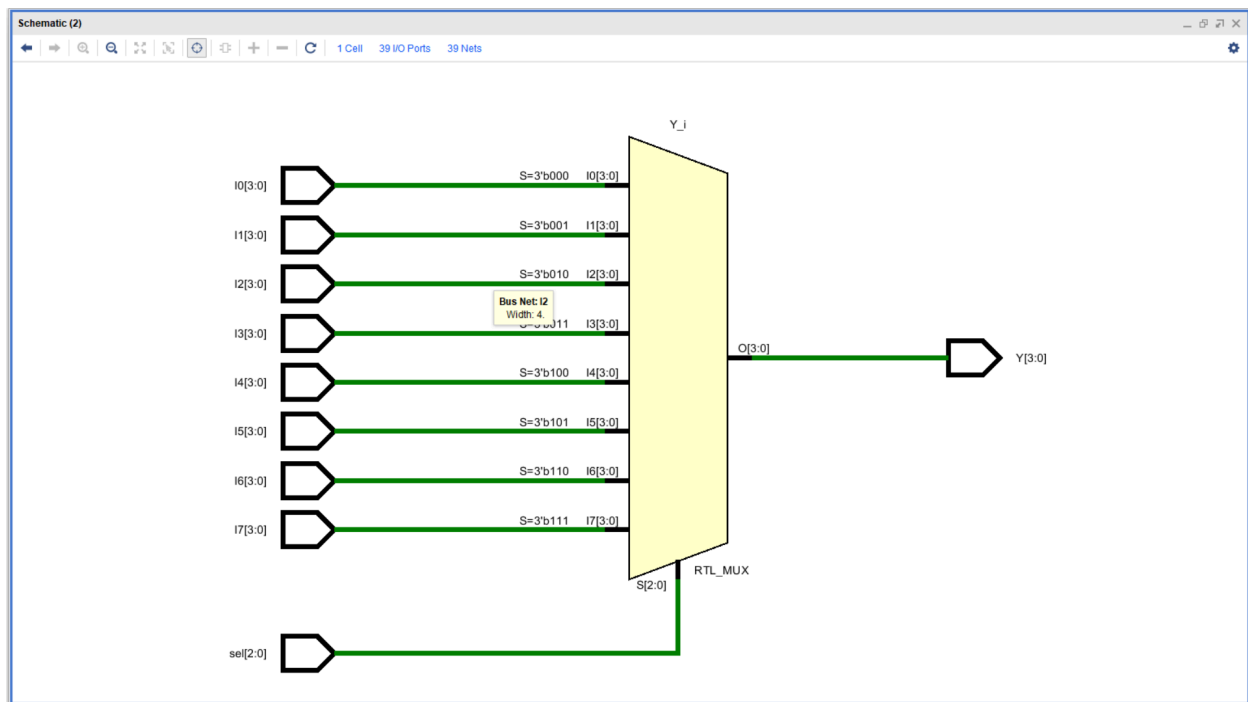
```

case sel is
  when "000" => Y <= I0;
  when "001" => Y <= I1;
  when "010" => Y <= I2;
  when "011" => Y <= I3;
  when "100" => Y <= I4;
  when "101" => Y <= I5;
  when "110" => Y <= I6;
  when "111" => Y <= I7;
  when others => Y <= (others => '0');
end case;
end process;

```

```
end Behavioral;
```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux8_4bit_tb is
--  Port ( );
end mux8_4bit_tb;

architecture Behavioral of mux8_4bit_tb is

component mux8_4bit
    port(
        sel : in std_logic_vector(2 downto 0);
        I0, I1, I2, I3, I4, I5, I6, I7 : in std_logic_vector(3
downto 0);
        Y : out std_logic_vector(3 downto 0)
    );
end component;

    signal sel : std_logic_vector(2 downto 0) := "000";
    signal I0, I1, I2, I3, I4, I5, I6, I7 : std_logic_vector(3 downto
0);
    signal Y : std_logic_vector(3 downto 0);

begin

    uut: mux8_4bit
        port map (
            sel => sel,
            I0  => I0,
            I1  => I1,
            I2  => I2,
            I3  => I3,
            I4  => I4,
            I5  => I5,
            I6  => I6,
            I7  => I7,
            Y   => Y
        );

    stim_proc: process
    begin
        I0 <= "0001"; I1 <= "0010"; I2 <= "0011"; I3 <= "0100";
        I4 <= "0101"; I5 <= "0110"; I6 <= "0111"; I7 <= "1000";
    
```

```

wait for 10 ns;

sel <= "000"; wait for 10 ns;
sel <= "001"; wait for 10 ns;
sel <= "010"; wait for 10 ns;
sel <= "011"; wait for 10 ns;
sel <= "100"; wait for 10 ns;
sel <= "101"; wait for 10 ns;
sel <= "110"; wait for 10 ns;
sel <= "111"; wait for 10 ns;

-- index No 240297 - 111 010 101 010 101 001

I0 <= "1110"; I1 <= "1101"; I2 <= "1010"; I3 <= "0101";
I4 <= "1010"; I5 <= "1001"; I6 <= "1111"; I7 <= "0001";
wait for 10 ns;

sel <= "111"; wait for 10 ns;
sel <= "010"; wait for 10 ns;
sel <= "101"; wait for 10 ns;
sel <= "001"; wait for 10 ns;

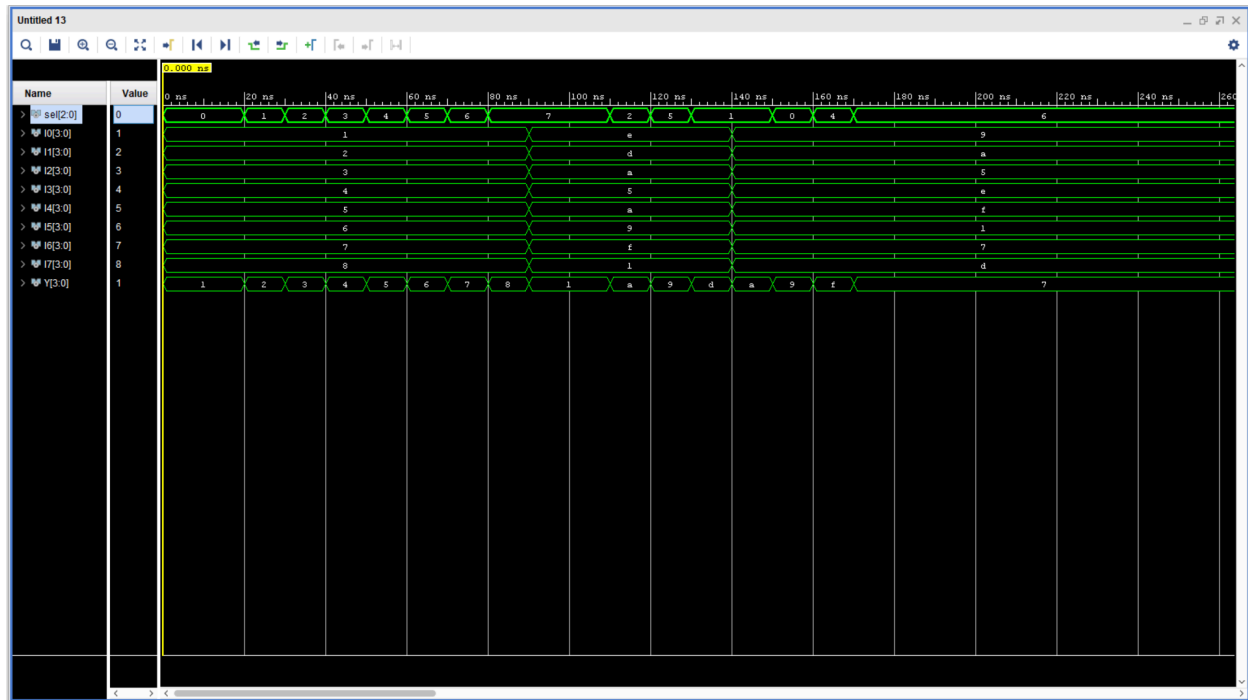
I0 <= "1001"; I1 <= "1010"; I2 <= "0101"; I3 <= "1110";
I4 <= "1111"; I5 <= "0001"; I6 <= "0111"; I7 <= "1101";
wait for 10 ns;

sel <= "000"; wait for 10 ns;
sel <= "100"; wait for 10 ns;
sel <= "110"; wait for 10 ns;

wait;
end process;
end Behavioral;

```

## Timing Diagram



## 7.8. Multiplexer (2 Way 4-bit)

### Design Source Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux2_4bit is
    Port ( sel : in STD_LOGIC;
          A : in STD_LOGIC_VECTOR (3 downto 0);
          B : in STD_LOGIC_VECTOR (3 downto 0);
          Y : out STD_LOGIC_VECTOR (3 downto 0));
end mux2_4bit;

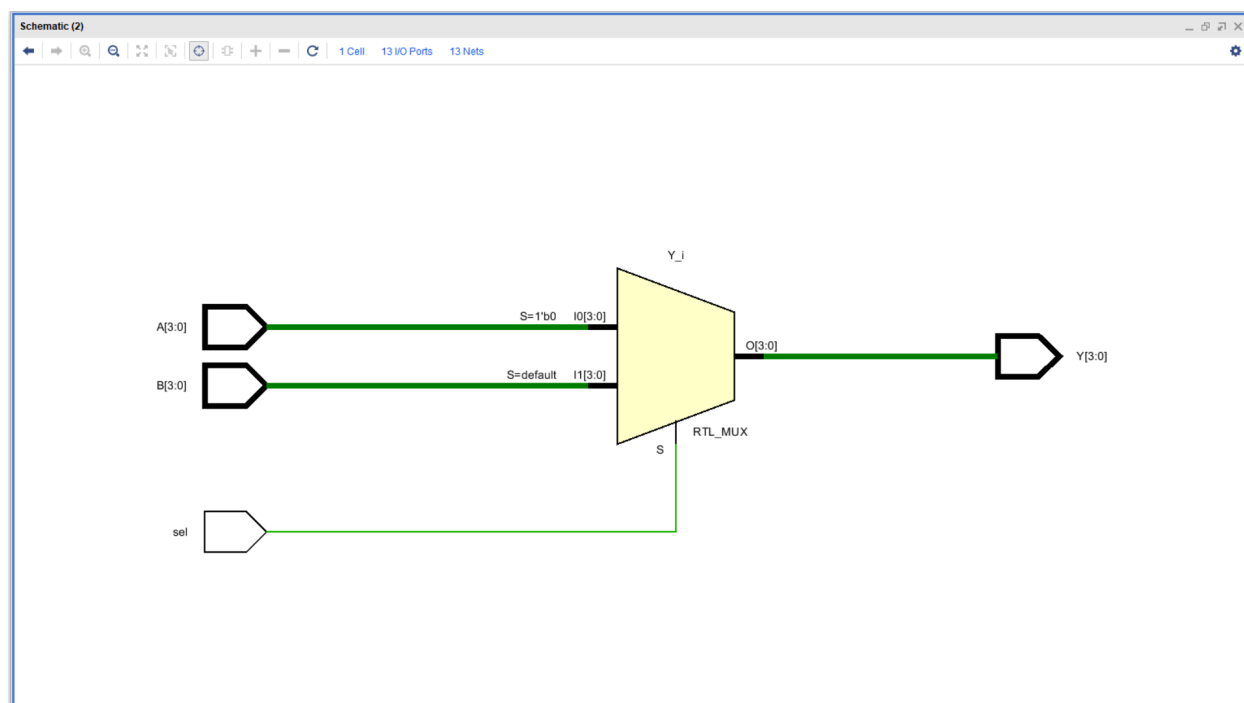
architecture Behavioral of mux2_4bit is

begin

    Y <= A when sel = '0' else B;

end Behavioral;
```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux2_4bit_tb is
-- Port ( );
end mux2_4bit_tb;

architecture Behavioral of mux2_4bit_tb is

component mux2_4bit
    Port ( sel : in  STD_LOGIC;
          A   : in  STD_LOGIC_VECTOR (3 downto 0);
          B   : in  STD_LOGIC_VECTOR (3 downto 0);
          Y   : out STD_LOGIC_VECTOR (3 downto 0));
end component;

signal sel : STD_LOGIC := '0';
signal A   : STD_LOGIC_VECTOR(3 downto 0) := "0000";
signal B   : STD_LOGIC_VECTOR(3 downto 0) := "0000";
signal Y   : STD_LOGIC_VECTOR(3 downto 0);

```

```

begin

 uut: mux2_4bit
   port map (
     sel => sel,
     A   => A,
     B   => B,
     Y   => Y
   );

 stim_proc: process
 begin
   sel <= '0'; A <= "1010"; B <= "0101"; wait for 10 ns;
   sel <= '1'; wait for 10 ns;
   sel <= '0'; wait for 10 ns;
   A <= "1111"; B <= "0000"; wait for 10 ns;
   sel <= '1'; wait for 10 ns;

   -- index No 240297 - 111 010 101 010 101 001

   A <= "1110"; B <= "1101"; sel <= '0'; wait for 10 ns;
   sel <= '1'; wait for 10 ns;

   A <= "1010"; B <= "0101"; sel <= '0'; wait for 10 ns;
   sel <= '1'; wait for 10 ns;

   A <= "1001"; B <= "1010"; sel <= '0'; wait for 10 ns;
   sel <= '1'; wait for 10 ns;

   A <= "1001"; B <= "0001"; sel <= '0'; wait for 10 ns;
   sel <= '1'; wait for 10 ns;

   wait;
 end process;

end Behavioral;

```





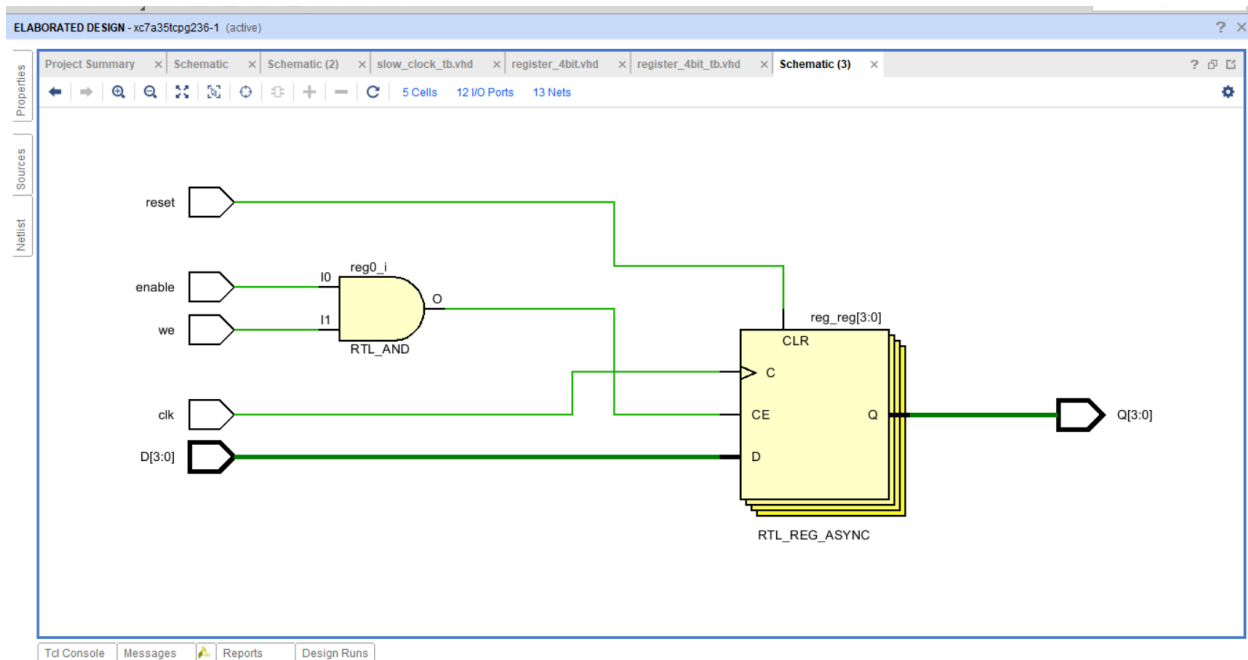
```

begin
    if reset = '1' then
        reg <= "0000";
    elsif rising_edge(clk) then
        if (enable = '1' and we = '1') then
            reg <= D;
        end if;
    end if;
end process;
Q <= reg;

end Behavioral;

```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity register_4bit_tb is
    -- Port ( );
end register_4bit_tb;

```

architecture Behavioral of register\_4bit\_tb is  
 component register\_4bit

```

    Port ( clk : in STD_LOGIC;
           reset : in STD_LOGIC;
           enable : in STD_LOGIC;
           we : in STD_LOGIC;
           D : in STD_LOGIC_VECTOR (3 downto 0);
           Q : out STD_LOGIC_VECTOR (3 downto 0));
  end component;
```

```

signal clk      : STD_LOGIC := '0';
signal reset    : STD_LOGIC := '0';
signal enable   : STD_LOGIC := '0';
signal we       : STD_LOGIC := '0';
signal D        : STD_LOGIC_VECTOR(3 downto 0) := "0000";
signal Q        : STD_LOGIC_VECTOR(3 downto 0);
```

begin

```

  uut: register_4bit
    port map (
      clk      => clk,
      reset    => reset,
      enable   => enable,
      we       => we,
      D        => D,
      Q        => Q
    );
```

```

  clk_process : process
  begin
    while true loop
      clk <= '0'; wait for 10 ns;
      clk <= '1'; wait for 10 ns;
    end loop;
  end process;
```

```

  stim_proc: process
  begin
    --index no : 240312A - 111 010 101 010 111 000
                      -- 0011 1010 1010 1011 1000
```

```

    reset <= '1'; enable <= '0'; we <= '0'; D <= "1000"; wait for
25 ns;
    reset <= '0'; wait for 10 ns;
```

```

enable <= '1'; we <= '1'; D <= "1011"; wait for 20 ns;
we <= '0'; D <= "1010"; wait for 20 ns;
we <= '1'; D <= "0011"; wait for 20 ns;
enable <= '0'; we <= '1'; D <= "1000"; wait for 20 ns;

reset <= '1'; wait for 15 ns;
reset <= '0'; wait for 15 ns;

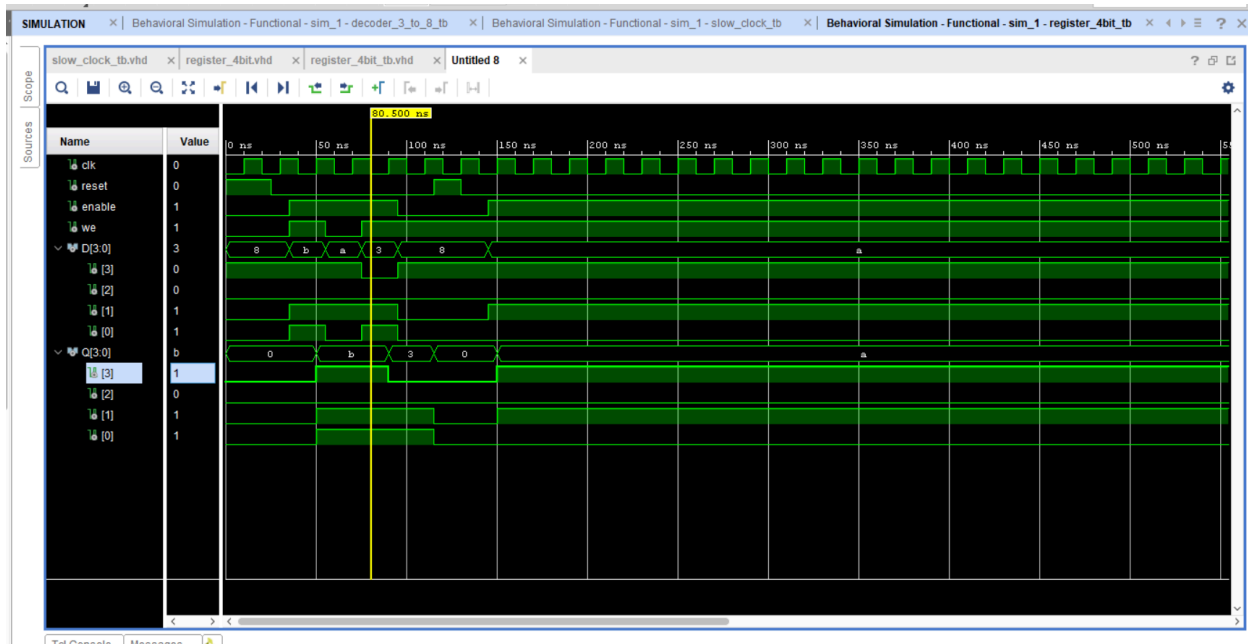
enable <= '1'; we <= '1'; D <= "1010"; wait for 20 ns;

wait;
end process;

end Behavioral;

```

## Timing Diagram



## 7.10.PC Register (3-bit)

### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity pc_register is
    Port ( clk : in STD_LOGIC;
          enable : in STD_LOGIC;
          reset : in STD_LOGIC;
          D : in STD_LOGIC_VECTOR (2 downto 0);
          Q : out STD_LOGIC_VECTOR (2 downto 0));
end pc_register;

architecture Behavioral of pc_register is
    signal reg : std_logic_vector(2 downto 0);

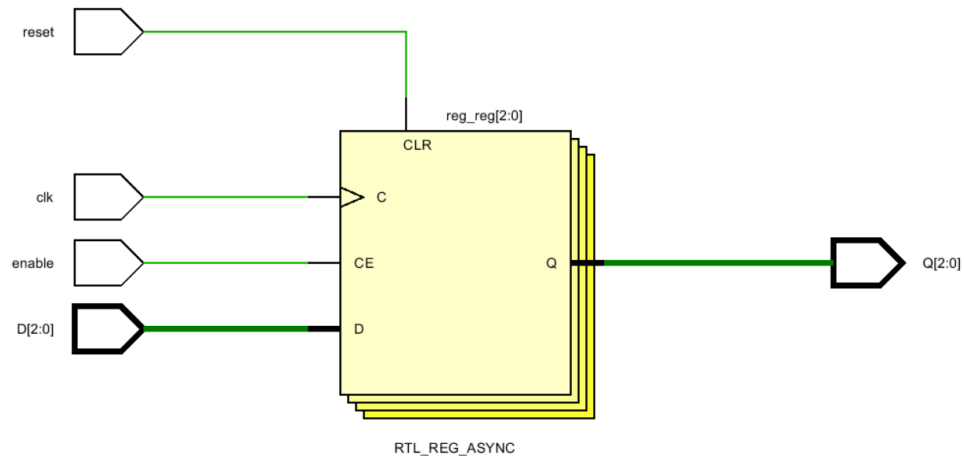
begin

    process(clk, reset)
    begin
        if reset = '1' then
            reg <= "000";
        elsif rising_edge(clk) then
            if enable='1' then
                reg <= D;
            end if;
        end if;
    end process;
    Q <= reg;

end Behavioral;

```

## Elaborated Design Schematic



## Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity pc_register_tb is

end pc_register_tb;

architecture Behavioral of pc_register_tb is

    component pc_register
        Port (
            clk      : in STD_LOGIC;
            enable   : in STD_LOGIC;
            reset    : in STD_LOGIC;
            D        : in STD_LOGIC_VECTOR (2 downto 0);
            Q        : out STD_LOGIC_VECTOR (2 downto 0)
        );
    end component;
end architecture Behavioral;

```

```

signal clk_tb      : STD_LOGIC := '0';
signal enable_tb   : STD_LOGIC := '0';
signal reset_tb    : STD_LOGIC := '0';
signal D_tb        : STD_LOGIC_VECTOR (2 downto 0) := (others =>
'0');
signal Q_tb        : STD_LOGIC_VECTOR (2 downto 0);

-- Clock period definition (100 MHz clock for standard timing)
constant CLK_PERIOD : time := 10 ns;

begin

    uut: pc_register
        port map (
            clk      => clk_tb,
            enable   => enable_tb,
            reset    => reset_tb,
            D        => D_tb,
            Q        => Q_tb
        );

    -- Clock Generation Process
    clk_process : process
    begin
        while now < 200 ns loop -- Limit simulation time to 200 ns
            clk_tb <= '0';
            wait for CLK_PERIOD / 2;
            clk_tb <= '1';
            wait for CLK_PERIOD / 2;
        end loop;
        wait;
    end process;

    stim_proc: process
    begin
        -- index no: 240297E - 111 010 101 010 101 001
        reset_tb <= '1';
        wait for 15 ns;
        reset_tb <= '0';
        wait for 10 ns; -- Q should be "000" here

        D_tb <= "001";
        enable_tb <= '0';
        wait for CLK_PERIOD; -- Q should still remain "000"
    end process;

```

edge

```
enable_tb <= '1';
wait for CLK_PERIOD; -- Q should update to "101" on the rising
```

```
D_tb <= "101";
wait for CLK_PERIOD; -- Q should update to "011"
```

```
enable_tb <= '0';
D_tb <= "010";
wait for CLK_PERIOD * 2;
```

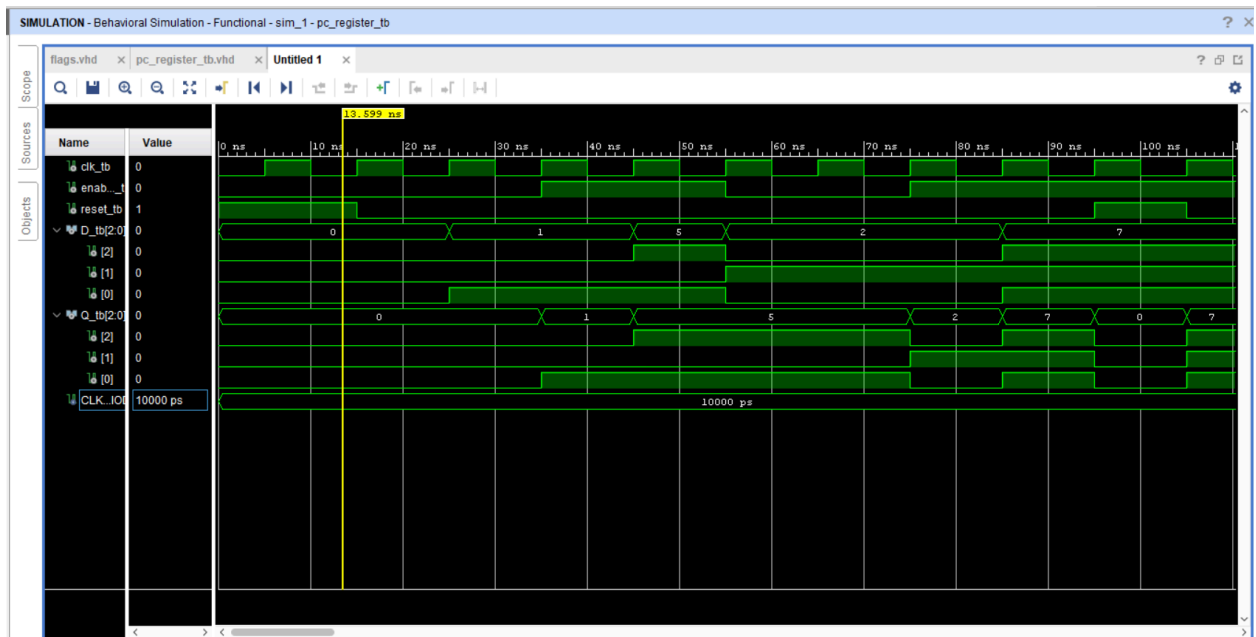
```
enable_tb <= '1';
wait for CLK_PERIOD;
```

```
D_tb <= "111";
wait for CLK_PERIOD;
```

```
reset_tb <= '1';
wait for 10 ns; -- Q should instantly drop back to "000"
reset_tb <= '0';
```

```
wait;
end process;
end Behavioral;
```

## Timing Diagram





## 7.11. Flags

| Item                  | Optimized Version | Extended Version             | Change Type       |
|-----------------------|-------------------|------------------------------|-------------------|
| Zero flag source      | From ADD_SUB_4bit | From top_alu                 | Changed           |
| Overflow flag         | From ADD_SUB_4bit | Only valid for ADD/SUB       | Same but rerouted |
| GT/EQ/LT flags        | Not present       | From comparator module       | New flags         |
| Flag update condition | Every ADD/SUB     | Conditional based on alu_top | Changed logic     |

### Optimized Version

#### Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity flags is
Port (
    clk      : in  STD_LOGIC;
    enable    : in  STD_LOGIC;
    reset     : in  STD_LOGIC;
    flag_en   : in  STD_LOGIC;

    alu_zero  : in  STD_LOGIC;
    alu_overflow : in STD_LOGIC;

    Z         : out STD_LOGIC;
    V         : out STD_LOGIC );
end flags;

architecture Behavioral of flags is

    signal Z_reg : std_logic := '0';
    signal V_reg : std_logic := '0';
begin
    process(clk, reset)

```

```

begin
    if reset = '1' then
        Z_reg <= '0';
        V_reg <= '0';

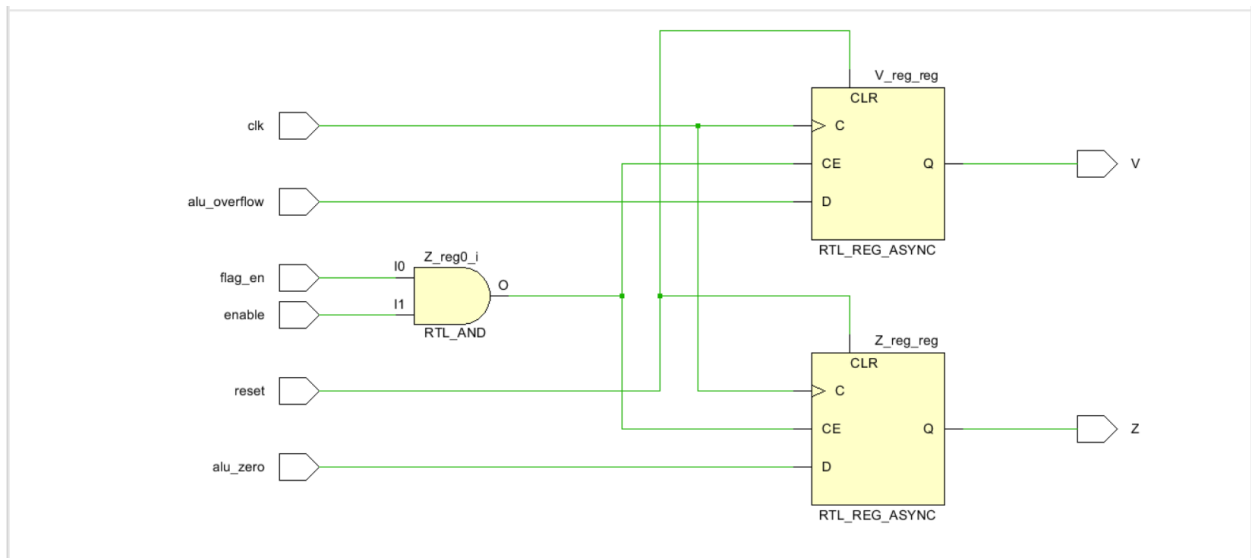
    elsif rising_edge(clk) then
        if (flag_en = '1' and enable = '1') then
            Z_reg <= alu_zero;
            V_reg <= alu_overflow;
        end if;
    end if;
end process;

Z <= Z_reg;
V <= V_reg;

end Behavioral;

```

## Elaborated Design Schematic



## **Extended Version**

## Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity flags is
Port (
    clk          : in  STD_LOGIC;
    enable       : in  STD_LOGIC;
    reset        : in  STD_LOGIC;
    flag_en      : in  STD_LOGIC;
    cmp_en       : in  STD_LOGIC;
    alu_zero     : in  STD_LOGIC;
    alu_overflow : in  STD_LOGIC;
    alu_GT       : in  STD_LOGIC;
    alu_EQ       : in  STD_LOGIC;
    alu_LT       : in  STD_LOGIC;
    Z            : out STD_LOGIC;
    V            : out STD_LOGIC;
    GT           : out STD_LOGIC;
    EQ           : out STD_LOGIC;
    LT           : out STD_LOGIC );
end flags;
architecture Behavioral of flags is

    signal Z_reg : std_logic := '0';
    signal V_reg : std_logic := '0';
    signal GT_reg : std_logic := '0';
    signal EQ_reg : std_logic := '0';
    signal LT_reg : std_logic := '0';

begin
    process(clk, reset)
    begin
        if reset = '1' then
            Z_reg <= '0';
            V_reg <= '0';
            GT_reg <= '0';
            EQ_reg <= '0';
            LT_reg <= '0';

        elsif rising_edge(clk) then
            if enable = '1' then
                if flag_en = '1' then
                    Z_reg <= alu_zero;
                    V_reg <= alu_overflow;
                    GT_reg <= '0';
                    EQ_reg <= '0';
                    LT_reg <= '0';
                elsif cmp_en = '1' then

```

```

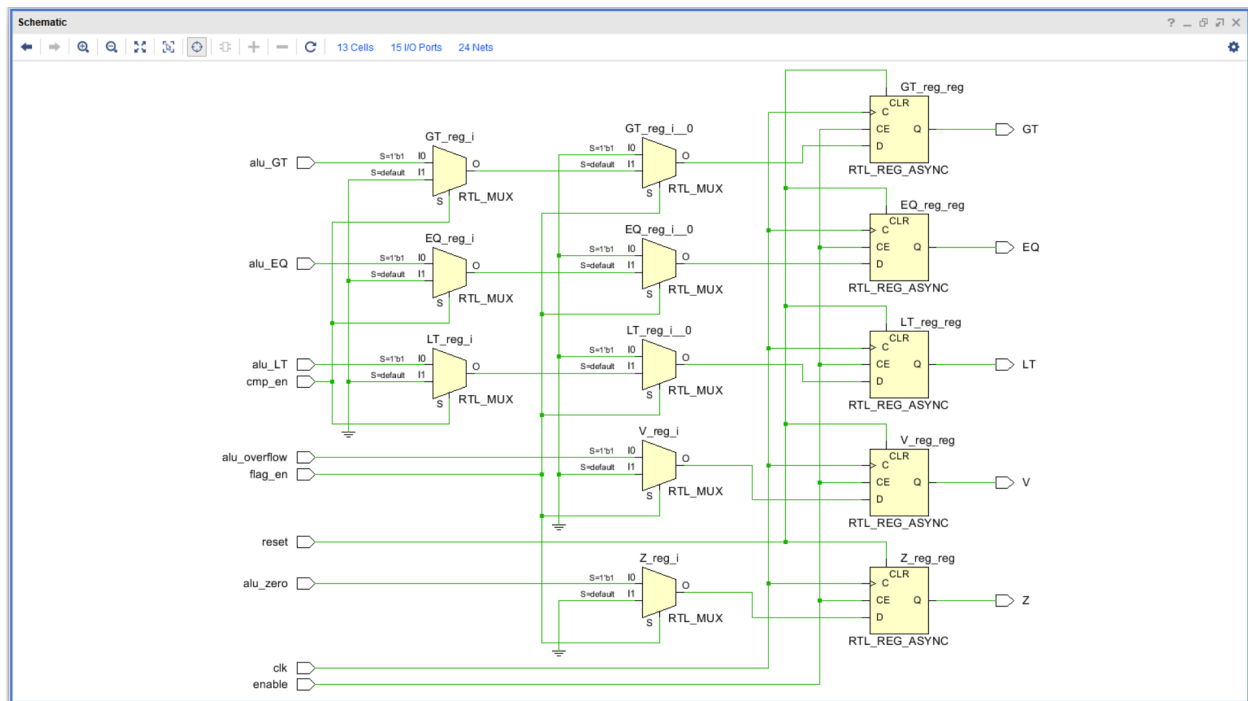
        GT_reg <= alu_GT;
        EQ_reg <= alu_EQ;
        LT_reg <= alu_LT;
        Z_reg <= '0';
        V_reg <= '0';
    else
        Z_reg <= '0';
        V_reg <= '0';
        GT_reg <= '0';
        EQ_reg <= '0';
        LT_reg <= '0';
    end if;

end if;
end if;
end process;

Z <= Z_reg;
V <= V_reg;
GT <= GT_reg;
EQ <= EQ_reg;
LT <= LT_reg;
end Behavioral;

```

## Elaborated Design Schematic



## 8.Constraints File

### Optimized Version

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports Clk_100MHz]
    set_property IOSTANDARD LVCMOS33 [get_ports Clk_100MHz]
    create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5}
[get_ports Clk_100MHz]

## LEDs
set_property PACKAGE_PIN U16 [get_ports {led[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
set_property PACKAGE_PIN E19 [get_ports {led[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
set_property PACKAGE_PIN U19 [get_ports {led[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
set_property PACKAGE_PIN V19 [get_ports {led[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]

set_property PACKAGE_PIN P1 [get_ports {led[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
set_property PACKAGE_PIN L1 [get_ports {led[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]

##7 segment display
set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]

set_property PACKAGE_PIN U2 [get_ports {an[0]}]
```

```

        set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]

##Buttons
set_property PACKAGE_PIN U18 [get_ports btnC]
        set_property IOSTANDARD LVCMOS33 [get_ports btnC]

```

## Extended Version

```

## Clock signal
set_property PACKAGE_PIN W5 [get_ports Clk_100MHz]
        set_property IOSTANDARD LVCMOS33 [get_ports Clk_100MHz]
        create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5}
[get_ports Clk_100MHz]

## Switches
set_property PACKAGE_PIN V17 [get_ports {sw[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {sw[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
set_property PACKAGE_PIN W16 [get_ports {sw[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
set_property PACKAGE_PIN W17 [get_ports {sw[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
set_property PACKAGE_PIN W15 [get_ports {sw[4]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
set_property PACKAGE_PIN V15 [get_ports {sw[5]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
set_property PACKAGE_PIN W14 [get_ports {sw[6]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
set_property PACKAGE_PIN W13 [get_ports {sw[7]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
set_property PACKAGE_PIN V2 [get_ports {sw[8]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[8]}]
set_property PACKAGE_PIN T3 [get_ports {sw[9]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[9]}]
set_property PACKAGE_PIN T2 [get_ports {sw[10]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {sw[10]}]
set_property PACKAGE_PIN R3 [get_ports {sw[11]}]

```

```

    set_property IOSTANDARD LVCMOS33 [get_ports {sw[11]]}
set_property PACKAGE_PIN W2 [get_ports {sw[12]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {sw[12]]}
set_property PACKAGE_PIN U1 [get_ports {sw[13]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {sw[13]]}
set_property PACKAGE_PIN T1 [get_ports {sw[14]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {sw[14]]}
set_property PACKAGE_PIN R2 [get_ports {sw[15]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {sw[15]]}

```

## ## LEDs

```

set_property PACKAGE_PIN U16 [get_ports {led[0]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {led[0]]}
set_property PACKAGE_PIN E19 [get_ports {led[1]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {led[1]]}
set_property PACKAGE_PIN U19 [get_ports {led[2]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {led[2]]}
set_property PACKAGE_PIN V19 [get_ports {led[3]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {led[3]]}
set_property PACKAGE_PIN U3 [get_ports {led[8]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {led[8]]}
set_property PACKAGE_PIN P3 [get_ports {led[7]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {led[7]]}
set_property PACKAGE_PIN N3 [get_ports {led[6]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {led[6]]}
set_property PACKAGE_PIN P1 [get_ports {led[4]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {led[4]]}
set_property PACKAGE_PIN L1 [get_ports {led[5]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {led[5]]}

```

## ##7 segment display

```

set_property PACKAGE_PIN W7 [get_ports {seg[0]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]]}
set_property PACKAGE_PIN W6 [get_ports {seg[1]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]]}
set_property PACKAGE_PIN U8 [get_ports {seg[2]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]]}
set_property PACKAGE_PIN V8 [get_ports {seg[3]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]]}
set_property PACKAGE_PIN U5 [get_ports {seg[4]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]]}
set_property PACKAGE_PIN V5 [get_ports {seg[5]]}
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]]}

```

```

set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]

set_property PACKAGE_PIN U2 [get_ports {an[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]

##Buttons
set_property PACKAGE_PIN U18 [get_ports btnC]
    set_property IOSTANDARD LVCMOS33 [get_ports btnC]
set_property PACKAGE_PIN T18 [get_ports btnU]
    set_property IOSTANDARD LVCMOS33 [get_ports btnU]

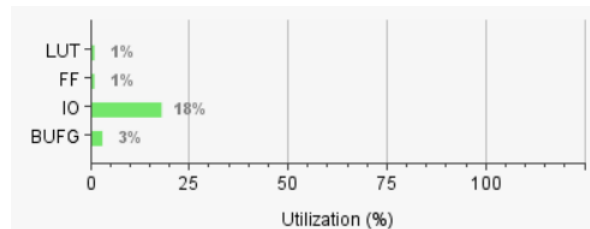
```

## 9.Resource Utilization

### Optimized Version

| Name                              | Slice LUTs<br>(20800) | Slice Registers<br>(41600) | Bonded IOB<br>(106) | BUFGCTRL<br>(32) |
|-----------------------------------|-----------------------|----------------------------|---------------------|------------------|
| ▼ <b>N</b> system_top             | 28                    | 57                         | 19                  | 1                |
| ▼ <b>I</b> U_CPU (Nanoprocess...  | 26                    | 40                         | 0                   | 0                |
| > <b>I</b> u_regbank (register... | 11                    | 8                          | 0                   | 0                |
| > <b>I</b> u_pc (pc)              | 6                     | 3                          | 0                   | 0                |
| <b>I</b> u_flags (flags)          | 0                     | 2                          | 0                   | 0                |
| <b>I</b> slow_clk (slow_cloc...   | 9                     | 27                         | 0                   | 0                |

| Resource | Estimation | Available | Utilization % |
|----------|------------|-----------|---------------|
| LUT      | 28         | 20800     | 0.13          |
| FF       | 57         | 41600     | 0.14          |
| IO       | 19         | 106       | 17.92         |
| BUFG     | 1          | 32        | 3.13          |

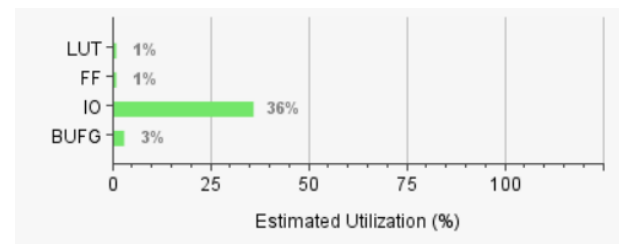




## Extended Version

| Name                              | Slice LUTs<br>(20800) | Slice Registers<br>(41600) | F7<br>Muxes<br>(16300) | Bonded IOB<br>(106) | BUFGCTRL<br>(32) |
|-----------------------------------|-----------------------|----------------------------|------------------------|---------------------|------------------|
| ▼ <b>N</b> system_top             | 100                   | 100                        | 11                     | 38                  | 1                |
| ▼ <b>I</b> U_CPU (Nanoprocess...  | 98                    | 83                         | 11                     | 0                   | 0                |
| <b>I</b> slow_clk (slow_cloc...   | 8                     | 27                         | 0                      | 0                   | 0                |
| <b>I</b> u_flags (flags)          | 0                     | 5                          | 0                      | 0                   | 0                |
| <b>I</b> u_inst_sel (inst_sel...  | 35                    | 14                         | 0                      | 0                   | 0                |
| > <b>I</b> u_pc (pc)              | 12                    | 3                          | 0                      | 0                   | 0                |
| > <b>I</b> u_regbank (register... | 41                    | 32                         | 11                     | 0                   | 0                |
| <b>I</b> u_step (step_button)     | 2                     | 2                          | 0                      | 0                   | 0                |

| Resource | Estimation | Available | Utilization % |
|----------|------------|-----------|---------------|
| LUT      | 100        | 20800     | 0.48          |
| FF       | 100        | 41600     | 0.24          |
| IO       | 38         | 106       | 35.85         |
| BUFG     | 1          | 32        | 3.13          |



## 10.Conclusion

In this lab, we successfully designed and built a 4-bit nanoprocessor that can execute a specific set of instructions. By developing two versions - one optimized for resource efficiency and one extended for enhanced functionality - we showed the ability to balance hardware limits with computational power. The project helped us understand important microprocessor components, including the 4-bit arithmetic unit, instruction decoder and program counter. We verified all components through simulations and successfully tested them on the Basys 3 development board. Additionally, working as a team gave us valuable experience in coordinating complex digital designs, integrating different sub-modules and sharing technical tasks.

## 11.Team Contribution

| Name                     | Contribution                                           | Hours spent |
|--------------------------|--------------------------------------------------------|-------------|
| Jayathilake P. I . K.    | ALU, Seven Segment, Top Module, Presentation, Report   | 24 hours    |
| Jayawardana Y. G. L. S.  | Program ROM, Instruction Decoder, Presentation, Report | 22 hours    |
| Jayawardhana J. A. R. P. | Program Counter, Multiplexers, Report                  | 20 hours    |
| Kalpashaka N. G. L. G.   | Register Bank, Decoder, Report                         | 20 hours    |